ĐẠI HỌC DUY TÂN
TRƯỜNG KHOA HỌC MÁY TÍNH & TRÍ TUỆ NHÂN TẠO
KHOA KHOA HỌC MÁY TÍNH

BÁO CÁO TỔNG KẾT
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN

TÊN ĐỀ TÀI:

"CHATBOT KHAI THÁC DRAMA KẾT HỢP DASHBOARD THỐNG KÊ VÀ PHÂN TÍCH"

Thuộc nhóm ngành khoa học : **Khoa học dữ liệu**

Giảng viên hướng dẫn : **T.S Lê Thanh Long**

Nhóm sinh viên thực hiện : **Nguyễn Đăng Thịnh**

Trương Đức Quốc Huy

Nguyễn Bá Thiện

Lê Thành Quang

Đà Nẵng, tháng 05/2026

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 7

1.1. Vấn đề tồn tại 7

1.2. Tổng quan các nghiên cứu trước đây và Khoảng trống nghiên cứu 8

1.2.1. Nhóm các hệ thống Social Listening (Lắng nghe mạng xã hội) thương mại 8

1.2.2. Nhóm nghiên cứu về Phân tích cảm xúc (Sentiment Analysis) tiếng Việt 8

1.2.3. Sự cấp thiết của "Drama Intelligence System" 9

1.3. Mục tiêu của dự án và giải pháp 9

1.3.1. Mục tiêu tổng quát 9

1.3.2. Mục tiêu cụ thể 10

1.3.3. Lý do lựa chọn Facebook làm nguồn dữ liệu chính 10

1.4. Phạm vi nghiên cứu 12

1.4.1. Phạm vi lĩnh vực nghiên cứu 12

1.4.2. Phạm vi thời gian và nguồn dữ liệu 12

1.5. Cơ sở lý thuyết và Kiến trúc hệ thống tổng thể 13

1.5.1. Kiến trúc 3 lớp (3-tier Architecture) và API-first 13

1.5.2. Nền tảng điều phối công việc (n8n) 16

1.5.2.1. Khái niệm n8n 16

1.5.2.2. Ứng dụng n8n vào dự án này 17

1.5.3. Chatbot và mô hình ngôn ngữ lớn (LLM) 18

1.5.3.1. Định nghĩa chatbot và LLM 18

1.5.3.2. Phân loại các loại Chatbot 18

1.5.3.3. Ứng dụng của nó vào đề tài 19

1.5.4. Kiến trúc RAG (Retrieval-Augmented Generation) 20

1.5.4.1. Khái niệm chung của RAG (Retrieval-Augmented Generation) 20

1.5.4.2. Nguyên lý hoạt động của RAG (Retrieval-Augmented Generation) .20
1.5.4.2.1. Retrieval (Truy xuất) 20
1.5.4.2.2. Augmented (Tăng cường).....20
1.5.4.2.3. Generation (Sinh phản hồi).....20
1.5.4.3. Ưu điểm của RAG so với LLM thuần.....21
1.5.5. Kiến trúc Multi-Agentic RAG (Multi-Agentic Retrieval-Augmented Generation).....22
1.5.5.1. Khái niệm và Nguyên lý hoạt động của Multi-Agentic RAG.....22
1.5.5.2. Nguyên lý hoạt động của Agentic RAG.....22
1.5.5.3. Ưu điểm của Agentic RAG so với RAG thuần 23
1.5.6. Vector Database và Cosine-Search.....24
1.5.6.1. Khái niệm Vector Embedding.....24
1.5.6.2. Khái niệm Cosine-Search (Độ tương đồng Cosine).....24
1.5.6.3. Supabase Vector Store (Cơ sở dữ liệu Vector Supabase) 25
CHƯƠNG 2. THIẾT KẾ HỆ THỐNG VÀ DỮ LIỆU 26
2.1. Dữ liệu và Đường ống xử lý tự động (Data Pipeline).....26
2.1.1. Tổng quan, đặc tả và Kiến trúc của Dữ liệu 26
2.1.2. Tổng quan quy trình và luồng xử lý dữ liệu 29
2.1.2.1. Tổng quan.....30
2.1.2.2. Pipeline thu thập dữ liệu tự động bằng n8n.....32
2.1.2.2.1. Khởi tạo, Sinh truy vấn và Chuẩn bị liên kết (URL Preparation) 33
2.1.2.2.2. Cào và Tổng hợp dữ liệu báo chí (Web Content Pipeline).....34
2.1.2.2.3. Luồng khai thác dữ liệu bình luận Mạng xã hội (Facebook Scraping Pipeline) 35
2.1.3. Chuẩn hóa, tiền xử lý dữ liệu và phân tích chuyên sâu cho AI (Tại Local) 37

2.1.3.1. Chuẩn hóa dữ liệu....37
2.1.3.1.1. Phân tích sự sai lệch cấu trúc (Old Schema vs. New Schema).37
2.1.3.1.2. Quy trình xử lý và chuẩn hóa dữ liệu cũ....38
2.1.3.2. Tiền xử lý dữ liệu....39
2.1.3.2.1. Tổng hợp và gộp dữ liệu (merge_data.py)....39
2.1.3.2.2. Sắp xếp lại thứ tự và lọc nhiễu (tienxuly_vietlaiorder.py)....40
2.1.3.2.3. Kỹ thuật tiền lọc Rule-based và Xử lý nhiễu cơ học....42
2.1.3.2.4. Pipeline tiền xử lý song song bằng Apache PySpark....44
2.1.3.2.5. Đóng gói bộ lọc rule-based thành Spark UDF cho kiến trúc phân tán....45
2.1.3.3. Phân tích dữ liệu....47
2.1.3.3.1. Chiến lược xử lý hai bước (Two-step Processing)....47
2.1.3.3.2. Cơ chế gọi API và xử lý lỗi....50
2.1.3.3.3. Đầu ra phân tích — Hai cấp độ kết quả....51
2.1.3.3.4. Sự khác biệt giữa Education và Showbiz....52
2.1.3.4. Hệ thống nhãn phân loại dữ liệu (Data Labeling System)....53
2.1.4. Phương pháp Trích xuất Từ khóa và Phân tích Cụm từ (Bag of Words & PMI)....54
2.1.4.1. Tiền xử lý và Chuẩn hóa từ vựng (Preprocessing & Stopwords)....54
2.1.4.2. Khởi tạo N-grams bằng Cửa sổ trượt (Sliding Window)....56
2.1.4.3. Thuật toán Pointwise Mutual Information (PMI)....57
2.1.4.4. Đếm tần suất và Công thức xếp hạng (Scoring & Ranking)....57
2.1.4.5. Phân tích kết quả trích xuất đặc trưng hai lĩnh vực....58
2.1.4.6. Cấu hình --strip-names cho pipeline PySpark....58
2.1.4.7. Sơ đồ kiến trúc hai pipeline song song....59
2.1.5. Tổng hợp dữ liệu....59

2.2. Xây dựng cơ sở dữ liệu Embedding và AI Agent 61
2.2.1. Xây dựng cơ sở dữ liệu Embeddings 61
2.2.2. RAG + Reranking 64
CHƯƠNG 3. TRIỂN KHAI VÀ TÍCH HỢP HỆ THỐNG 67
3.1. Agent n8n 67
3.1.1. Hệ thống Multi-Agent 67
3.1.2. Cơ chế chống Ảo giác (Anti-Hallucination) và Data Loop bằng Firecrawl 74
3.2. Triển khai Website tin tức tổng hợp và Hệ thống Chatbot 75
3.2.1.Triển khai website 75
3.2.1.1. Xây dựng Frontend và Backend 75
3.2.1.2. Kiến trúc hệ thống 75
3.2.1.2.1. Sơ Đồ Cấu Trúc 75
3.2.1.2.2. Luồng Dữ Liệu 76
3.2.1.3. Các công nghệ sử dụng 76
3.2.1.3.1. Core Dependencies 76
3.2.1.3.2. Dev Dependencies 76
3.2.1.3.3. Triển khai Apps Script 77
3.2.1.3.3.1. Giới thiệu 77
3.2.1.3.3.2. Mục tiêu 77
3.2.1.3.3.3. Kiến trúc hệ thống 77
3.2.1.3.3.4. Cấu trúc dữ liệu 77
3.2.1.3.3.5. Xây dựng API bằng Apps Script 78
3.2.1.3.3.6. Kết nối với Frontend (React) 78
3.2.2. Tích hợp Chatbot 79
3.3. Trực quan hóa Dữ liệu và Phân tích Insight trên Dashboard (AI Studio) 80

3.3.1. Trực quan hóa Dữ liệu thô (Raw Data) 80
3.3.2. Trực quan hóa Dữ liệu phân tích (Processed Data) 82
3.3.3. Trực quan hóa Dữ liệu tương tác Chatbot (Interaction Data). 84
3.3.4. Trực quan hóa Dữ liệu Sự kiện và Từ khóa bằng Biểu đồ Bong bóng (Bubble Chart) 85
3.4. Triển khai môi trường thực nghiệm cho pipeline PySpark song song 86
3.4.1. Bảng cấu hình phần mềm 86
3.4.2. Lưu ý cài đặt trên hệ điều hành Windows 87
CHƯƠNG 4. TÍNH ỨNG DỤNG THỰC TẾ VÀ ĐÁNH GIÁ KẾT QUẢ 88
4.1. Tính ứng dụng thực tế 88
4.1.1. Cung cấp giải pháp tiếp nhận thông tin thông minh cho người dùng cá nhân đại chúng 88
4.1.2. Hỗ trợ cơ quan quản lý và nhà hoạch định chính sách (Đặc thù trong lĩnh vực Giáo dục) 89
4.1.3. Ứng dụng trong kiểm duyệt nội dung và quản trị khủng hoảng truyền thông (Lĩnh vực Giải trí/Showbiz) 90
4.2. Website Tin Tức và Fanpage vệ tinh 91
4.2.1. Website Báo chí Tổng hợp (Public Web). 91
4.2.2. Fanpage vệ tinh "Hóng chuyện" 92
4.3. Dashboard Phân tích Dư luận (AI Studio) trong thực tiễn 93
4.3.1. Trực quan hóa kết quả phân tích 93
4.3.2. Case Study (Nghiên cứu tình huống): 93
4.4. Đánh giá thực nghiệm 95
4.4.1. Đánh giá năng lực thu thập và xử lý AI 95
4.4.2. Phương pháp xây dựng tập dữ liệu kiểm thử (Ground Truth Dataset) 98
4.4.3. Kiểm chứng đối chiếu pipeline tiền xử lý song song 101
4.4.3.1. Đối chiếu số bình luận sạch 101

4.4.3.2. Đối chiếu trích xuất từ khóa PMI top 10 102
4.4.4. Đánh giá hiệu năng và khả năng mở rộng quy mô pipeline tiền xử lý 104
4.4.4.1. Bảng benchmark trên cùng tập 18.245 bình luận 104
4.4.4.2. Bảng benchmark mở rộng quy mô — 90.000 bình luận giả lập 104
4.4.4.3. Phân tích chiều hướng tăng trưởng 105
4.4.4.4. Phép ngoại suy lên 1.000.000 bình luận 105
4.4.4.5. Tiêu chí lựa chọn giữa Spark và Python đơn luồng 106
4.5. Đánh giá hiệu năng và Chiến lược Tối ưu hóa Token (Token Optimization & Model Routing) 106
4.5.1. Ưu điểm nổi bật 107
4.5.2. Hạn chế còn tồn tại 108
4.6. Định hướng phát triển trong tương lai 110
4.6.1. Tinh chỉnh PhoBERT trên domain dữ liệu chuyên ngành 110
4.6.2. Mạng nơ-ron đồ thị (Graph Neural Network) cho phân tích thread thảo luận 110
4.6.3. Triển khai pipeline PySpark trên cluster thực 113
4.6.4. Tự động hóa pipeline và xử lý dữ liệu thời gian thực 113
4.6.5. Mở rộng nguồn dữ liệu sang TikTok và YouTube 113
TÀI LIỆU THAM KHẢO 114

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Vấn đề tồn tại

Trong bối cảnh xã hội hiện đại, khối lượng thông tin và các sự kiện phát sinh ngày càng gia tăng. Nhiều sự kiện thu hút sự chú ý rộng rãi của công chúng, bao gồm các vấn đề trong lĩnh vực giải trí và các thông tin mới từ Bộ Giáo dục và Đào tạo. Điều này dẫn đến sự xuất hiện liên tục của các luồng thông tin mới, kéo theo đó là sự đa dạng về quan điểm và ý kiến từ dư luận xã hội.

Đặc biệt, trên nền tảng Facebook — một mạng xã hội có giao diện thân thiện, dễ tiếp cận và cho phép người dùng tương tác thông qua bình luận — đã thu hút một lượng lớn người sử dụng. Theo báo cáo *Digital 2026: Vietnam* của We Are Social và Meltwater, tính đến đầu năm 2024, Việt Nam có 85.6 triệu người dùng mạng Internet, tương đương 84.2% tổng dân số. Riêng Facebook có khả năng tiếp cận tới 79 triệu người dùng tại Việt Nam, trở thành nền tảng định hình luồng thông tin và xu hướng dư luận lớn nhất cả nước [1]. Sự phổ biến này biến Facebook trở thành một kho dữ liệu mở dồi dào, mang lại tiềm năng to lớn cho việc khai thác, theo dõi và phân tích các biến động của dư luận xã hội theo thời gian thực.

Xuất phát từ nhu cầu cung cấp thông tin kịp thời, đồng thời với nhu cầu cần phải phân tích và tìm hiểu hành vi người dùng dành cho các doanh nghiệp hay chính phủ nhà nước. Nghiên cứu này được thực hiện nhằm xây dựng một hệ thống tích hợp phục vụ việc thu thập, xử lý và phân tích dữ liệu liên quan đến các sự kiện nổi bật.

Về vấn đề của nghiên cứu này, dữ liệu để phục vụ cho dự án tồn tại tình trạng phân mảnh thông tin nghiêm trọng. Trong khi báo chí truyền thống chủ yếu cung cấp thông tin mang tính bối cảnh và tĩnh, thì các nền tảng mạng xã hội như Facebook lại phản ánh các phản ứng đa chiều, phi tuyến tính và chứa nhiều nhiễu từ cộng đồng. Sự thiếu hụt các công cụ tích hợp và đồng bộ hóa dữ liệu giữa hai nguồn này gây khó khăn đáng kể trong việc đánh giá một cách toàn diện bức tranh dư luận xã hội.

1.2. Tổng quan các nghiên cứu trước đây và Khoảng trống nghiên cứu

Việc khai thác và phân tích dữ liệu mạng xã hội đã thu hút sự chú ý của nhiều nhà nghiên cứu và doanh nghiệp. Tuy nhiên, qua quá trình tổng quan tài liệu, đề tài nhận thấy các hệ thống và phương pháp hiện tại vẫn tồn tại những hạn chế nhất định, được chia thành hai nhóm chính:

1.2.1. Nhóm các hệ thống Social Listening (Lắng nghe mạng xã hội) thương mại

Tại Việt Nam, các nền tảng Social Listening thương mại như Buzzmetrics, YouNet Media, hay Kompa [2], [3], [4] đang chiếm lĩnh thị trường. Các hệ thống này sở hữu cơ sở hạ tầng thu thập dữ liệu (crawling) từ Facebook rất mạnh mẽ, phục vụ chủ yếu cho mục đích đo lường sức khỏe thương hiệu (Brand Health) và chiến dịch tiếp thị của doanh nghiệp.

Mặc dù có ưu thế về lượng dữ liệu, các hệ thống này chủ yếu tập trung vào thống kê định lượng (ví dụ: đếm số lượng bình luận, đo lường tổng lượt tương tác, hoặc thống kê nhân khẩu học). Việc ứng dụng AI trong các nền tảng này phần lớn dừng lại ở các thuật toán thế hệ cũ để vẽ biểu đồ phân loại cảm xúc (Tích cực/Tiêu cực/Trung lập). Khoảng trống cốt lõi là các hệ thống này thiếu khả năng phân tích và sinh văn bản tự nhiên (Generative AI). Chúng không thể tự động đọc hàng ngàn bình luận để viết ra một bản tóm tắt diễn biến sự kiện . Hơn nữa, kiến trúc của các nền tảng này là tương tác một chiều (dashboard-based), hoàn toàn vắng bóng cơ chế Hỏi-Đáp (Conversational UI) cho phép người dùng chat trực tiếp với dữ liệu để truy vấn sâu vào các khía cạnh cụ thể của một sự kiện.

1.2.2. Nhóm nghiên cứu về Phân tích cảm xúc (Sentiment Analysis) tiếng Việt

Trong giới học thuật, bài toán Phân tích cảm xúc đối với tiếng Việt đã được giải quyết khá hiệu quả nhờ các thuật toán Học máy truyền thống (như Support Vector Machine - SVM, Naive Bayes) và gần đây là các mô hình Học sâu (Deep Learning). Tiêu biểu nhất là nghiên cứu của Nguyen và cộng sự với việc ra mắt PhoBERT — mô hình ngôn ngữ dựa trên kiến trúc Transformer được huấn luyện trước (pre-trained) dành riêng cho tiếng Việt [5]. Ứng dụng PhoBERT trong các bài toán dán nhãn cảm xúc (Aspect-Based Sentiment Analysis) mang lại độ chính xác rất cao.

Hạn chế cốt lõi của các nghiên cứu và mô hình này (bao gồm cả PhoBERT, SVM) là chúng thuộc nhóm Mô hình Phân loại (Discriminative Models). Chúng làm rất tốt nhiệm vụ nhận diện một câu bình luận là "Chửi bới" hay "Khen ngợi" (dán nhãn - Labeling), nhưng hoàn toàn không có khả năng sinh ngôn ngữ (Generative). Nghĩa là, các mô hình này không thể nhóm các bình luận lại, hiểu bối cảnh và tự động sinh ra một văn bản giải thích lý do tại sao dư luận lại phẫn nộ.

1.2.3. Sự cấp thiết của "Drama Intelligence System"

Từ hai khoảng trống trên, đề tài "Drama Intelligence System" được đề xuất nhằm lấp đầy sự thiếu hụt về mặt công nghệ và trải nghiệm người dùng. Thay vì chỉ dừng lại ở việc dán nhãn hay thống kê số liệu, hệ thống tích hợp sâu Kiến trúc Truy xuất nguồn lực (RAG - Retrieval-Augmented Generation) kết hợp cùng Mô hình Ngôn ngữ Lớn (LLMs). Sự kết hợp này mang lại hai giá trị quang trọn mà các công trình trước chưa làm được:

1. Tổng hợp ngữ nghĩa: Khả năng tự động đúc kết hàng ngàn bình luận nhiễu loạn thành một bản tóm tắt sự kiện mạch lạc bằng ngôn ngữ tự nhiên.

2. Tương tác chủ động: Biến kho dữ liệu tĩnh thành một trợ lý ảo, cho phép người dùng đặt câu hỏi trực tiếp về sự kiện (Ví dụ: *"Tại sao nhân vật A lại bị cộng đồng mạng chỉ trích?"*) và nhận được câu trả lời chính xác, được trích xuất trực tiếp từ các bằng chứng (bình luận) có thật trên mạng xã hội [5].

1.3. Mục tiêu của dự án và giải pháp

1.3.1. Mục tiêu tổng quát

Nghiên cứu đề xuất xây dựng “Drama Intelligence System” – một hệ sinh thái AI–Data chuyên dụng nhằm thu thập, tích hợp và phân tích dữ liệu dư luận từ nhiều nguồn, phục vụ việc cung cấp, theo dõi và khai thác thông tin về các sự kiện nổi bật trong hai lĩnh vực trọng tâm là Giáo dục và Giải trí (Showbiz).

Hệ thống được xây dựng nhằm lấp đầy khoảng trống về mặt công nghệ mà các nghiên cứu trước đây để lại. Về mặt kiến trúc, "Drama Intelligence System" tích hợp Kiến trúc Truy xuất Nguồn lực (RAG) kết hợp Mô hình Ngôn ngữ Lớn (LLMs) theo mô hình đa tác nhân (Multi-Agent), được triển khai trên nền tảng điều phối n8n và cơ sở dữ liệu vector Supabase.

1.3.2. Mục tiêu cụ thể

Thứ nhất, hỗ trợ nhóm người dùng không theo dõi thường xuyên các tin tức, sự kiện và không lướt mạng xã hội nhiều, có nhu cầu tìm hiểu về một thông tin gì đó thông qua hệ thống AI thông minh. Hệ thống cung cấp khả năng truy vấn và tổng hợp các sự kiện đang được quan tâm (đặc biệt trong lĩnh vực giải trí và xã hội), bao gồm nội dung sự kiện, mức độ phổ biến, các nhân vật có liên quan đến sự kiện đó và mức độ thảo luận của cộng đồng.

Thứ hai, hỗ trợ các nhà quản lý, doanh nghiệp hoặc cơ quan nhà nước trong việc đánh giá phản ứng dư luận đối với một chính sách, thông tư hoặc sự kiện cụ thể. Hệ thống cho phép phân tích số lượng chủ đề liên quan, các vấn đề phát sinh, cũng như phân loại ý kiến người dân theo các nhóm như đồng thuận, phản đối hoặc trung lập.

1.3.3. Lý do lựa chọn Facebook làm nguồn dữ liệu chính

Sự phát triển mạnh mẽ của hạ tầng kỹ thuật số tại Việt Nam đã thúc đẩy sự bùng nổ của các nền tảng mạng xã hội, biến chúng thành không gian sinh hoạt trực tuyến và nơi trao đổi thông tin chính của người dân. Trong khuôn khổ dự án "Drama Intelligence System", việc lựa chọn mạng xã hội làm nguồn cung cấp dữ liệu đầu vào là một quyết định mang tính chiến lược. Dựa trên số liệu thống kê mới nhất từ báo cáo Digital 2026: Vietnam *(DataReportal, 2026)* , Facebook và Zalo hiện đang là hai nền tảng thống trị thị trường Việt Nam.

Tiêu chí	Zalo	YouTube	Facebook
Quy mô người dùng (Triệu người)	78,3	62,1	79,0
Tỷ lệ so với tổng dân số (%)	77,0%	61,0%	77,6%
Phân bổ giới tính (Nữ / Nam)	48,8% / 51,2%	48,7% / 51,3%	50.2%/ 48.1%
Tăng trưởng so với cùng kỳ năm trước	+ 0,9% (700 nghìn)	- 0,3% (200 nghìn)	+ 7,2% (5,30 triệu)

Bảng 1.1. So sánh quy mô người dùng giữa các nền tảng mạng xã hội lớn tại Việt Nam (Nguồn: DataReportal, "Digital 2026: Vietnam", trích xuất dữ liệu quảng cáo của Meta, VNG và Google) [1]

Qua bảng số liệu trên, mặc dù Zalo giữ vị thế là ứng dụng nhắn tin (OTT) phổ biến nhất với 78,3 triệu người dùng đang hoạt động hàng tháng (Monthly Active Users), Facebook vẫn tiếp tục duy trì vị trí là nền tảng mạng xã hội có quy mô lớn nhất Việt Nam. Cụ thể, dữ liệu từ công cụ quảng cáo của Meta ghi nhận Facebook có 79,0 triệu người dùng vào cuối năm 2025, tương đương 77,6% tổng dân số [1]. Đặc biệt, mức tăng trưởng của Facebook rất ấn tượng khi thu hút thêm 5,30 triệu người dùng (+7,2%) chỉ trong vòng một năm, cho thấy sức sống và độ bao phủ không ngừng mở rộng của nền tảng này .

Việc quyết định chọn Facebook làm nguồn thu thập dữ liệu chính (Data Ingestion) cho dự án được lý giải bởi các nguyên nhân trọng yếu sau:

- Tính đại chúng và quy mô mẫu (Sample Size) lớn: Với 79 triệu người dùng, chiếm hơn 3/4 dân số, Facebook cung cấp một tập dữ liệu (dataset) khổng lồ, đa dạng về nhân khẩu học và khu vực địa lý. Quy mô này đảm bảo tính đại diện cao cho các phân tích dư luận xã hội.

- Giao diện tương tác và Tính năng thảo luận cộng đồng: Khác với Zalo (thiên về giao tiếp cá nhân, bảo mật nhóm kín) hay YouTube (thiên về tiêu thụ nội dung video một chiều), Facebook sở hữu kiến trúc giao diện tối ưu cho việc thảo luận mở. Thông qua cấu trúc bài đăng (Post) và chuỗi bình luận phân cấp (Comment/Reply), người dùng dễ dàng thể hiện quan điểm, cảm xúc và tranh luận trực tiếp. Sự tương tác này tạo ra một nguồn dữ liệu văn bản phi cấu trúc (unstructured text data) phong phú, phản ánh rõ nét thái độ (sentiment) và ý kiến của đám đông trước một sự kiện truyền thông cụ thể.

- Tính thời sự và lan truyền thông tin (Virality): Facebook là trung tâm phát tán thông tin (Information Hub) tức thời. Bất kỳ một sự kiện nào xảy ra trong xã hội đều được phản ánh, chia sẻ và mổ xẻ trên nền tảng này chỉ trong vài phút. Dữ liệu văn bản từ Facebook có tính bối cảnh cao, cung cấp cho dự án một cái nhìn thực tế và cập nhật nhất về các dòng chảy dư luận.

Để xây dựng một hệ sinh thái phân tích dư luận xã hội toàn diện, "Drama Intelligence System" cần xử lý một khối lượng lớn thông tin phi cấu trúc từ Internet. Chương này trình bày chi tiết về chiến lược thu thập, cấu trúc lược đồ (schema) và các thuật toán tiền xử lý, phân tích dữ liệu chuyên sâu được áp dụng trong hệ thống.

1.4. Phạm vi nghiên cứu

Để đảm bảo tính khả thi, tập trung tài nguyên hệ thống và tối ưu hóa độ sâu của các thuật toán phân tích dư luận xã hội, đề tài xác định phạm vi nghiên cứu cụ thể trên hai phương diện chính:

1.4.1. Phạm vi lĩnh vực nghiên cứu

Hệ thống "Drama Intelligence System" tập trung khoanh vùng thu thập và phân tích dữ liệu ở hai lĩnh vực trọng tâm là Giải trí (Showbiz) và Giáo dục

Quyết định lựa chọn này xuất phát từ các cơ sở thực tiễn sau:

- Lĩnh vực Giải trí (Showbiz): Đây là môi trường mạng tính đặc thù với tần suất phát sinh các sự kiện gây chú ý (khủng hoàng truyền thông) rất cao. Các sự kiện này luôn thu hút sự quan tâm mãnh liệt từ đại chúng, đặc biệt là nhóm nhân khẩu học thanh thiếu niên. Trong bối cảnh hiện tại, các nhân vật công chúng (nghệ sĩ, thần tượng) sở hữu mức độ phủ sóng rộng khắp và có tầm ảnh hưởng sâu sắc, khả năng định hướng trực tiếp đến hành vi cũng như cảm xúc của một bộ phận lớn người dân trong xã hội.

- Lĩnh vực Giáo dục: Trong giai đoạn gần đây, nhằm đáp ứng các tiêu chuẩn của cuộc Cách mạng công nghiệp 4.0 và định hướng phát triển trong kỷ nguyên vươn mình của đất nước, ngành giáo dục đã liên tục cập nhật, ban hành nhiều chính sách cải cách mới. Tuy nhiên, sự dịch chuyển và thay đổi với tốc độ nhanh chóng này đôi khi tạo ra những xáo trộn nhất định trong hệ thống giảng dạy, thi cử và quản lý. Những biến động này lập tức trở thành chủ đề nóng, làm nảy sinh nhiều luồng quan điểm đa chiều và thu hút sự thảo luận sôi nổi của dư luận cả nước, đặc biệt là từ nhóm đối tượng phụ huynh và học sinh bị ảnh hưởng trực tiếp.

1.4.2. Phạm vi thời gian và nguồn dữ liệu

Giới hạn thời gian thu thập: Để đảm bảo tính thời sự và khả năng phản ánh đúng thực trạng xã hội đương đại, toàn bộ dữ liệu (bao gồm nội dung sự kiện báo chí và bình luận mạng xã hội) được giới hạn thu thập nghiêm ngặt trong giai đoạn từ năm 2025 đến năm 2026.

1.5. Cơ sở lý thuyết và Kiến trúc hệ thống tổng thể

1.5.1. Kiến trúc 3 lớp (3-tier Architecture) và API-first

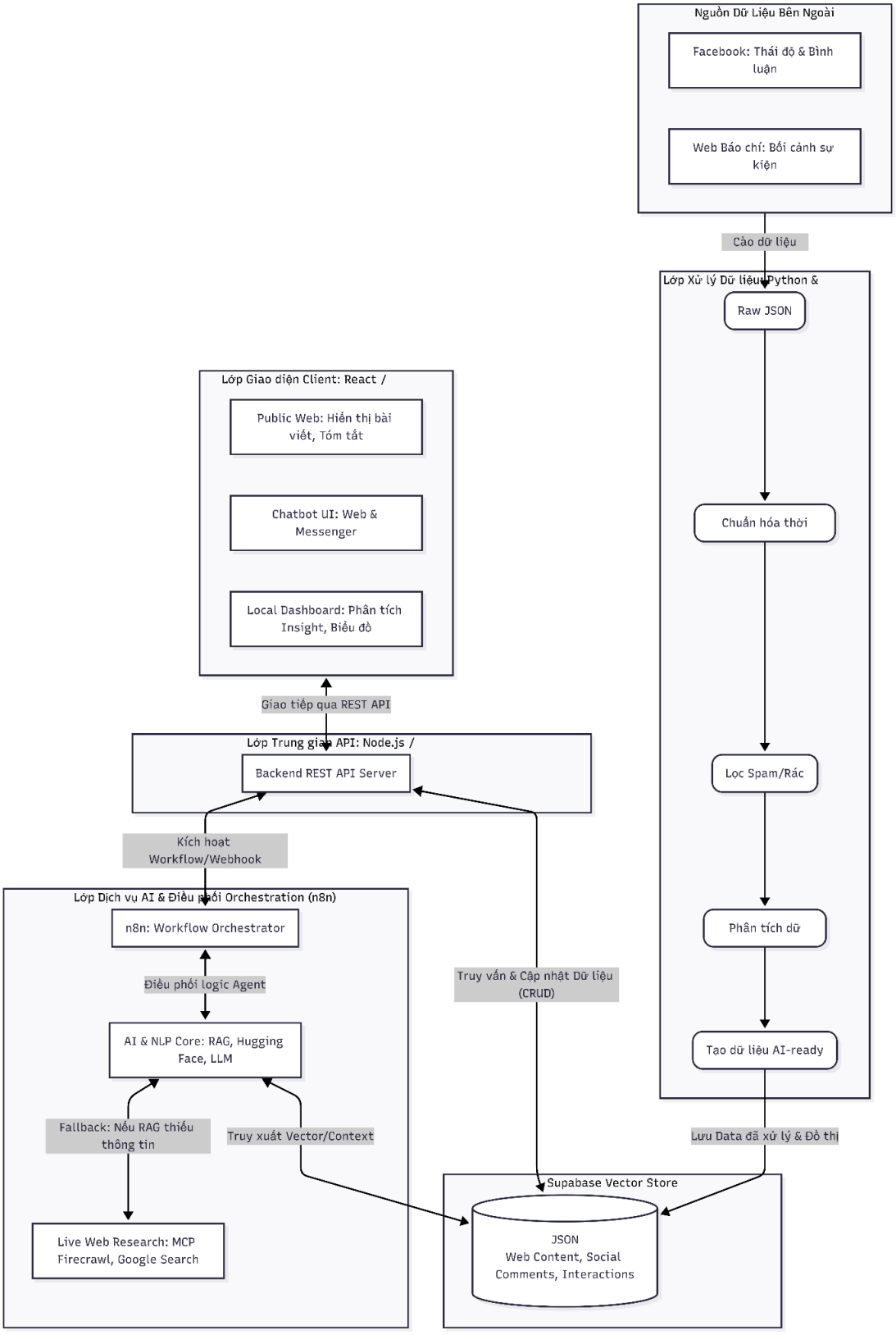
Kiến trúc 3 lớp (3-tier Architecture) là một mô hình thiết kế công nghệ phần mềm kinh điển, phân tách hệ thống thành ba tầng logic và vật lý độc lập bao gồm: Lớp Trình diễn (Presentation Tier), Lớp Ứng dụng (Application Tier) và Lớp Dữ liệu/Dịch vụ (Data/Service Tier). Đối với dự án "Drama Intelligence System", mô hình này được áp dụng song hành cùng nguyên lý thiết kế API-first (Ưu tiên giao diện lập trình ứng dụng). Sự kết hợp này mang ý nghĩa chiến lược, giúp tách biệt hoàn toàn phần giao diện tương tác của người dùng khỏi sự phức tạp của các luồng tính toán trí tuệ nhân tạo (AI) và RAG (Retrieval-Augmented Generation). Nhờ đó, hệ thống đạt được tính linh hoạt cao, cho phép phát triển, nâng cấp và bảo trì từng module một cách độc lập mà không gây ảnh hưởng chéo lên toàn bộ hệ thống.

Cụ thể, kiến trúc hệ thống được triển khai qua ba tầng như sau:

- Lớp Giao diện và Trực quan hóa (Frontend - Presentation Tier): Được phát triển dựa trên các nền tảng công nghệ hiện đại như React và Next.js (hoạt động theo cơ chế Single Page Application - SPA), lớp này đóng vai trò là điểm tiếp xúc trực tiếp với người dùng. Cấu trúc Frontend được chia thành ba phân hệ chính:
    - Public Web (Website báo chí) đảm nhiệm việc hiển thị các bài viết tin tức, tóm tắt sự kiện và các liên kết nguồn nổi bật.
    - Chatbot UI là giao diện hội thoại đa nền tảng, được tích hợp trực tiếp trên Web và Facebook Messenger nhằm tăng cường trải nghiệm hỏi-đáp thông minh theo thời gian thực.
    - Local Dashboard (AI Studio) đóng vai trò là bảng điều khiển quản trị, cung cấp các biểu đồ trực quan hóa số liệu thống kê, phân tích insight và theo dõi xu hướng biến động của dư luận.

- Lớp Trung gian (Backend API - Application Tier): Lớp này được xây dựng trên nền tảng môi trường Node.js kết hợp cùng framework Express.js. Khác với các hệ thống nguyên khối (Monolithic), Backend tại đây hoạt động đơn thuần như một REST API Server. Chức năng cốt lõi của nó là tiếp nhận yêu cầu từ Frontend, xử lý các logic nghiệp vụ cơ bản, giao tiếp với cơ sở dữ liệu lưu trữ (như MongoDB), và làm cầu nối định tuyến (route) để luân chuyển dữ liệu với hệ thống AI. Việc thiết kế lớp API trung gian giúp quản lý luồng dữ liệu I/O mượt mà, đảm bảo lớp giao diện luôn phản hồi nhanh chóng ngay cả khi các tác vụ AI ở phía sau đang xử lý lượng dữ liệu lớn.

- Lớp Dịch vụ Dữ liệu và AI (Data & AI Services Tier): Đây là thành phần xử lý trung tâm của toàn bộ hệ sinh thái , hoạt động độc lập ở một tầng hạ tầng riêng biệt. Lớp này chứa đựng các thành phần tính toán cốt lõi bao gồm: hệ thống RAG, các mô hình Xử lý ngôn ngữ tự nhiên (NLP với Hugging Face Transformers), các mô hình ngôn ngữ lớn (LLM), và các công cụ xử lý dữ liệu quy mô lớn (như PySpark). Điểm đặc biệt của tầng này là việc ứng dụng n8n làm công cụ điều phối luồng công việc (Workflow Orchestrator), thay thế hoàn toàn nhiệm vụ xử lý logic truyền thống của web server. Kiến trúc phân tách tuyệt đối này không chỉ cung cấp cho các mô hình AI một không gian tính toán biệt lập, tối ưu hóa tài nguyên phần cứng, mà còn tạo điều kiện thuận lợi để hệ thống dễ dàng mở rộng, module hóa và phục vụ hiệu quả cho các mục đích nghiên cứu học thuật chuyên sâu.



Hình 1.1. Tổng thể kiến trúc của dự án.

1.5.2. Nền tảng điều phối công việc (n8n)

1.5.2.1. Khái niệm n8n

n8n (đọc là "nodemation") là một nền tảng tự động hóa luồng công việc (workflow automation) mạnh mẽ, mã nguồn mở (hiện chuyển sang giấy phép fair-code/source-available). Khác với các công cụ đóng gói dạng hộp đen (black-box), n8n cung cấp một môi trường trực quan dựa trên các nút (node-based) cho phép các nhà phát triển kết nối bất kỳ ứng dụng, API hoặc dịch vụ cơ sở dữ liệu nào lại với nhau để tự động hóa các tác vụ phức tạp mà không cần hoặc cần rất ít mã lập trình (Low-code/No-code). [6]

Chức năng cốt lõi của n8n là tích hợp (Integration) và Điều phối (Orchestration). Hệ thống cung cấp hàng trăm node tích hợp sẵn cho các nền tảng phổ biến (như Google Workspace, Facebook, Github) và hỗ trợ node HTTP Request để gọi bất kỳ REST API nào. Nền tảng này cho phép thực hiện các thao tác xử lý dữ liệu phức tạp như phân nhánh điều kiện (If/Switch), vòng lặp (Loop), gom nhóm dữ liệu (Merge), và phân tích chuỗi JSON.

n8n hoạt động dựa trên luồng sự kiện (Event-driven). Một luồng (workflow) luôn bắt đầu bằng một Trigger (Nút kích hoạt - ví dụ: Webhook nhận tin nhắn, khoảng thời gian Cron-job định kỳ). Khi Trigger được kích hoạt, dữ liệu sẽ chảy qua các Action Nodes (Nút hành động) theo dạng chuỗi JSON. Tại mỗi node, dữ liệu sẽ được biến đổi, tính toán hoặc luân chuyển đến các hệ thống bên thứ ba trước khi trả về kết quả cuối cùng.

Sự phổ biến của n8n đến từ ba yếu tố cốt lõi. Thứ nhất, tính linh hoạt nhờ khả năng tự lưu trữ (Self-hosting), giúp các tổ chức bảo mật tuyệt đối dữ liệu nội bộ và không bị giới hạn số lượng tác vụ (tasks) như các nền tảng SaaS trả phí (như Zapier hay Make). Thứ hai, n8n cực kỳ thân thiện với lập trình viên (Developer-friendly) khi cho phép viết trực tiếp các đoạn mã JavaScript (Code Node) để can thiệp sâu vào luồng dữ liệu. Thứ ba, khả năng xử lý lỗi vượt trội với các cơ chế thử lại (Retry) và rẽ nhánh khi API thất bại.

1.5.2.2. Ứng dụng n8n vào dự án này

Trong các kiến trúc phần mềm truyền thống, Web Server (như Node.js, Python Django) thường phải gánh vác toàn bộ việc tiếp nhận yêu cầu (request) và xử lý logic luồng dữ liệu. Tuy nhiên, để đáp ứng tính phức tạp của dự án "Drama Intelligence System", hệ thống ứng dụng n8n đảm nhận vai trò Workflow Orchestrator (Điều phối luồng công việc) hoàn toàn độc lập với lớp giao diện. N8n hoạt động như một hệ thống điều phối trung tâm trung tâm, thay thế các đoạn mã lập trình luồng cứng nhắc để tự động hóa toàn bộ vòng đời của dữ liệu và hệ thống AI thông qua hai luồng nhiệm vụ chính:

- Tự động hóa đường ống thu thập dữ liệu (Data Pipeline): n8n chịu trách nhiệm lên lịch chạy ngầm (Cron-job) để quét dữ liệu chờ từ Google Sheets. Từ đó, hệ thống tự động sinh các truy vấn qua SerpApi, điều phối công cụ MCP Firecrawl để cào dữ liệu báo chí (Web Content) và gọi API của Apify để thu thập bình luận Facebook. Hệ thống cho phép n8n kiểm soát hoàn toàn các vấn đề kỹ thuật như: tự động xoay vòng tài khoản (Account Rotation) khi hết hạn mức API, áp dụng độ trễ (Wait node) để tránh bị máy chủ chặn, và lưu trữ trực tiếp file JSON kết quả lên Google Drive.

- Điều phối hệ thống Đa tác nhân (Multi-Agent) và Chatbot RAG: Khi người dùng nhắn tin, n8n đóng vai trò định tuyến (Router), phân tích ý định để chuyển câu hỏi tới các Tác nhân truy xuất (Retrieval Agent) của mảng Giáo dục, Showbiz hoặc mảng phân tích Insight. Đặc biệt, thông qua n8n, dự án đã xây dựng thành công cơ chế dự phòng chống ảo giác AI (Anti-Hallucination Fallback). Khi dữ liệu RAG nội bộ không đủ để đáp ứng câu hỏi, luồng n8n sẽ tự động rẽ nhánh (Node If) để kích hoạt cơ chế Live Web Research, gọi Firecrawl đi cào dữ liệu trực tiếp trên Internet, sau đó đưa cho LLM tổng hợp và trả lời người dùng.

1.5.3. Chatbot và mô hình ngôn ngữ lớn (LLM)

1.5.3.1. Định nghĩa chatbot và LLM

- Chatbot: Là một chương trình phần mềm được thiết kế để mô phỏng và tự động hóa các cuộc trò chuyện với con người thông qua văn bản (text) hoặc giọng nói (voice). Mục tiêu cốt lõi của chatbot là cung cấp giao diện tương tác tự nhiên (Conversational UI), giúp người dùng dễ dàng truy vấn thông tin hoặc thực hiện tác vụ mà không cần thông qua các thao tác điều hướng đồ họa truyền thống.

- Mô hình ngôn ngữ lớn (LLM - Large Language Models): Là các mô hình Trí tuệ Nhân tạo thế hệ mới được xây dựng trên nền tảng kiến trúc học sâu (Deep Learning), tiêu biểu là kiến trúc Transformer. LLM được huấn luyện trên một tập dữ liệu văn bản khổng lồ với hàng tỷ tham số, mang lại khả năng đọc hiểu ngữ cảnh, suy luận logic và sinh văn bản tự nhiên (Generative AI) với độ chính xác ở mức cao.

- Sự kết hợp giữa hai công nghệ này tạo ra một bước tiến lớn: LLM đóng vai trò là "bộ não" nhận thức, trong khi Chatbot đóng vai trò là "giao diện" giao tiếp, biến hệ thống thành công cụ hỗ trợ hỏi đáp trực quan.

1.5.3.2. Phân loại các loại Chatbot

Dựa trên cơ chế hoạt động và mức độ phức tạp của thuật toán xử lý, chatbot hiện nay được chia thành hai nhóm chính:

- Chatbot dựa trên tập luật (Rule-based Chatbot): Đây là thế hệ chatbot truyền thống, hoạt động theo các kịch bản (flow) đã được lập trình sẵn với cấu trúc điều kiện "if-else" cứng nhắc. Loại chatbot này chỉ có thể phản hồi các câu hỏi khớp chính xác với từ khóa đã được định nghĩa. Điểm yếu của chúng là không hiểu được bối cảnh thực tế và không có khả năng tự sinh văn bản mới.

- Chatbot dựa trên Trí tuệ Nhân tạo (AI-driven Chatbot / Generative Chatbot): Là hệ thống tiên tiến sử dụng Xử lý ngôn ngữ tự nhiên (NLP) và LLM để tự động phân tích ý định (intent) của người dùng. Thay vì trả lời theo mẫu có sẵn, chúng có khả năng nhóm các thông tin phân mảnh, hiểu bối cảnh sâu xa và tự động sinh ra một văn bản giải thích hoàn toàn mới, tự nhiên và linh hoạt.

1.5.3.3. Ứng dụng của nó vào đề tài

Trong dự án "Drama Intelligence System", hệ thống Chatbot không sử dụng các tập luật "if-else" truyền thống mà tích hợp sâu Kiến trúc Truy xuất Nguồn lực (Agentic RAG) kết hợp cùng LLM để mang lại khả năng tương tác chủ động và phân tích dữ liệu chuyên sâu.

Hệ thống tận dụng sức mạnh của các mô hình ngôn ngữ lớn thế hệ mới như Gemini 2.5 Flash, Llama-3.3, và Qwen để thực hiện các tác vụ phân tích văn bản phi cấu trúc phức tạp. Các mô hình này không chỉ đóng vai trò xử lý ngôn ngữ thuần túy mà còn thực hiện các nhiệm vụ chuyên sâu bao gồm:

- Tiền xử lý và làm sạch dữ liệu nhiễu: Ứng dụng mô hình Gemini để tổng hợp, "viết lại order" và hợp nhất hàng ngàn đoạn bình luận, bài báo rời rạc thành một văn bản bối cảnh thống nhất. LLM cũng đóng vai trò như một bộ lọc thông minh ở Bước 1 của luồng phân tích để loại bỏ các bình luận rác không mang giá trị.

- Đánh giá cảm xúc (Stance Detection): Hệ thống ứng dụng Llama-3.3-70B-Versatile để chạy song song (bất đồng bộ) nhằm đánh giá chuyên sâu thái độ của dư luận, phân loại chính xác 5 sắc thái cảm xúc (Phẫn nộ, Cà khịa, Đồng cảm, Ủng hộ, Trung lập), đồng thời nhận diện các bình luận độc hại, công kích cá nhân.

- Trích xuất đặc trưng và Nhận diện thực thể (NER): Các mô hình ngôn ngữ lớn đóng vai trò phân tích chuỗi văn bản để tự động trích xuất đặc trưng actor_related, giúp hệ thống xác định chính xác các nhân vật trọng tâm liên quan đến sự kiện (Ví dụ: tên cá nhân, tổ chức, trường học) mà không cần nhập liệu thủ công.

1.5.4. Kiến trúc RAG (Retrieval-Augmented Generation)

1.5.4.1. Khái niệm chung của RAG (Retrieval-Augmented Generation)

Kiến trúc Truy xuất nguồn lực (RAG) là một khung kỹ thuật được giới thiệu lần đầu bởi Lewis và cộng sự (2020) nhằm tối ưu hóa đầu ra của các Mô hình ngôn ngữ lớn (LLM). RAG kết hợp hai loại bộ nhớ: Bộ nhớ tham số (Parametric memory) là tri thức đã được học trong quá trình huấn luyện của LLM và Bộ nhớ phi tham số (Non-parametric memory) là các dữ liệu thực tế từ các nguồn bên ngoài như cơ sở dữ liệu vector hoặc tài liệu nội bộ. [7]

Thay vì chỉ dựa vào dữ liệu tĩnh, RAG cho phép mô hình truy cập trực tiếp vào kho tri thức động để trả lời câu hỏi, giúp giải quyết triệt để hạn chế về "điểm dừng dữ liệu" (knowledge cutoff) của các mô hình AI truyền thống.

1.5.4.2. Nguyên lý hoạt động của RAG (Retrieval-Augmented Generation)

1.5.4.2.1. Retrieval (Truy xuất)

Khi nhận được truy vấn từ người dùng, hệ thống không gửi trực tiếp câu hỏi tới LLM. Thay vào đó, câu hỏi được chuyển đổi thành các vector toán học (embeddings) thông qua một mô hình nhúng (ví dụ: Gemini embedding 002). Hệ thống sau đó thực hiện tìm kiếm ngữ nghĩa trong cơ sở dữ liệu vector (như Supabase Vector Store) để trích xuất ra các đoạn văn bản (chunks) có liên quan nhất đến chủ đề đang được hỏi.

1.5.4.2.2. Augmented (Tăng cường)

Dữ liệu thô vừa truy xuất được sẽ được làm sạch và chèn vào một khung mẫu câu lệnh (Prompt Template). Tại bước này, câu hỏi gốc của người dùng được "tăng cường" bằng cách bổ sung thêm bối cảnh (context) từ dữ liệu thực tế. Cấu trúc prompt lúc này sẽ bao gồm: "Dựa vào thông tin sau đây: [Dữ liệu truy xuất], hãy trả lời câu hỏi: [Câu hỏi của người dùng]".

1.5.4.2.3. Generation (Sinh phản hồi)

Cuối cùng, gói thông tin đã được tăng cường sẽ được gửi đến mô hình ngôn ngữ lớn (LLM). Nhờ có bằng chứng xác thực nằm trong context, LLM sẽ đóng vai trò như một chuyên gia đọc hiểu và tóm tắt để sinh ra câu trả lời chính xác, mạch lạc và có căn cứ, thay vì tự suy diễn dựa trên dữ liệu cũ. [8]

1.5.4.3. Ưu điểm của RAG so với LLM thuần

Việc áp dụng RAG mang lại những lợi thế đáng kể so với việc sử dụng các mô hình LLM độc lập (Standard LLM):

Tiêu chí đánh giá	LLM thuần túy (Standard LLM)	Kiến trúc RAG kết hợp LLM
1. Nguồn tri thức	Dựa hoàn toàn vào dữ liệu tĩnh được nạp trong quá trình huấn luyện (Parametric memory).	Kết hợp bộ nhớ nội tại và cơ sở dữ liệu bên ngoài có thể mở rộng (Non-parametric memory).
2. Hiện tượng "ảo giác" (Hallucination)	**Nguy cơ cao.** Mô hình dễ tự suy diễn và sinh ra thông tin sai lệch, bịa đặt khi đối mặt với câu hỏi nằm ngoài vùng kiến thức đã học.	**Thấp** hơn đáng kể so với LLM thuần túy**.** Câu trả lời bị ràng buộc nghiêm ngặt bởi các "bằng chứng" được truy xuất từ cơ sở dữ liệu nội bộ.
3. Tính thời sự (Up-to-date)	**Bị giới hạn** bởi "điểm dừng dữ liệu" (Knowledge cutoff). Không thể nắm bắt các sự kiện truyền thông mới phát sinh sau thời điểm huấn luyện.	**Liên tục cập nhật.** Có thể trả lời các sự kiện vừa diễn ra theo thời gian thực chỉ bằng cách nạp thêm văn bản vào cơ sở dữ liệu Vector.
4. Khả năng kiểm chứng (Auditability)	Dạng "hộp đen" (Black box). Người dùng không thể biết mô hình lấy thông tin từ đâu để đưa ra câu trả lời.	**Minh bạch.** Hệ thống có thể cung cấp chính xác nguồn trích dẫn (link bài viết, ID bình luận gốc) đã được dùng làm căn cứ.
5. Tối ưu chi phí và Tài nguyên	Tốn kém nguồn lực khổng lồ để cập nhật kiến thức mới (phải Fine-tuning hoặc huấn luyện lại mô hình từ đầu).	**Tiết kiệm đáng kể.** Cho phép sử dụng các LLM nhỏ gọn, tối ưu chi phí API nhưng vẫn đạt hiệu suất cao nhờ tận dụng kho dữ liệu bên ngoài.

Bảng 1.2. So sánh giữa LLM thuần và RAG [9].

1.5.5. Kiến trúc Multi-Agentic RAG (Multi-Agentic Retrieval-Augmented Generation)

Mặc dù kiến trúc RAG truyền thống giải quyết tốt vấn đề "ảo giác" của LLM, nhưng nó vẫn tồn tại điểm yếu về tính tuyến tính (chỉ truy xuất một lần rồi sinh kết quả). Để giải quyết các truy vấn phức tạp, hệ thống "Drama Intelligence System" ứng dụng một bản nâng cấp toàn diện hơn: Multi-Agentic RAG (RAG Đa tác nhân).

1.5.5.1. Khái niệm và Nguyên lý hoạt động của Multi-Agentic RAG

Agentic RAG là sự kết hợp giữa kiến trúc RAG và các Tác nhân Trí tuệ nhân tạo (AI Agents). Khác với RAG truyền thống hoạt động theo một đường ống cố định (Pipeline), Agentic RAG trao quyền tự chủ cho LLM. LLM lúc này đóng vai trò như một "bộ não" có khả năng lập kế hoạch, suy luận, tự quyết định sử dụng các công cụ tìm kiếm (Tools) và đánh giá thông tin trước khi trả lời [10].

Multi-Agentic RAG là bước phát triển cao hơn, trong đó hệ thống không chỉ có một Agent mà gồm nhiều Agent chuyên biệt phối hợp với nhau. Mỗi Agent đảm nhận một vai trò cụ thể (như Tác nhân Điều phối, Tác nhân Truy xuất dữ liệu Giáo dục, Tác nhân Truy xuất dữ liệu Giải trí) để chia nhỏ và giải quyết các tác vụ phức tạp một cách song song hoặc nối tiếp [11].

1.5.5.2. Nguyên lý hoạt động của Agentic RAG

Nguyên lý hoạt động của Agentic RAG thường dựa trên khung tư duy ReAct (Reasoning and Acting - Suy luận và Hành động) [12]. Thay vì truy xuất dữ liệu một cách thụ động, quá trình này diễn ra qua các bước lặp mang tính chủ động:

- Định tuyến và Lập kế hoạch (Routing & Planning): Khi nhận truy vấn, Tác nhân Điều phối (Dispatcher Agent) sẽ phân tích ý định của người dùng và quyết định xem cần giao nhiệm vụ cho Tác nhân nào (Ví dụ: Chuyển câu hỏi về học đường cho Agent Education).

- Sử dụng công cụ và Truy xuất lặp (Tool Use & Iterative Retrieval): Tác nhân Truy xuất (Retrieval Agent) tự động tạo ra các từ khóa tìm kiếm tối ưu. Nó sử dụng các công cụ (như Supabase Vector Store hoặc Live Web Search) để trích xuất dữ liệu. Đặc biệt, nếu kết quả trả về không đủ thông tin, Agent có khả năng tự thay đổi từ khóa và tìm kiếm lại (Iterative search).

- Đánh giá ngữ cảnh (Context Evaluation): Khác với RAG thuần lấy mọi thứ tìm được đưa vào LLM, Agentic RAG có bước tự đánh giá xem "Dữ liệu này đã đủ để trả lời câu hỏi chưa?". Nếu thiếu, hệ thống tự động gọi công cụ tìm kiếm trên Internet (như Firecrawl) để bổ sung.

- Sinh phản hồi (Generation): Dữ liệu sau khi được đánh giá và chọn lọc kỹ lưỡng mới được gửi tới LLM tổng hợp cuối cùng để sinh ra câu trả lời mạch lạc.

1.5.5.3. Ưu điểm của Agentic RAG so với RAG thuần

Sự chuyển dịch từ hệ thống RAG tuyến tính sang Multi-Agentic RAG mang lại những bước tiến lớn về khả năng xử lý lỗi và linh hoạt công cụ, được thể hiện chi tiết qua Bảng 1.3:

Tiêu chí đánh giá	RAG thuần túy (Standard RAG)	Agentic RAG / Multi-Agentic RAG
1. Luồng xử lý dữ liệu	Tuyến tính, tĩnh (Chỉ 1 chiều: User -> Truy xuất -> Trả lời).	Linh hoạt, đệ quy (Có thể lặp lại việc tìm kiếm nhiều lần cho đến khi đủ thông tin).
2. Khả năng truy vấn phức tạp (Multi-hop Query)	**Kém**. Dễ thất bại nếu câu hỏi yêu cầu tổng hợp thông tin từ nhiều nguồn khác nhau.	**Xuất sắc.** Các Agent có thể chia nhỏ câu hỏi, tìm kiếm từng phần rồi ghép lại.
3. Khả năng tương tác với Công cụ (Tool Use)	Chỉ truy cập được một Vector Database duy nhất được định sẵn.	Tương tác đa công cụ: Gọi API thời tiết, cào web trực tiếp (Firecrawl), tra cứu nhiều Database khác nhau.
4. Cơ chế tự sửa lỗi (Self-Correction)	**Không có**. Nếu tìm kiếm sai từ khóa ở bước đầu, kết quả cuối cùng sẽ hoàn toàn sai.	**Có.** Agent tự nhận biết kết quả tìm kiếm bị rỗng hoặc sai ngữ cảnh và tự động đổi từ khóa tìm kiếm khác.
5. Khả năng mở rộng (Scalability)	Khó bảo trì khi hệ thống lớn lên và có nhiều loại dữ liệu khác nhau.	**Rất dễ mở rộng.** Chỉ cần tạo thêm các Agent chuyên biệt cho từng loại dữ liệu (Giáo dục, Giải trí, Kinh tế...).

Bảng 1.3. So sánh kiến trúc RAG thuần túy và Agentic RAG.

1.5.6. Vector Database và Cosine-Search

Để kiến trúc RAG có thể truy xuất chính xác ngữ cảnh từ một kho dữ liệu khổng lồ, hệ thống cần một phương pháp để máy tính có thể "hiểu" và so sánh ý nghĩa của văn bản. Khung công nghệ này được xây dựng dựa trên ba thành phần cốt lõi: Vector Embedding, thuật toán Cosine-Search và Cơ sở dữ liệu Vector.

1.5.6.1. Khái niệm Vector Embedding

Máy tính không thể trực tiếp xử lý ngôn ngữ tự nhiên dưới dạng văn bản (text). Do đó, Vector Embedding (Nhúng vector) là kỹ thuật biểu diễn các từ, câu, hoặc toàn bộ đoạn văn bản thành các vector toán học (một mảng chứa các số thực) trong một không gian đa chiều (high-dimensional space) [13].

Mục tiêu của Embedding là nắm bắt ngữ nghĩa (semantics) và ngữ cảnh (context) của ngôn ngữ. Trong không gian vector này, các đoạn văn bản có ý nghĩa tương đồng sẽ được biểu diễn bằng các vector nằm gần nhau. Ví dụ, câu *"Nhân vật A đạo nhái tác phẩm"* và *"Ca sĩ A ăn cắp bản quyền bài hát"* dù không có nhiều từ vựng trùng lặp nhưng về mặt ngữ nghĩa là giống nhau, do đó vector biểu diễn của chúng sẽ có khoảng cách rất gần nhau. Dự án "Drama Intelligence System" sử dụng mô hình nhúng tiên tiến (như Gemini Embedding) để chuyển đổi toàn bộ bài đăng và bình luận trên mạng xã hội thành các chuỗi vector này trước khi lưu trữ.

1.5.6.2. Khái niệm Cosine-Search (Độ tương đồng Cosine)

Khi người dùng đặt câu hỏi, câu hỏi đó cũng được chuyển thành một vector. Để tìm ra dữ liệu phù hợp nhất trả lời cho câu hỏi đó, hệ thống cần đo lường khoảng cách giữa vector câu hỏi và hàng ngàn vector dữ liệu trong kho. Phương pháp phổ biến và hiệu quả nhất trong Xử lý ngôn ngữ tự nhiên là Độ tương đồng Cosine (Cosine Similarity) [14].

Độ tương đồng Cosine là một độ đo toán học dùng để đánh giá sự giống nhau giữa hai vector khác không trong không gian nội tích. Nó đo lường cosin của góc tạo bởi hai vector đó, được tính bằng công thức:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

- A và B là hai vector cần so sánh (ví dụ: vector câu hỏi và vector bình luận).
- A **.** B là tích vô hướng của hai vector.
- |A| và |B| là độ dài (chuẩn Euclidean) của mỗi vector.

Ý nghĩa kết quả: Giá trị của độ tương đồng Cosine dao động từ -1 đến 1. Trong ngữ cảnh phân tích văn bản (nơi các giá trị vector thường dương), giá trị này chạy từ 0 đến 1.

- Giá trị càng tiến gần về 1: Góc giữa hai vector càng nhỏ, chứng tỏ hai đoạn văn bản có ngữ nghĩa càng giống nhau.
- Giá trị tiến về 0: Hai vector vuông góc, đoạn văn bản hoàn toàn không liên quan đến nhau.

Nhờ thuật toán này, hệ thống RAG có thể xếp hạng (Ranking) và trích xuất ra Top K tài liệu có độ tương đồng Cosine cao nhất để cung cấp làm bối cảnh cho LLM.

1.5.6.3. Supabase Vector Store (Cơ sở dữ liệu Vector Supabase)

Các cơ sở dữ liệu quan hệ (SQL) truyền thống được thiết kế để tìm kiếm dạng chuỗi văn bản đối xứng (exact match), do đó chúng rất yếu và chậm khi phải thực hiện các phép toán tìm kiếm khoảng cách vector. Đây là lý do Cơ sở dữ liệu Vector (Vector Database) ra đời. Chúng được tối ưu hóa bằng các thuật toán như HNSW (Hierarchical Navigable Small World) để lưu trữ và truy vấn hàng triệu vector cực kỳ nhanh chóng [15].

Trong dự án này, Supabase Vector Store được lựa chọn làm cơ sở dữ liệu chính. Supabase là một nền tảng Backend-as-a-Service (BaaS) mã nguồn mở, được xây dựng trên nền tảng PostgreSQL. Để hỗ trợ Vector Database, Supabase sử dụng tiện ích mở rộng **pgvector** – cho phép lưu trữ trực tiếp các embeddings đa chiều cùng với dữ liệu JSON (Metadata) ngay bên trong các bảng của PostgreSQL [16]. Đồng thời Supabase Vector Store cho phép truy cập dữ liệu từ n8n, đây là lí do chính vì sao đề tài này lại sử dụng Supabase Vector Store.

Sự tích hợp này giúp "Drama Intelligence System" vừa có thể thực hiện tìm kiếm ngữ nghĩa siêu tốc (thông qua vector), vừa có thể lọc dữ liệu theo các trường thông tin (metadata filtering như lọc theo ID sự kiện, thể loại showbiz/giáo dục) trong cùng một truy vấn, tạo nền tảng vững chắc cho các Tác nhân AI (Agents) truy xuất dữ liệu.

CHƯƠNG 2. THIẾT KẾ HỆ THỐNG VÀ DỮ LIỆU

2.1. Dữ liệu và Đường ống xử lý tự động (Data Pipeline)

2.1.1. Tổng quan, đặc tả và Kiến trúc của Dữ liệu

Hệ thống xử lý khối lượng thông tin lớn qua nhiều giai đoạn, trong đó dữ liệu được phân chia thành ba tầng cốt lõi nhằm phục vụ các mục đích lưu trữ và tính toán khác nhau:

- Tầng 1 - Dữ liệu thô (Raw Data): Dữ liệu thu thập ban đầu được lưu trữ nguyên trạng dưới dạng JSON thô. Tầng này có vai trò phục vụ cho mục đích truy vết (trace) và đảm bảo khả năng tái tạo (reproducibility) của toàn bộ hệ thống.

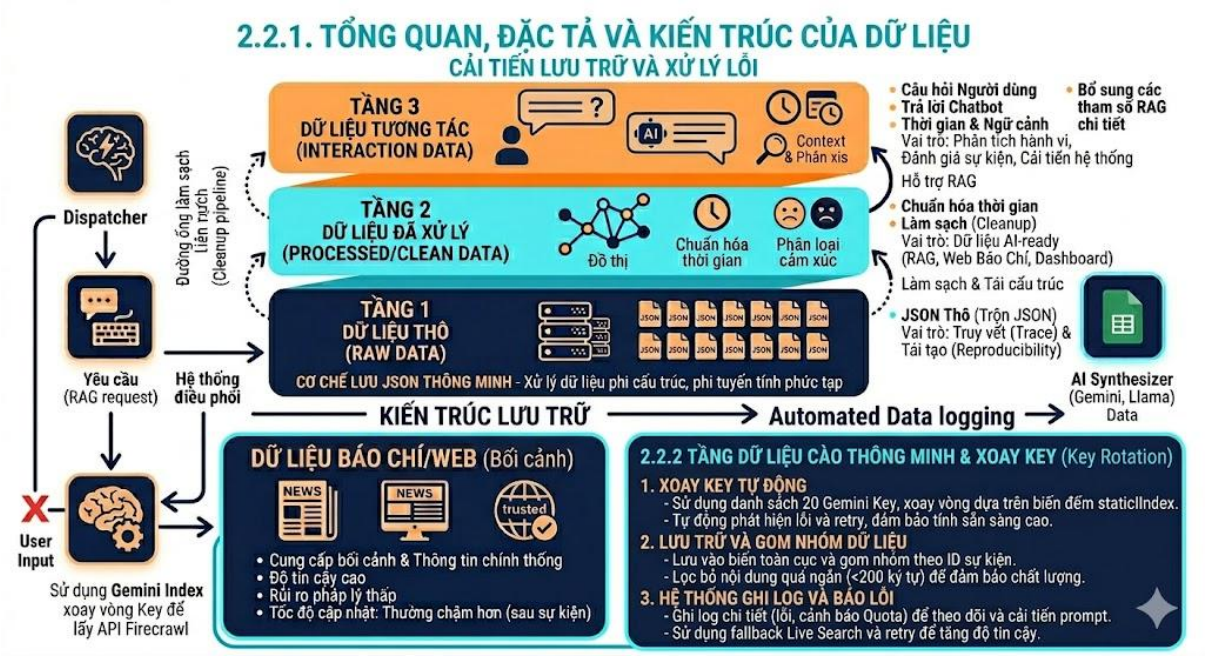
- Tầng 2 - Dữ liệu đã xử lý (Processed/Clean Data): Dữ liệu thô sau khi đi qua đường ống làm sạch (loại bỏ rác, chuẩn hóa thời gian) sẽ được tái cấu trúc thành dạng đồ thị và phân loại cảm xúc. Đây là tập dữ liệu AI-ready được sử dụng cho hệ thống RAG, web báo chí và dashboard.

- Tầng 3 - Dữ liệu tương tác (Interaction Data): Lớp dữ liệu sinh ra trong quá trình vận hành thực tế, bao gồm câu hỏi của người dùng, câu trả lời của Chatbot, thời gian và ngữ cảnh giao tiếp. Dữ liệu này được dùng để phân tích hành vi theo dõi sự kiện của người dùng, đánh giá sự kiện nổi bật và cải tiến hệ thống.

Phân loại nguồn dữ liệu: Hệ thống khai thác hai nguồn dữ liệu độc lập nhằm tối ưu hóa quá trình phân tích:

- Dữ liệu Báo chí/Web (Bối cảnh): Cung cấp bối cảnh, nội dung và thông tin tổng hợp chính thống của sự kiện. Nguồn này có độ tin cậy cao, rủi ro pháp lý thấp, nhưng tốc độ cập nhật thường chậm hơn (sau sự kiện).

- Dữ liệu Facebook (Thái độ dư luận): Phản ánh quan điểm, cảm xúc và thái độ gần như theo thời gian thực (real-time) của cộng đồng. Điểm đặc thù của nguồn này là tính động, khả năng bắt kịp xu hướng nhanh, nhưng chứa mức độ nhiễu cao và cấu trúc phi tuyến tính phức tạp.



Hình 2.1. Tổng quan về dữ liệu.

Lược đồ Bối cảnh (Web Content Schema): Dữ liệu báo chí được tổng hợp và chuẩn hóa dưới dạng JSON để cung cấp ngữ cảnh đầy đủ cho sự kiện. Lược đồ này bao gồm trường định danh id_content (với định dạng type_stt) để liên kết các dữ liệu.Ngoài ra, các trường khác mang ý nghĩa sau:

- "ten_su_kien":Đây là tên của sự kiện đó, nó là từ khóa để dùng và cào dữ liệu trên Website.
- "content": Nội dung được LLMs tổng hợp được từ nhiều trang website sau khi đã cào.
- "time_event": Đây là thời gian của các sự kiện được lưu, LLM sẽ đọc các trang bài báo, bài nào có thông tin về thời gian sự kiện thì LLM sẽ lựa chọn mốc thời gian sớm nhất làm thời điểm đại diện cho sự kiện.
- actor_related: Đây là điểm đặc biệt, trường lưu trữ danh sách các nhân vật liên quan được tự động trích xuất thông qua kỹ thuật Nhận diện thực thể có tên (NER).

```
  "id_content": "education_014",
  "ten_su_kien": "Nữ sinh TP.HCM tố bị đánh cắp bài thi Genius Olympiad",
  "danh_muc": "Giáo dục",
  "content": "Sự việc nữ sinh M.C. đến từ TP.HCM tố cáo nam sinh Q.U. và giáo viên hướng dẫn chung đ
  "actor_related": [
    {
      "name": "M.C.",
      "role": "Nữ sinh tố cáo bị đánh cắp bài thi"
    },
    {
      "name": "Q.U.",
      "role": "Nam sinh bị tố sao chép bài thi"
    },
    {
      "name": "Giáo viên hướng dẫn",
      "role": "Người hướng dẫn cả hai học sinh, bị cấm tham gia Genius Olympiad 2024"
    },
    {
      "name": "Ban tổ chức Genius Olympiad",
      "role": "Đơn vị tổ chức cuộc thi, đưa ra kết luận và hình thức xử lý"
    }
  ],
  "time_event": "2023-07-11"
},
{
  "id_content": "education_015",
  "ten_su_kien": "Trường Quốc tế Mỹ bị giải thể do Chủ trường bị điều tra vì huy động vốn từ phụ huy
  "danh_muc": "Giáo dục",
  "content": "Trường Quốc tế Mỹ (AISVN) đã chính thức bị giải thể theo Quyết định số 3693/QĐ-UBND do
  "actor_related": [
    {
      "name": "Nguyễn Thị Út Em",
```


Hình 2.2. Cấu trúc dữ liệu của data web.

Lược đồ Đồ thị Thảo luận (Discussion Graph Schema): Khác với dữ liệu báo chí tuyến tính, dữ liệu bình luận mạng xã hội mang cấu trúc dạng mạng lưới (network) phi tuyến tính, nơi người dùng liên tục trả lời (reply) các bình luận khác để tạo thành các nhánh thảo luận. Để mô hình hóa cấu trúc này, hệ thống thiết kế lược đồ theo dạng đồ thị thảo luận (discussion graph). Chìa khóa của cấu trúc này nằm ở trường khóa ngoại reply_to_id (lưu trữ ID của bình luận cha mà người dùng đang phản hồi). Nhờ trường dữ liệu này, hệ thống có thể kết nối các bình luận cha-con, tái tạo nguyên vẹn mạch tranh luận, từ đó hỗ trợ thuật toán phân tích luồng quan điểm và mức độ tương tác sâu của các nhóm dư luận.

```
"id_content": "showbiz_001",
"event_name": "Bình Gold [illegible] ồn ào khi nhiều bài rap và phát ngôn trên mạng xã hội bị cho là dung tục, phản cảm và gây tranh cãi mạnh trong
"sort_mode": "most_relevant",
"post_content": "BÌNH GOLD VÀ LOẠT “CHIẾN TÍCH”: VAR CĂNG VỚI HIEUTHUHAI, BỊ VTV RÉO TÊN VÌ MV NHAYCAM\n\n“Bình Vàng” trở thành cái tên n
"comments": [
    {
        "comment_id": "cmt_showbiz001_0001",
        "order": 1,
        "text": "Nhưng nhạc nó ra Chất thật :v",
        "likes": 1,
        "reply_count": 7,
        "replies": [
            {
                "comment_id": "cmt_showbiz001_0001_r01",
                "order": 1,
                "reply_to_id": "cmt_showbiz001_0001",
                "text": "Nguyễn Phương thì chơi ma toé biểu sao ko chất! 🥵",
                "likes": 1
            },
            {
                "comment_id": "cmt_showbiz001_0001_r02",
                "order": 2,
                "reply_to_id": "cmt_showbiz001_0001",
                "text": "Nguyễn Kỳ Anh Tú chắc chơi xong cảm xúc nó thăng hoa và ca từ bay bổng🤩",
                "likes": 0
            }]
        },
    {
        "comment_id": "cmt_showbiz001_0002",
        "order": 2,
        "text": "Ngông cuồng",
        "likes": 12,
        "reply_count": 0,
        "replies": []
```


Hình 2.3. Cấu trúc dữ liệu đồ thị của data Facebook.

2.1.2. Tổng quan quy trình và luồng xử lý dữ liệu

Để đảm bảo luồng dữ liệu được đi từ dạng thô sơ (Raw Data) đến khi trở thành kho tri thức phân tích chuyên sâu (AI-Ready & Insights), hệ thống áp dụng một quy trình xử lý bao gồm 4 giai đoạn nối tiếp nhau.



Hình 2.4. Sơ Pipeline thu thập và xử lý dữ liệu.

2.1.2.1. Tổng quan

Dưới đây là đặc tả tổng quan cho từng giai đoạn trong đường ống dữ liệu (Data Pipeline):

Giai đoạn 1: Xác định sự kiện và Khởi tạo dữ liệu

- Hệ thống xác định các vấn đề, sự kiện trọng tâm thuộc hai lĩnh vực chính: Giải trí (Showbiz) và Giáo dục (Education).
- Mô hình AI (Gemini) được sử dụng để tìm kiếm, trích xuất và xác nhận thông tin tổng quan về các sự kiện này, sau đó lưu trữ tạm thời vào file Excel/Google Sheets để làm đầu vào (input) cho quá trình thu thập tự động.

Giai đoạn 2: Thu thập dữ liệu thô (Raw Data Collection)

- Hệ thống sử dụng nền tảng n8n để tự động đọc danh sách từ file dữ liệu đầu vào, từ đó sinh ra các luồng (workflow) tìm kiếm link bài viết liên quan cho cả Web báo chí và Facebook.
- Sau khi các URL được chuẩn hóa, n8n kích hoạt các luồng thu thập dữ liệu riêng biệt: một luồng chuyên biệt để cào nội dung báo chí (Web Content) và một luồng khác xử lý các rào cản Anti-scraping để cào dữ liệu bình luận từ Facebook.

Giai đoạn 3: Tiền xử lý và Chuẩn hóa dữ liệu (Preprocessing) Dữ liệu thô tải về máy trạm (Local) sẽ được xử lý qua chuỗi các kịch bản (scripts) Python để đồng bộ định dạng:

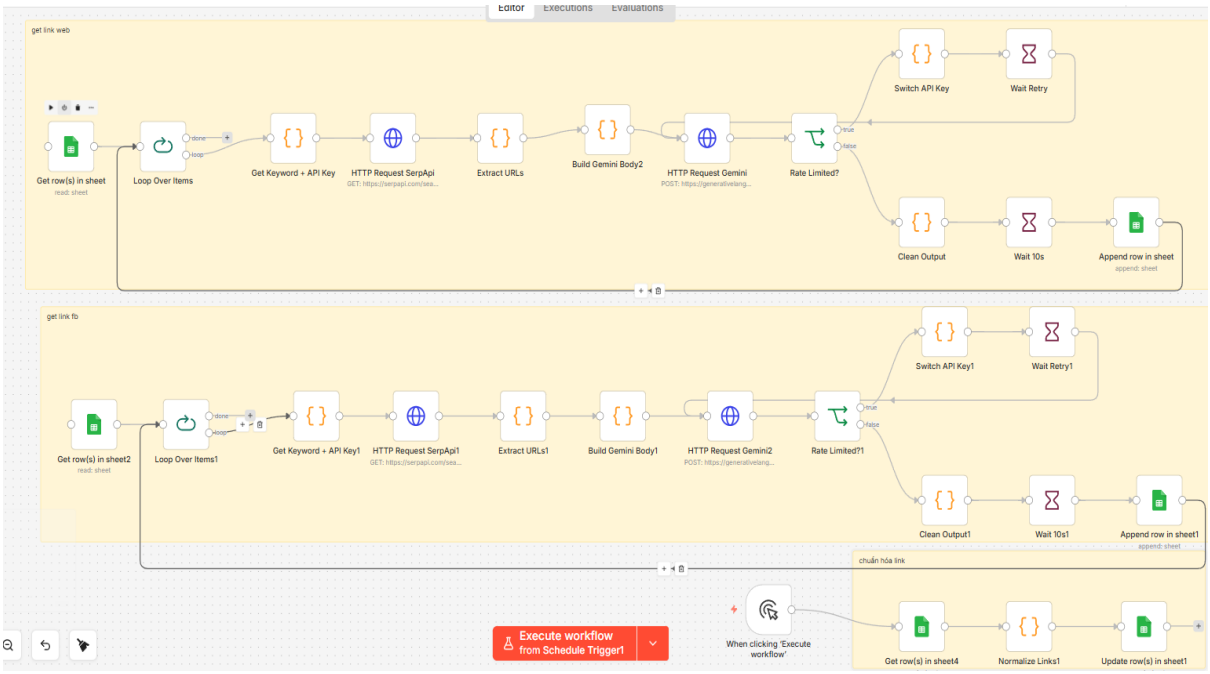
- Chuẩn hóa Format: Sử dụng các scripts normalize_education.py và normalize_showbiz.py để chuẩn hóa dữ liệu bình luận Facebook cho khớp với các trường dữ liệu mới (schema) của hệ thống.
- Hợp nhất dữ liệu: Kịch bản merge_data.py được gọi để gộp tất cả các tệp sự kiện Facebook rời rạc lại với nhau, kết hợp mô hình LLM để viết lại nội dung.
- Làm sạch và Sắp xếp: Áp dụng script tienxuly_vietlaiorder.py để đánh lại số thứ tự (order) của bình luận (do quá trình cào n8n trả về dữ liệu không theo thứ tự thời gian) và tiền lọc bớt các bình luận rác (noise).

Giai đoạn 4: Phân tích chuyên sâu và Đúc kết Insight (Data Analysis) Dữ liệu sạch được đưa vào đường ống phân tích cảm xúc và trích xuất thông tin:

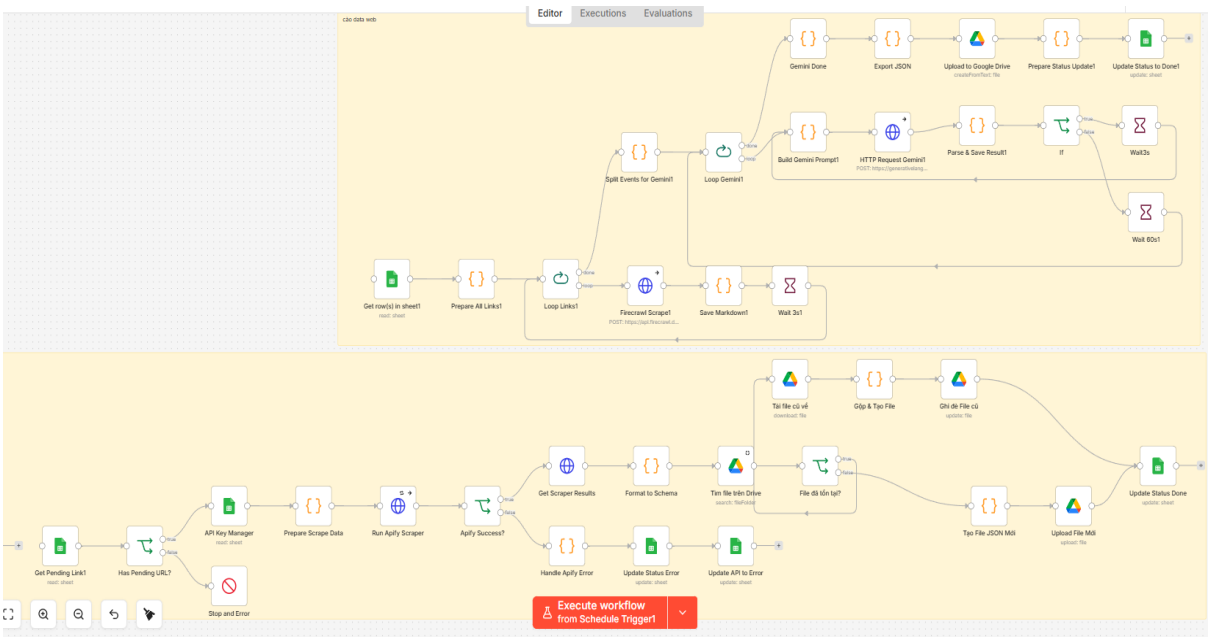
- Phân tích theo lĩnh vực: Hệ thống chạy script analyze_comments.py để đánh giá luồng dữ liệu Showbiz, và kích hoạt analyze_education_async.py để xử lý bất đồng bộ (async) luồng dữ liệu Giáo dục.
- Gộp file kết quả: Script merge_final_data.py đảm nhiệm việc hợp nhất toàn bộ kết quả phân tích, xuất ra 2 tệp Master Data cuối cùng cho hệ thống (Education và Showbiz).
- Trích xuất Insight: Cuối cùng, script analyze_insights.py tổng hợp các chỉ số từ thư mục output để tạo ra tệp insight.json, sẵn sàng phục vụ cho việc trực quan hóa trên Dashboard và làm cơ sở dữ liệu cho hệ thống Chatbot RAG.

2.1.2.2. Pipeline thu thập dữ liệu tự động bằng n8n

Để hiện thực hóa chiến lược thu thập dữ liệu, hệ thống sử dụng n8n đóng vai trò là hệ thống điều phối trung tâm (Workflow Orchestrator). Quá trình thu thập từ khi xác định sự kiện đến khi xuất ra tệp dữ liệu JSON hoàn chỉnh được tự động hóa hoàn toàn thông qua một đường ống (pipeline) gồm 3 giai đoạn cốt lõi:



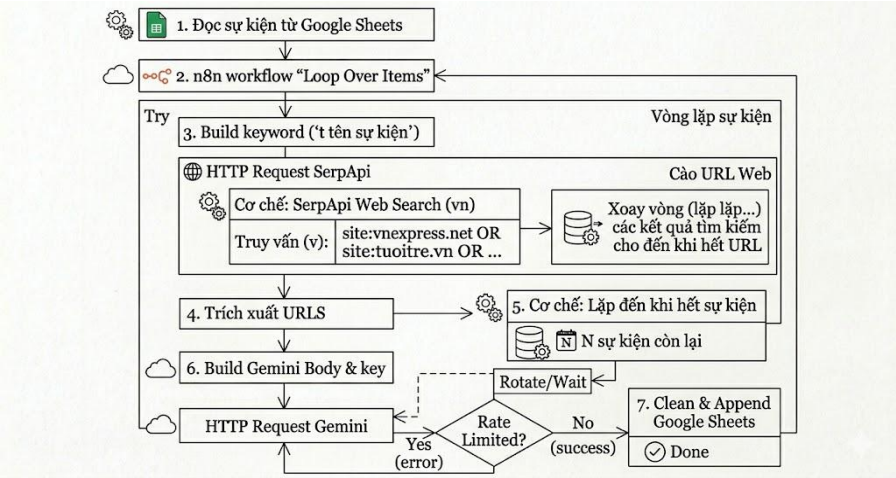
Hình 2.5. Luồng n8n lấy link và chuẩn hóa link.



Hình 2.6. Luồng n8n cào data của Facebook và Website.

2.1.2.2.1. Khởi tạo, Sinh truy vấn và Chuẩn bị liên kết (URL Preparation)

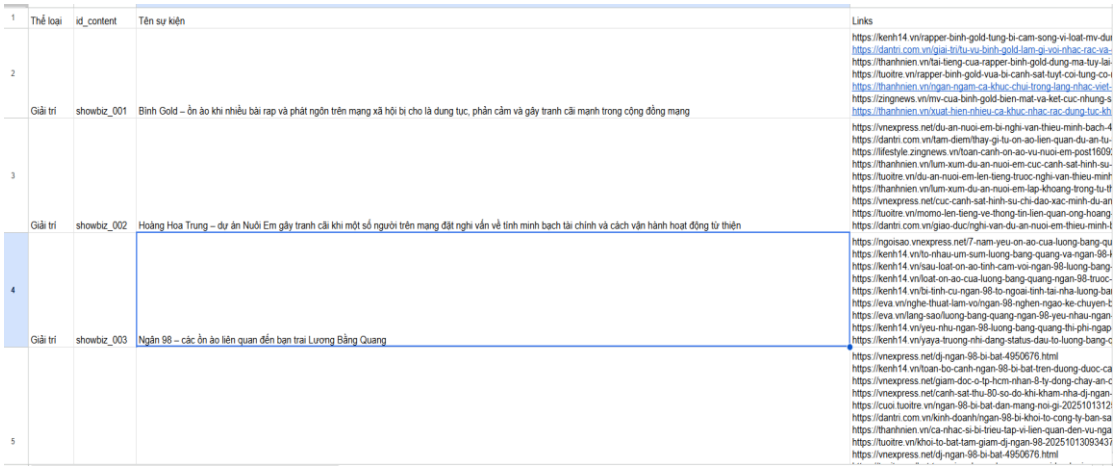
Quá trình bắt đầu bằng việc n8n kết nối và đọc dữ liệu từ tệp "Drama 2025-Đầu 2026" (sheet "Web" và sheet Facebook) trên hệ thống Google Sheets.



Hình 2.7. Luồng tạo URL cho các sự kiện (dữ liệu Web và Facebook)

- Sinh truy vấn tự động: Hệ thống tích hợp API của nền tảng SerpApi (https://serpapi.com/search) để tự động tìm kiếm thông tin sự kiện trên Internet. Riêng đối với mục tiêu thu thập bình luận Facebook, thuật toán tự động nối thêm từ khóa "facebook" vào cuối tên sự kiện (Ví dụ: "hủy kết quả thi khoa học kỹ thuật quốc gia vì đạo nhái facebook") để tạo ra truy vấn đích xác. Hệ thống sử dụng cơ chế xoay vòng để lặp đến khi vét cạn toàn bộ URL của một sự kiện.

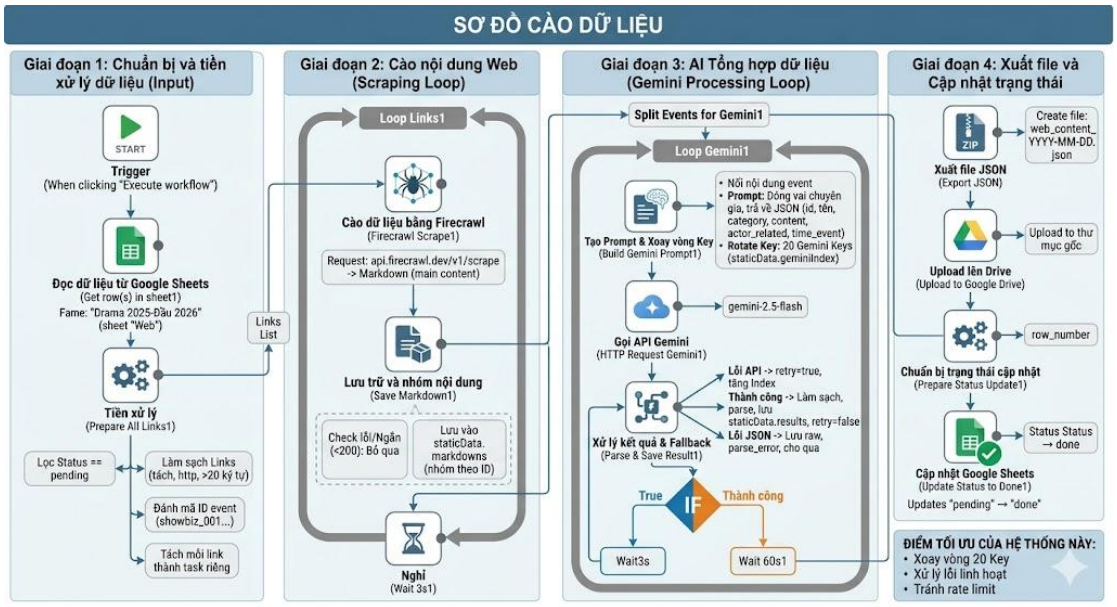
- Tiền xử lý dữ liệu (Node Prepare All Links): Hệ thống chỉ trích xuất các dòng có cột Status là pending. Tại đây, thuật toán làm sạch được kích hoạt nhằm tách link, loại bỏ ký tự rác, và chỉ giữ lại các URL hợp lệ (bắt đầu bằng http và độ dài > 20 ký tự). Đồng thời, hệ thống tự động đánh mã định danh id_content cho từng sự kiện (ví dụ: showbiz_001, education_001) và tách mỗi liên kết thành một tác vụ (task) độc lập để chuẩn bị đưa vào luồng cào dữ liệu.



Hình 2.8. Ảnh minh họa dữ liệu sau khi cào lấy URL

2.1.2.2.2. Cào và Tổng hợp dữ liệu báo chí (Web Content Pipeline)

Sau khi danh sách URL Web được làm sạch, hệ thống bước vào vòng lặp trích xuất và tổng hợp nội dung tin tức.



Hình 2.9. Luồng hoạt động cào data Web.

- Khai thác qua Firecrawl: Tại mỗi vòng lặp, n8n gửi request đến API Firecrawl (api.firecrawl.dev/v1/scrape) để bóc tách nội dung chính (main content) dưới định dạng Markdown. Hệ thống bỏ qua các URL lỗi hoặc nội dung quá ngắn (< 200 ký tự), các bài viết hợp lệ sẽ được lưu vào biến toàn cục và gom nhóm theo id_content. Hệ thống thiết lập độ trễ (Wait 3s) giữa mỗi lần cào để tránh bị block.

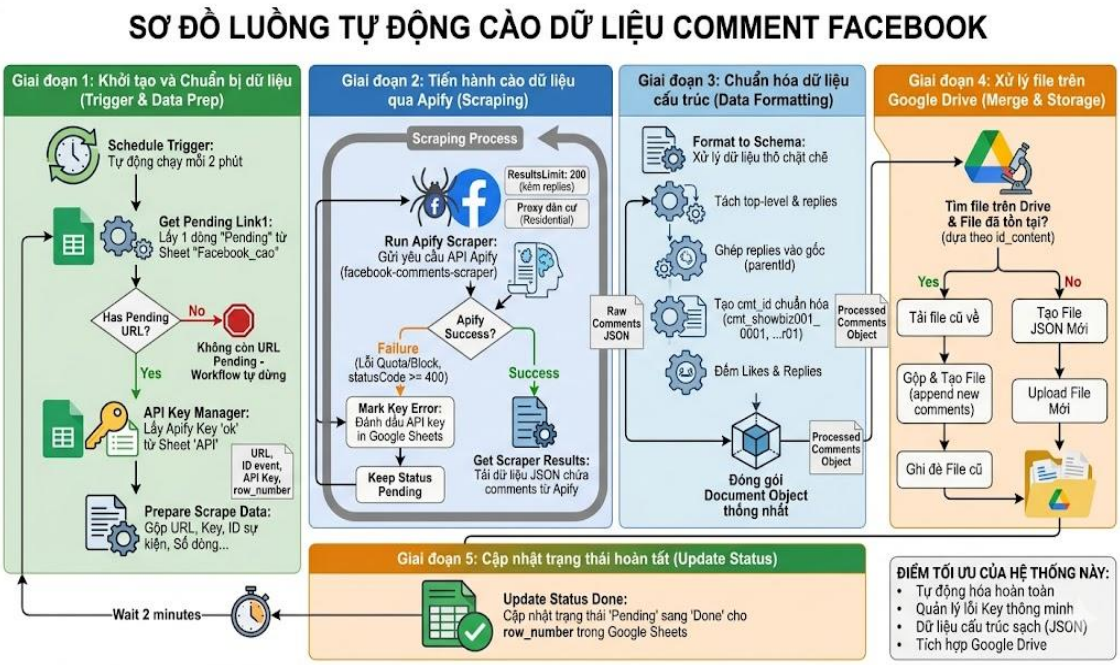
- AI Tổng hợp dữ liệu (Gemini Processing): Toàn bộ nội dung của các bài báo thuộc cùng một sự kiện được nối lại với nhau. Hệ thống truyền vào một Prompt chuyên sâu, yêu cầu Gemini 2.5 Flash đóng vai chuyên gia tổng hợp tin tức, trả về định dạng JSON nghiêm ngặt (chứa id, content, actor_related, time_event).

- Cơ chế quản lý API LLM: Hệ thống tự động xoay vòng danh sách 20 API Key của Gemini. Nếu gặp lỗi quota, biến đếm geminiIndex sẽ tăng để lấy Key tiếp theo, bật cờ Retry và chờ 3 giây để thử lại. Nếu thành công, hệ thống chờ 60 giây trước khi xử lý sự kiện tiếp theo nhằm tránh bị Rate Limit.

- Lưu trữ: Kết quả cuối cùng được gom thành 1 file JSON (ví dụ: web_content_YYYY-MM-DD.json), tự động tải lên Google Drive và cập nhật cột Status trong Google Sheets thành done.

2.1.2.2.3. Luồng khai thác dữ liệu bình luận Mạng xã hội (Facebook Scraping Pipeline)

Khác với báo chí, dữ liệu Facebook mang tính phi tuyến tính và có rào cản Anti-Scraping rất lớn. Để vượt qua, hệ thống triển khai một luồng n8n độc lập gồm 5 giai đoạn tự động hóa:



Hình 2.10. Luồng hoạt động cào data Facebook.

Giai đoạn 1: Khởi tạo và Chuẩn bị (Trigger & Data Prep) Hệ thống chạy ngầm định kỳ (Cron-job mỗi 2 phút), truy xuất các URL sự kiện đang ở trạng thái Pending từ Google Sheets. Đồng thời, n8n tự động lấy một API Key của Apify còn hiệu lực để đóng gói lệnh cào dữ liệu.

Giai đoạn 2: Cào dữ liệu qua Apify (Scraping) Tác vụ được ủy thác cho nền tảng Apify kết hợp mạng Proxy dân cư (Residential Proxy) để qua mặt cơ chế chống Bot của Facebook.

- Nếu thất bại (Block/Hết Quota): API Key bị đánh dấu lỗi, URL giữ nguyên trạng thái Pending để thử lại ở chu kỳ sau bằng Key khác (chiến lược Account Rotation).
- Nếu thành công: Hệ thống tải toàn bộ tệp JSON chứa bình luận về.

Giai đoạn 3: Chuẩn hóa cấu trúc (Data Formatting) Dữ liệu thô được tái cấu trúc thành dạng Đồ thị thảo luận (Discussion Graph). Thuật toán định danh lại comment_id, thống kê lượt tương tác, và đặc biệt sử dụng khóa ngoại (parentId /

reply_to_id) để nối các bình luận trả lời (replies) vào đúng bình luận gốc, giúp duy trì nguyên vẹn mạch tranh luận.

Giai đoạn 4: Hợp nhất và Lưu trữ (Merge & Storage) Hệ thống kiểm tra tệp sự kiện trên Google Drive. Nếu tệp đã tồn tại, n8n sẽ tải về, gộp nối tiếp (append) bình luận mới và ghi đè để không mất dữ liệu cũ. Nếu chưa có, hệ thống tạo tệp JSON mới.

Giai đoạn 5: Cập nhật vòng đời (Update Status) Sau khi lưu trữ, hệ thống cập nhật trạng thái URL trong Google Sheets từ Pending sang Done. Thao tác này đảm bảo tính lũy đẳng (Idempotency), giúp hệ thống không bao giờ cào lặp lại một liên kết.

2.1.3. Chuẩn hóa, tiền xử lý dữ liệu và phân tích chuyên sâu cho AI (Tại Local)

2.1.3.1. Chuẩn hóa dữ liệu

Trong quá trình thu thập, hệ thống ghi nhận sự không đồng nhất về cấu trúc dữ liệu (Data Schema) giữa các tệp dữ liệu cũ (legacy data) và luồng dữ liệu mới. Để hệ thống AI có thể phân tích chính xác, toàn bộ dữ liệu cần được chuẩn hóa về một định dạng thống nhất trước khi tiến hành gộp (merge).

2.1.3.1.1. Phân tích sự sai lệch cấu trúc (Old Schema vs. New Schema)

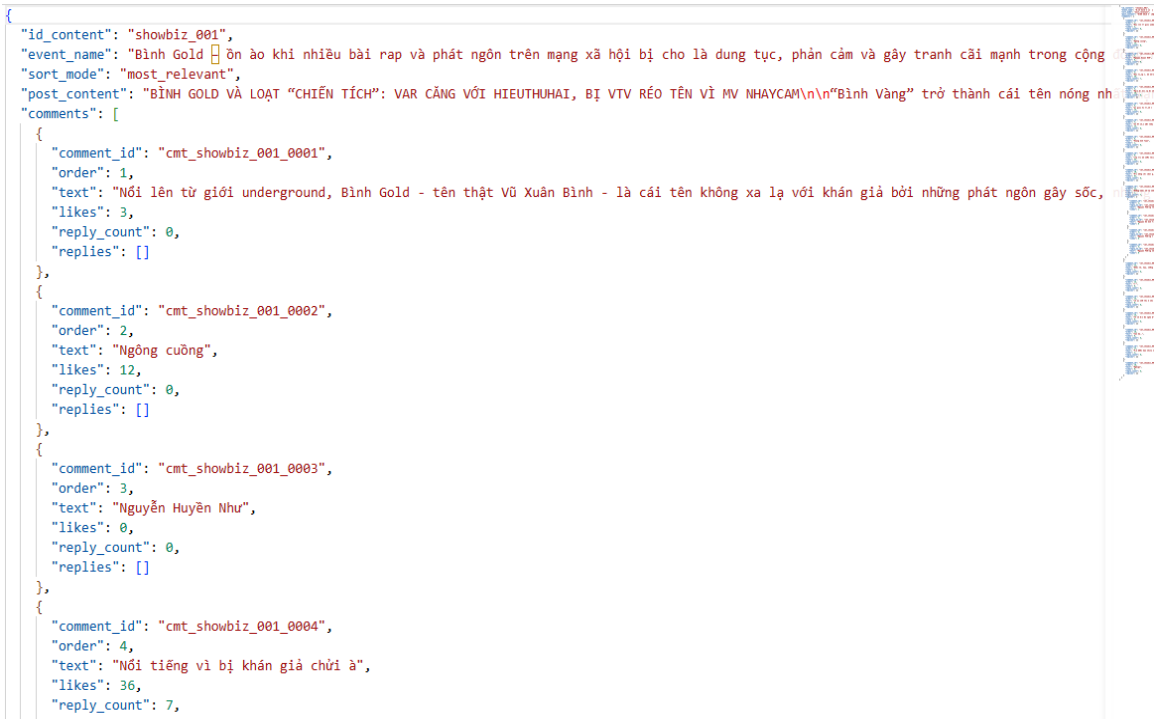
- **Định dạng dữ liệu cũ:** Trong tệp **Showbiz.json**, dữ liệu được tổ chức theo phân cấp lấy **Tên nhân vật** làm khóa chính (Primary Key). Dưới tên nhân vật là các sự kiện, bài đăng (post_content) và cuối cùng là bình luận. Cấu trúc này bộc lộ hạn chế khi không định danh được mã sự kiện độc lập, đồng thời các bình luận (comments) bị thu thập thiếu trường **comment_id** và mất liên kết phân cấp (parent - reply).

- **Định dạng chuẩn mới:** Yêu cầu lấy **Mã sự kiện (id_content)** làm khóa chính (ví dụ: **showbiz_020**). Mỗi sự kiện là một thực thể độc lập chứa bài đăng tổng hợp và cây bình luận đã được định danh ID rõ ràng.



```
{[
    {
        "name": "mailisa",
        "id_content": "showbiz_020",
        "events": [
            {
                "events": "2700_ti_dong",
                "post_content": "🎇 2.700.000.000.000 đồng - là số tiền chảy vào túi vợ chồng Mailisa khi bán 3 mã sản phẩm chủ lực ra thị
                "comments": [
                    {
                        "text": "Hệ thống Mailisa đã phân phối hơn 8 triệu sản phẩm của gần 100 mã hàng mang thương hiệu Doctor Magic, ngoà
                        "profileName": "Kenh14.vn",
                        "likesCount": "145",
                        "commentsCount": 2
                    },
                    {
                        "text": "2700 tỷ \nviết ngắn gọn thôi shop",
                        "profileName": "Tâm An",
                        "likesCount": "814",
                        "commentsCount": 67
                    },
                    {
                        "text": "CE phụ nữ công nhận tiêu tiền ác thật 😜",
                        "profileName": "ママ レジ",
                        "likesCount": "51",
                        "commentsCount": 9
                    },
                    {
                        "text": "ママ レジ từ xưa đến giờ rồi,kinh doanh buôn bán mặt hàng dành cho ce phụ nữ là khi nào cũng lời nhiều nhất
                        "profileName": "Đào Minh San",
                        "parentReply": {
                            "author": {
                                "__typename": "User",
                                "name": "ママ レジ",
                                "gender": "FEMALE"
```


Hình 2.11. Cấu trúc dữ liệu cũ phân cấp theo Tên nhân vật.



Hình 2.12. Data Schema chuẩn mới với id_content làm khóa chính.

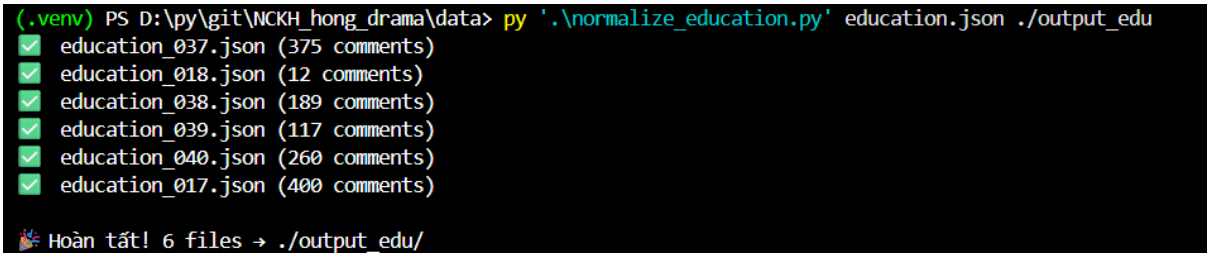
2.1.3.1.2. Quy trình xử lý và chuẩn hóa dữ liệu cũ

Để khắc phục sự sai lệch, một script tiền xử lý được xây dựng nhằm bóc tách và định hình lại toàn bộ dữ liệu cũ theo 3 bước:

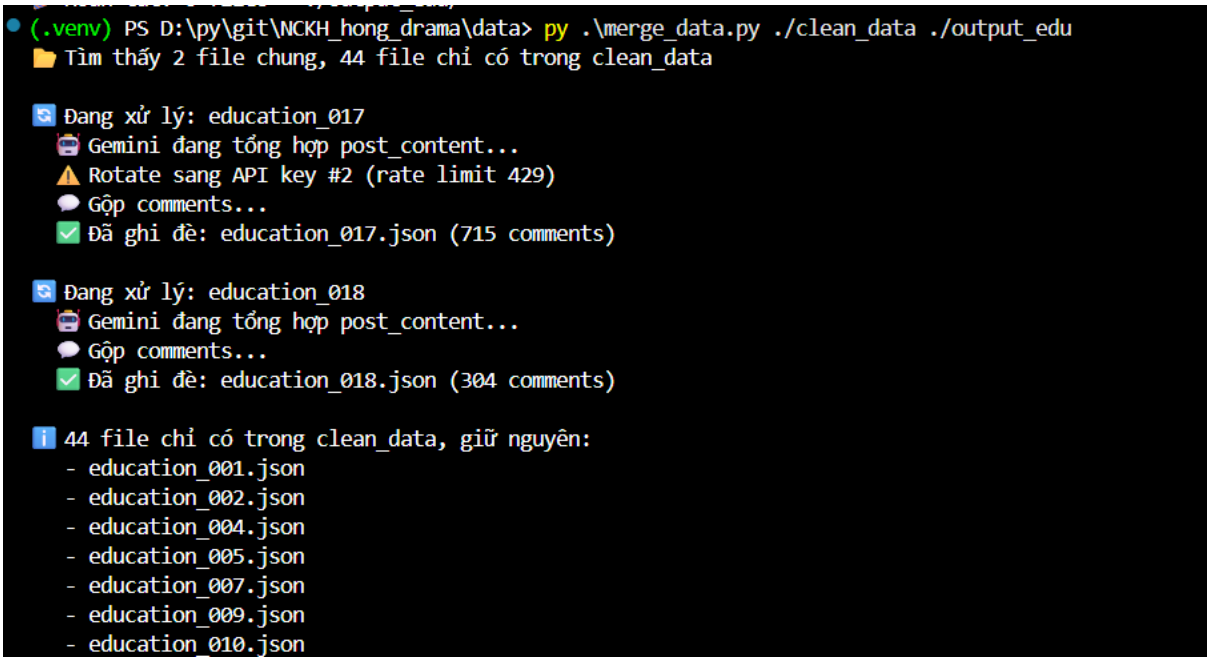
1. **Tái cấu trúc khóa chính và Phân tách tệp:** Hệ thống duyệt qua tệp **Showbiz.json** cũ, loại bỏ "Tên nhân vật" khỏi vị trí khóa chính. Các sự kiện được gán thêm trường **id_content** (được mapping thủ công từ trước) và xuất ra thành từng tệp JSON riêng biệt (ví dụ: **showbiz_020.json**).

2. **Đồng bộ hóa trường post_content:** Do hệ thống cũ thu thập từ nhiều nguồn link cho cùng một sự kiện, dẫn đến việc tồn tại nhiều mảng **post_content** rời rạc. Script thực hiện gom cụm (aggregation) bằng cách nối chuỗi toàn bộ nội dung thành một khối duy nhất theo nguyên tắc:
post_content_final = <post_content_1> + <post_content_2> + ... + <post_content_n>

3. **Tái tạo cấu trúc cây bình luận (Comment Hierarchy):** Dữ liệu bình luận cũ được gán bổ sung các **id_comments** duy nhất. Đồng thời, thuật toán sắp xếp lại luồng bình luận, định vị lại các bình luận trả lời (replies) để liên kết đúng với bình luận gốc (parent comment), đảm bảo tính nhất quán hoàn toàn với schema mới.



Hình 2.13. Sử dụng normalize_education.py để chuẩn hóa các sự kiện giáo dục cũ.



Hình 2.14. Script merge_data dùng để gộp dữ liệu và tổng hợp post_content.

2.1.3.2. Tiền xử lý dữ liệu

2.1.3.2.1. Tổng hợp và gộp dữ liệu (merge_data.py)

- **Tổng hợp nội dung bài đăng:** Sử dụng script **merge_data.py** kết hợp với việc gọi API mô hình **Gemini 2.5 Flash** nhằm tổng hợp và biên soạn lại trường **post_content** từ cả hai nguồn dữ liệu cũ và mới. Các đoạn **post_content** rời rạc thuộc dữ liệu cũ sẽ được AI xử lý và hợp nhất thành một văn bản thống nhất.

- **Gộp bình luận:** Cơ chế của file này tiếp tục thực hiện việc hợp nhất toàn bộ kho bình luận (comments) từ cả hai nguồn dữ liệu lại với nhau thành một khối duy nhất.

2.1.3.2.2. Sắp xếp lại thứ tự và lọc nhiễu (tienxuly_vietlaiorder.py)

Script **tienxuly_vietlaiorder.py** được triển khai để giải quyết hai vấn đề quan trọng về chất lượng dữ liệu:

- **Chuẩn hóa định vị bình luận (Re-ordering):** Trong quá trình nền tảng n8n thu thập và gộp dữ liệu, thứ tự các bình luận thường không được sắp xếp đúng theo luồng từ trên xuống. Điều này khiến **comment_id** không được định vị chính xác vị trí, gây lỗi truy xuất khi các thuật toán phân tích dữ liệu sau này gọi đến **comment_id**. Script sẽ duyệt qua toàn bộ dữ liệu và đánh lại số thứ tự (order) chuẩn xác cho từng sự kiện.

- **Lọc dữ liệu rác (Noise Filtering):** Đồng thời, thuật toán sẽ tiến hành thanh lọc một phần dữ liệu bị nhiễu không mang lại giá trị phân tích. Cụ thể, các nội dung bị loại bỏ bao gồm: bình luận vô nghĩa, cú pháp gắn thẻ tài khoản (người A tag người B), hoặc các bình luận chỉ chứa biểu tượng cảm xúc (emotions như 🙄, 😂, 😣,...).

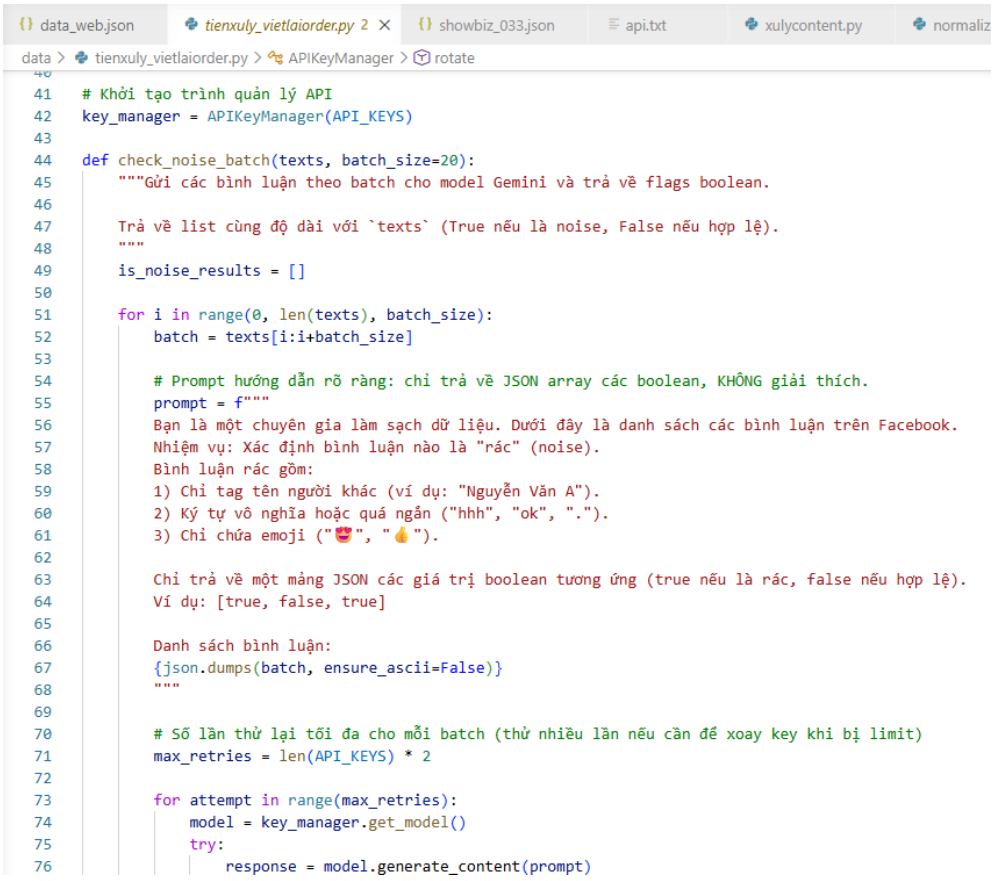
Dưới đây là ví dụ minh họa cụ thể cho quy trình này:


```
{
  "comment_id": "cmt_education005_0081",
  "order": 81,
  "text": "Vặt vãnh chi cho khổ cô hì",
  "likes": 1,
  "reply_count": 0,
  "replies": []
},
{
  "comment_id": "cmt_education005_0082",
  "order": 82,
  "text": "Thường thì khi sửa họ sẽ nâng chứ ít ai thần kinh mà đi hạ. Cái \" thần kinh\" này có lẽ để mời học sinh đến để cô phụ đạo.",
  "likes": 0,
  "reply_count": 0,
  "replies": []
},
{
  "comment_id": "cmt_education005_0001",
  "order": 1,
  "text": "Rồi tình trang các Cô sửa sai thành đúng chắc cấp 1 phạt có mà đầy . 4-5 sữa 9-10. Còn đứa nào bị đi mà học giỏi lỡ 1 môn 8đ thì
  "likes": 9,
  "reply_count": 2,
  "replies": [
    {
      "comment_id": "cmt_education005_0001_r01",
      "order": 1,
      "reply_to_id": "cmt_education005_0001",
      "text": "Trúc Nguyễn thường cho ít ai nói, chứ hạ kiểu này dễ bị kiện lắm",
      "likes": 0
    },
    {
      "comment_id": "cmt_education005_0001_r02",
      "order": 2,
      "reply_to_id": "cmt_education005_0001",
      "text": "Hip Hip cái nào cũng kiện hết.",
```


Hình 2.15. Tình trạng sai lệch thứ tự (order) của dữ liệu bình luận thô trước khi tiền xử lý.

```
    {
      "comment_id": "cmt_education001_0003",
      "order": 3,
      "text": "Đoàn Đức Thành",
      "likes": 2,
      "reply_count": 0,
      "replies": []
    },
    {
      "comment_id": "cmt_education001_0004",
      "order": 4,
      "text": "Hhh",
      "likes": 0,
      "reply_count": 0,
      "replies": []
    },
    {
      "comment_id": "cmt_education001_0005",
      "order": 5,
      "text": "Trúc Phương bạn hiểu ý mình trứ 😎",
      "likes": 1,
      "reply_count": 1,
      "replies": [
        {
          "comment_id": "cmt_education001_0005_r01",
          "order": 1,
          "reply_to_id": "cmt_education001_0005",
          "text": "Santoso Niran đã quáaaa yomost ơi",
          "likes": 0
        }
      ]
```


Hình 2.16. Một số ví dụ về Noise.



Hình 2.17. Sử dụng gemini để lọc noise và viết lại order.

2.1.3.2.3. Kỹ thuật tiền lọc Rule-based và Xử lý nhiễu cơ học

Để tối ưu hóa chi phí vận hành và giảm tải cho mô hình ngôn ngữ lớn (LLM), hệ thống triển khai một lớp lọc tiền xử lý nhằm loại bỏ "rác cơ học" trước khi dữ liệu được gửi đến API. Quá trình này được thực hiện thông qua sự phối hợp của hai hàm cốt lõi: flatten_comments() và is_trash_rule().

A. Hàm dàn phẳng cây bình luận: flatten_comments()

Dữ liệu thô từ Facebook thường có cấu trúc phân cấp (bình luận gốc và các phản hồi). Hàm này có nhiệm vụ:

- **Cấu trúc lại dữ liệu:** Chuyển đổi cấu trúc cây phức tạp thành một danh sách phẳng (Flat list) để dễ dàng quản lý và xử lý hàng loạt.
- **Truy vết ngữ cảnh:** Lưu trữ thông tin **parent_text** cho các phản hồi (replies) để giữ lại ngữ cảnh cần thiết khi phân tích ở các bước sau.
- **Tích hợp lọc sớm:** Ngay trong quá trình dàn phẳng, mỗi bình luận sẽ được kiểm tra qua hàm **is_trash_rule()** để gắn cờ **rule_trash** (True/False).

```
323 def flatten_comments(comments):
324     flat = []
325     for c in comments:
326         flat.append({
327             "comment_id":  c["comment_id"],
328             "text":        c.get("text", ""),
329             "parent_text": None,
330             "rule_trash":  is_trash_rule(c.get("text", ""))
331         })
332         for r in c.get("replies", []):
333             flat.append({
334                 "comment_id":  r["comment_id"],
335                 "text":        r.get("text", ""),
336                 "parent_text": c.get("text", ""),
337                 "rule_trash":  is_trash_rule(r.get("text", ""))
338             })
339     return flat
340
```

Hình 2.18. Đoạn code mô tả quá trình làm phẳng cây dữ liệu.

B. Hàm nhận diện rác cơ học: is_trash_rule()

Đây là "bộ lọc cứng" sử dụng các quy tắc biểu thức chính quy (Regex) và logic nghiệp vụ để xác định các nội dung không mang giá trị thông tin cho việc phân tích khủng hoảng truyền thông. Các tiêu chí loại bỏ chính bao gồm:

- **Định dạng và độ dài:** Loại bỏ các chuỗi rỗng, văn bản có ít hơn 4 ký tự sau khi xử lý khoảng trắng, hoặc văn bản không chứa ký tự chữ cái (chỉ chứa số hoặc ký hiệu đặc biệt).
- **Nhận diện Spam âm thanh & URL:** Gạt bỏ các biểu hiện lặp âm không giá trị (như "hhhh", "kkkk", "lol") và các nội dung chứa liên kết (http/https) hoặc tên miền (.vn).
- **Lọc từ khóa quảng cáo & Điều hướng:** Loại bỏ các bình luận chứa từ khóa đặc trưng của việc chia sẻ fanpage, quảng cáo tuyển sinh (thpt, thcs, học mãi...) hoặc các cụm từ chèo kéo khách hàng ("inbox mình", "share", "bit.ly").
- **Lọc theo thực thể người dùng:** Sử dụng thuật toán trợ giúp để nhận diện các bình luận chỉ chứa tên người (Tag tên). Nếu bình luận chỉ gồm tên người hoặc tên người kèm theo một đoạn văn bản quá ngắn (< 30 ký tự/chỉ chứa emoji), hệ thống sẽ đánh dấu là rác.

```
157  def is_trash_rule(text: str) -> bool:
158      raw = text.strip()
159      t   = raw.lower()
160
161      if len(t.replace(" ", "")) < 4:
162          return True
163      if not re.search(r'[a-zàáâãèéêìíòóôõùúăđĩũơ]', t):
164          return True
165      if re.fullmatch(r'(h+|k+|ok+e*|lol+|haha+|he+|hi+|uh+|ah+|ew+|hehe+)', t):
166          return True
167      if re.search(r'https?://', t):
168          return True
169      if re.search(r'\b(thpt|thcs|hocmai|fanpage)\b', t):
170          return True
171      if re.search(r'\b(trường thpt|trường thcs|trường đh|trường đại học)\b', t):
172          return True
173      if re.search(r'[a-z]+\.vn\b', t):
174          return True
175      if "@" in t and len(raw.split()) <= 4:
176          return True
177      spam_keywords = ["lụm về share", "lụm", "share", "quảng cáo",
178                       "inbox mình", "inbox em", "tag ", "bit.ly"]
179      if any(k in t for k in spam_keywords):
180          return True
181
182      words = raw.split()
183
184      if looks_like_name(raw):
185          return True
186
187      for name_len in range(4, 1, -1):
188          if len(words) >= name_len and looks_like_name(" ".join(words[:name_len])):
189              remaining = " ".join(words[name_len:]).strip()
190              if not remaining:
191                  return True
192              if len(remaining) < 30:
```

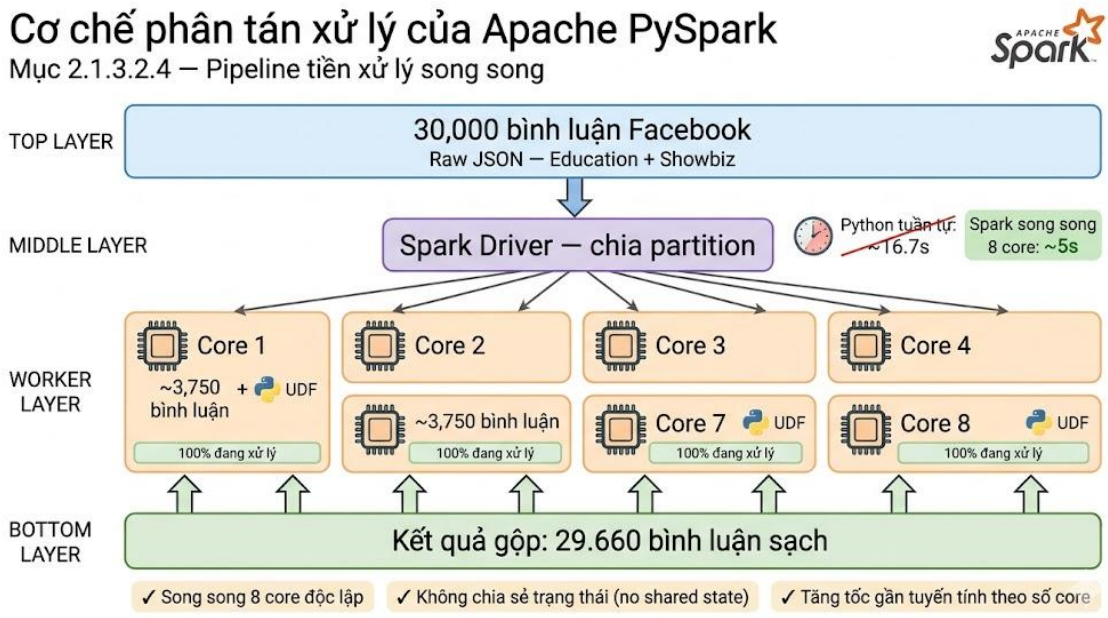
Hình 2.19. Đoạn code mô tả quá trình xử lý dữ liệu theo rule.

Lợi ích của bước này: Việc loại bỏ rác ở tầng "cơ học" giúp hệ thống giảm thiểu tới 40-60% lượng dữ liệu thừa (tùy thuộc vào độ "nhiễu" của post), từ đó tiết kiệm đáng kể chi phí API và tăng tốc độ xử lý tổng thể của Pipeline.

2.1.3.2.4. Pipeline tiền xử lý song song bằng Apache PySpark

Bên cạnh pipeline Python đơn luồng đã trình bày ở các mục trước, hệ thống triển khai thêm một pipeline tiền xử lý song song sử dụng nền tảng Apache PySpark. Cần lưu ý rằng, pipeline PySpark này không thay thế pipeline Python hiện có. Cả hai được thiết kế để chạy song song nhằm phục vụ hai mục đích nghiên cứu cốt lõi: kiểm chứng tính đúng đắn và độ ổn định của pipeline Python bằng cách đối chiếu chéo kết quả trên cùng một tập dữ liệu đầu vào, chuẩn bị sẵn sàng nền tảng kiến trúc phân tán cho giai đoạn mở rộng quy mô thu thập lên hàng triệu bình luận.

Pipeline PySpark được tổ chức thành 8 module độc lập (đặt tại thư mục data/spark/) với tổng dung lượng khoảng 1.135 dòng mã, mỗi tệp được giới hạn dưới 300 dòng nhằm tuân thủ nguyên tắc thiết kế modular. Các module này đảm nhiệm đầy đủ vòng đời xử lý tương đương pipeline Python: khởi tạo SparkSession, lọc rác, token hóa tiếng Việt (tích hợp thư viện underthesea), dàn phẳng cây bình luận theo reply_to_id, trích xuất N-gram, tính toán điểm PMI, và đặc biệt là các module dùng để kiểm chứng đối chiếu (benchmark/validation).



Hình 2.20. Cơ chế phân tán xử lý của Apache PySpark trên 8 core CPU độc lập.

2.1.3.2.5. Đóng gói bộ lọc rule-based thành Spark UDF cho kiến trúc phân tán

Để tích hợp bộ lọc rule-based đã trình bày ở mục 2.1.3.2.3 vào pipeline PySpark song song, hệ thống đóng gói toàn bộ logic năm tiêu chí lọc rác thành một hàm User-Defined Function (UDF) đăng ký với SparkSession.Cần nhấn mạnh rằng logic lọc không thay đổi so với mục 2.1.3.2.3 — đây là quyết định có chủ đích nhằm đảm bảo hai pipeline (Python và PySpark) cho ra cùng kết quả phân loại, qua đó cho phép kiểm chứng đối chiếu chéo (cross-validation) giữa hai pipeline ở mục 4.4.3.

Khác biệt cốt lõi của mục này so với mục 2.1.3.2.3 nằm ở cách thực thi, chứ không phải ở rule. Pipeline Python gọi hàm is_trash tuần tự trên một luồng đơn, xử lý từng bình luận một. Trong khi đó, Spark UDF được tuần tự hóa (serialize) bằng cơ chế Pickle, sau đó được Spark Driver phân phối tới mỗi Spark executor. Mỗi executor xử lý một phân vùng (partition) độc lập của DataFrame phẳng, cho phép lọc song song qua nhiều CPU core hoặc nhiều node trong cluster.

Có hai yêu cầu kỹ thuật bắt buộc khi chuyển một hàm Python thường thành Spark UDF, và cũng chính là lý do hệ thống không tích hợp bộ lọc LLM của pipeline Python vào Spark:

- **Tính picklable hoàn toàn:** Hàm không được tham chiếu tới biến toàn cục có trạng thái thay đổi, không được phụ thuộc tài nguyên I/O hoặc kết nối mạng tại thời điểm thực thi. Bộ lọc rule-based đáp ứng yêu cầu này tự nhiên do toàn bộ logic chỉ dùng regex và phép so chuỗi cục bộ. Ngược lại, gọi API LLM trong UDF gây lỗi tuần tự hóa và race condition khi nhiều executor đồng thời gửi yêu cầu mạng — cơ chế retry và rate-limit của Spark không tương thích với hạn mức quota của API bên thứ ba.

- **Tính thuần và xác định (pure & deterministic):** Hàm phải là pure function cùng đầu vào luôn cho cùng đầu ra. Yêu cầu này phù hợp tự nhiên với rule-based filter, nhưng không đảm bảo được với LLM do non-zero temperature và hiện tượng API drift theo thời gian. Trong môi trường phân tán, sự bất định này dẫn tới hiện tượng cùng một bình luận có thể bị phân loại khác nhau giữa các lần chạy, phá vỡ tính tái lập của thực nghiệm.

Bên cạnh hai yêu cầu trên, lợi thế hiệu năng cũng là một động lực quan trọng. Tốc độ thực thi của UDF rule-based nhanh hơn khoảng 1.000 lần so với phương pháp gọi LLM-based filter qua mạng (giả sử có thể đóng gói được), do loại bỏ hoàn toàn độ trễ mạng (network round-trip) và chi phí tuần tự hóa JSON request/response. Trên cluster đa node, lợi thế này còn được nhân lên theo số executor.

Đoạn mã dưới đây minh họa cách đăng ký UDF từ hàm is_trash thuần (đã trình bày ở mục 2.1.3.2.3) và áp dụng phân tán lên DataFrame phẳng:

```
from pyspark.sql.functions import udf

from pyspark.sql.types import BooleanType

from data.spark.is_trash_rule import is_trash

# Đóng gói hàm Python thuần (mục 2.1.3.2.3) thành Spark UDF.

# Spark Driver tuần tự hóa hàm bằng Pickle, phân phối tới từng
  executor.

is_trash_udf = udf(is_trash, BooleanType())

# Mỗi partition của DataFrame được xử lý độc lập trên 1 executor.

# Trên local[8], 8 partition chạy song song trên 8 CPU core.

clean_df = (  flat_df .withColumn("is_trash", is_trash_udf("text"))
  .filter("is_trash = false")  )

print(f"Comments sạch: {clean_df.count()}")  # 29.660 dòng
```

Đoạn mã 2.1. Đăng ký Spark UDF cho bộ lọc rule-based và áp dụng phân tán — trích từ data/spark/pyspark_preprocessing.py.

Kết quả lọc trên toàn bộ tập dữ liệu thực tế cho ra 29.660 bình luận sạch, trùng khớp tuyệt đối với pipeline Python (chi tiết kiểm chứng đối chiếu ở mục 4.4.3). Sự đồng nhất này xác nhận rằng việc chuyển từ thực thi tuần tự sang thực thi phân tán không thay đổi ngữ nghĩa logic, chỉ thay đổi mô hình tính toán — đây là tiêu chí quan trọng để hai pipeline có thể cùng tồn tại và đối chiếu chéo trong nghiên cứu.

2.1.3.3. Phân tích dữ liệu

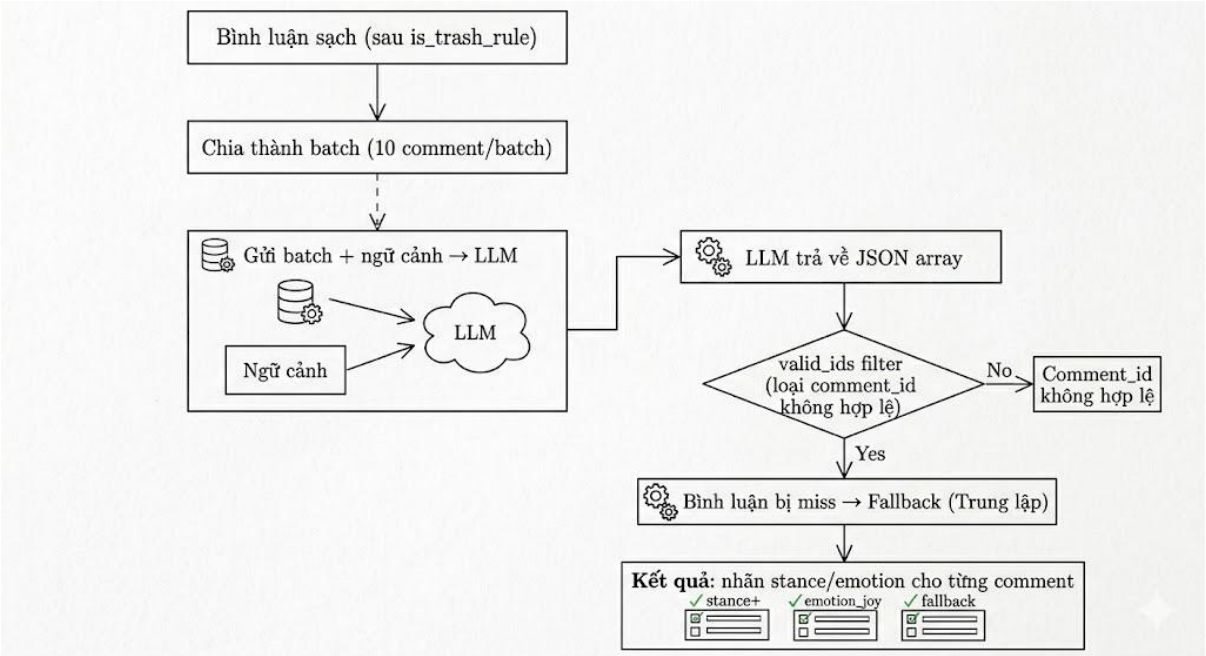
2.1.3.3.1. Chiến lược xử lý hai bước (Two-step Processing)

Sau khi bộ lọc cơ học loại bỏ rác, các bình luận còn lại được đưa vào pipeline phân tích chính. Hệ thống áp dụng chiến lược hai bước nhằm cân bằng giữa độ chính xác và hiệu suất xử lý.

Bước 1 — Phân loại cơ bản (Sequential Batch Processing):

Các bình luận sạch được chia thành các lô với kích thước cố định BATCH_SIZE = 10. Mỗi lô được xử lý tuần tự, trong đó LLM nhận vào ngữ cảnh bài đăng gốc, danh sách bình luận cần phân tích và bộ ví dụ minh họa (few-shot examples) để thực hiện hai nhiệm vụ đồng thời: kiểm tra rác lần cuối và gán nhãn thái độ sơ bộ.

Để đảm bảo tính toàn vẹn dữ liệu, hệ thống áp dụng cơ chế **valid_ids filter** — chỉ chấp nhận kết quả trả về từ LLM nếu comment_id nằm trong tập hợp đã gửi đi, loại bỏ hoàn toàn hiện tượng LLM tự sinh ra comment_id không tồn tại (hallucination). Các bình luận bị bỏ sót hoặc parse lỗi sẽ được tự động áp dụng giá trị fallback mặc định.



Hình 2.21. Sơ đồ luồng dữ lý batch tuần tự.

Kết thúc bước 1, các bình luận được gán một trong các nhãn:

Đối với lĩnh vực Giáo dục, bước 1 phân loại bình luận thành 4 nhãn sơ bộ:

- **Tích cực**: Đồng ý, ủng hộ, khen ngợi chính sách hoặc sự kiện
- **Tiêu cực**: Phản đối, chỉ trích, lo ngại, không đồng ý
- **Trung lập**: Thờ ơ, không quan tâm, không có lập trường rõ ràng

- **Cần xem xét**: Bình luận có ý kiến 2 chiều, đề xuất giải pháp, hoặc ủng hộ có điều kiện — nhóm này được chuyển sang bước 2 để phân tích sâu hơn

Đối với lĩnh vực Showbiz, chỉ có một bước phân tích duy nhất, trả ra ngay 5 nhãn cảm xúc cuối cùng:

- **Phẫn nộ**: Tức giận, bức xúc, lên án mạnh
- **Cà khịa**: Mỉa mai, châm biếm, đùa cợt có ý chỉ trích
- **Đồng cảm**: Thương, chia sẻ cảm xúc với nhân vật
- **Ủng hộ**: Đồng ý, bảo vệ, khen ngợi nhân vật
- **Trung lập**: Chia sẻ thông tin, hỏi han, không rõ cảm xúc

Đồng thời, bước này cũng phát hiện và phân loại nội dung độc hại (toxic) thành 3 nhóm: chửi bới, công kích cá nhân, công kích người thân.

Bước 2 — Phân tích sâu (Asynchronous Parallel Processing):

Các bình luận mang nhãn "cần xem xét" — thường là những bình luận có lập luận phức tạp, kết hợp nhiều chiều ý kiến — được xử lý bất đồng bộ bằng **asyncio.gather**, cho phép nhiều lô chạy song song cùng lúc. Mức độ đồng thời được kiểm soát thông qua **asyncio.Semaphore** với giá trị MAX_CONCURRENT = 5, tránh quá tải API.

Ở bước này, LLM thực hiện phân tích ngữ nghĩa sâu hơn với prompt chuyên biệt, tập trung vào việc phân biệt hai nhóm dễ nhầm lẫn nhất:

- **Trung lập**: Người dùng không có ý kiến, thờ ơ.
- **Ý kiến riêng**: Người dùng có lập luận rõ ràng, đề xuất giải pháp, hoặc ủng hộ có điều kiện.

Dưới đây là prompt và bảng ví dụ cho sánh :

```python
def build_prompt_step2(post_content, batch, event_name):
    items_str = ""
    for i, item in enumerate(batch):
        items_str += f"\n[{i}] comment_id: {item['comment_id']}\n"
        if item.get("parent_text"):
            items_str += f"    (reply của: \"{item['parent_text'][:80]}\")\n"
        items_str += f"    text: \"{item['text'][:300]}\"\n"

    return f"""Bạn là chuyên gia phân tích dư luận về chính sách giáo dục Việt Nam.

Sự kiện: {event_name}
Nội dung bài đăng: {post_content[:400]}

Phân loại chính xác các comment sau: "trung lập" hay "ý kiến riêng"?

COMMENTS:{items_str}

PHÂN BIỆT:
"trung lập": thờ ơ, không quan tâm, chỉ hỏi han, chờ xem, các câu hỏi mang tính chất
"kệ", không có ý kiến rõ ràng.
"ý kiến riêng": ủng hộ một phần + phê phán phần khác, đề xuất giải pháp, phân tích
2 chiều, có quan điểm rõ ràng dù có thể không hoàn toàn tích cực hoặc tiêu cực.

Chỉ trả về JSON array, không giải thích:
[{{"comment_id":"...","is_trash":false,"stance":"ý kiến riêng"}}]"""
```

Hình 2.22. Bảng so sánh ví dụ bình luận Trung lập và Ý kiến riêng.

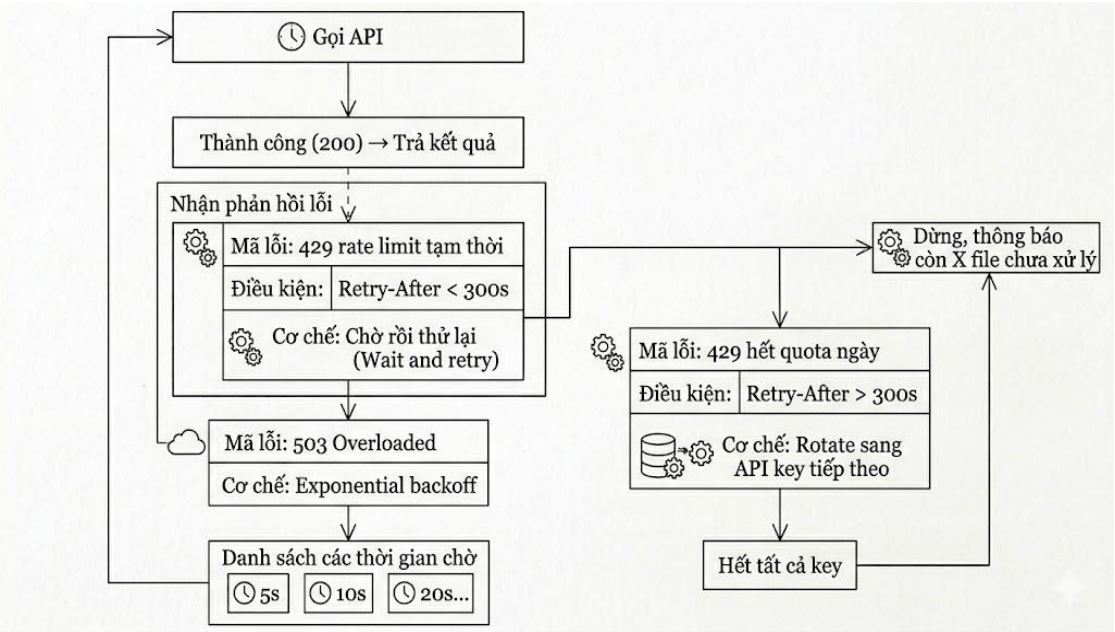

```
33    "trung lập": {
34      "count": 22,
35      "percent": "25.9%",
36      "summary": "Các phụ huynh và thí sinh đang bày tỏ ý kiến về kì thi tuyển sinh đại học 2025, với mong muốn Bộ Giáo dục xem xét lại một số ph
37      "examples": [
38        "Là phụ huynh có con 12, bản thân có một số ý kiến như sau, mong Bộ xem xét. Thứ nhất: Hiện tại đa số học sinh đang còn mơ hồ trong việc
39        "Lướt thấy đa số phụ huynh và học sinh mong mỏi được tăng số lượng nv, chứ 10nv ít quá, nv sư phạm cũng nên tăng lên",
40        "Năm trước xét chung các phương thức và quy đổi điểm theo bách phân vị nhưng mà đấy là theo top % mà không xem số lượng thí sinh  có tương
41      ]
42    },
43    "ý kiến riêng": {
44      "count": 15,
45      "percent": "17.6%",
46      "summary": "Nhiều phụ huynh và thí sinh mong muốn Bộ Giáo dục và Đào tạo tăng số lượng nguyện vọng cho thí sinh trong kỳ thi tuyển sinh đại
47      "examples": [
48        "Mong BGD cho thêm số  lượng  Nguyện vọng cho thí sinh, 10 nguyện vọng hơi ít.",
49        "Mong Bộ cho thêm nguyện vọng, đặc biệt là nguyện vọng sư phạm ạ.",
50        "Em mong BGD xem lại số lượng 10 nv. Thực sự 10 là quá ít, có thể giới hạn nhưng k thể hạn hẹp vậy được. Nó chả khác nào đóng sập lại cán
51      ]
52    }
53  },
```


Hình 2.23. Bảng so sánh ví dụ bình luận Trung lập và Ý kiến riêng.

2.1.3.3.2. Cơ chế gọi API và xử lý lỗi

Hệ thống sử dụng Groq API với model Llama-3.3-70B-Versatile thông qua thư viện **aiohttp** để thực hiện các cuộc gọi bất đồng bộ. Để đảm bảo tính ổn định khi xử lý khối lượng lớn, hệ thống tích hợp các cơ chế sau:

- **Multi-account Key Rotation**: Hệ thống quản lý nhiều API key từ các tài khoản độc lập. Khi một key gặp lỗi 429 (rate limit) với thời gian chờ vượt ngưỡng 300 giây — dấu hiệu quota ngày đã cạn kiệt — hệ thống tự động chuyển sang key tiếp theo mà không làm gián đoạn luồng xử lý.
- **Exponential Backoff**: Với các lỗi tạm thời (503 Overloaded), hệ thống áp dụng chiến lược chờ lũy tiến, tránh spam API.
- **Resume tự động**: Mỗi file đầu ra được kiểm tra trước khi xử lý. Nếu đã tồn tại, hệ thống bỏ qua và tiếp tục với file tiếp theo, cho phép tiếp tục từ điểm dừng mà không mất dữ liệu đã xử lý.
- **Session Logging**: Toàn bộ quá trình chạy được ghi lại vào file **run_log.json**, bao gồm thời gian bắt đầu/kết thúc, số file đã xử lý, số file còn lại và lý do dừng (**completed** hoặc **quota_exhausted**).



Hình 2.24. Gợi ý minh họa: Sơ đồ luồng xử lý lỗi và key rotation.

2.1.3.3.3. Đầu ra phân tích — Hai cấp độ kết quả

Với mỗi sự kiện, hệ thống tạo ra hai file kết quả phục vụ các mục đích khác nhau:

File data_full_analyzed: Lưu trữ toàn bộ bình luận kèm nhãn chi tiết ở cấp độ từng comment và reply. Mỗi bình luận được bổ sung các trường:

- **emotion** hoặc **stance**: Nhãn phân loại cảm xúc/quan điểm.
- **is_trash**: Đánh dấu rác (bao gồm cả rác được phát hiện bởi LLM).
- **toxic**, **toxic_type**: Phát hiện nội dung độc hại và phân loại chi tiết.
- **thread_stats**: Thống kê phân phối cảm xúc trong cùng một luồng thảo luận, bao gồm cờ **controversial** đánh dấu luồng tranh cãi khi nhãn chiếm đa số dưới 70%.


```
{ "id_content": "showbiz_001",
"event_name": "Bình Gold – ồn ào về bài rap dung tục",
"total_comments": 13,
"emotion_stats": {
  "Phẫn nộ": 4, "Cà khịa": 6,
  "Đồng cảm": 0, "Ủng hộ": 1, "Trung lập": 2  },
"toxic_stats": {
  "total_toxic": 1,
  "công kích cá nhân": 1  },
"controversial_threads": [
  {  "comment_id": "cmt_showbiz_001_0011",
    "text": "Nhưng nhạc nó ra Chất thật :v",
    "thread_stats": {
      "emotions": {"Ủng hộ": 1, "Cà khịa": 2, "Phẫn nộ": 1},
      "controversial": true
    }   }
]  }
```


File data_analyzed (summary) của sự kiện showbiz_001 — thể hiện phân phối cảm xúc, toxic và luồng tranh cãi (Bản rút gọn).

File data_analyzed (Summary): Tổng hợp các chỉ số thống kê cấp sự kiện phục vụ dashboard và RAG, bao gồm:

- Phân phối cảm xúc/quan điểm theo số lượng và tỷ lệ phần trăm.
- Thống kê nội dung độc hại (riêng showbiz).
- Top 3 bình luận nổi bật theo điểm tương tác (score = likes + reply_count).
- Top đại diện cho mỗi nhãn cảm xúc.
- Danh sách luồng tranh cãi (controversial_threads).
- Kết luận tổng hợp xu thế dư luận do LLM sinh ra.

2.1.3.3.4. Sự khác biệt giữa Education và Showbiz

Pipeline xử lý về cơ bản là đồng nhất giữa hai lĩnh vực. Sự khác biệt chỉ nằm ở bộ nhãn phân loại và cấu trúc đầu ra, phản ánh đặc thù ngôn ngữ và mục tiêu phân tích của từng lĩnh vực:

Tiêu chí	Education	Showbiz
Bộ nhãn	Tích cực / Tiêu cực / Trung lập / Ý kiến riêng	Phẫn nộ / Cà khịa / Đồng cảm / Ủng hộ / Trung lập
Toxic detection	Không	Có (chửi bới / công kích cá nhân / người thân)
Phân tích bước 2	Có (phân biệt Trung lập vs Ý kiến riêng)	Không
Đặc thù ngôn ngữ	Lập luận chính sách, ít slang	Slang, emoji ẩn ý, ngôn ngữ Gen Z

Ví dụ mẫu:

	Education	Showbiz
Bình luận mẫu	*"Luật này hay nhưng nên sửa điều 5 cho rõ hơn"*	*"Sao nó xấu kinh dị vậy trời 🤦"*
Nhãn gán	stance: "Ý kiến riêng"	emotion: "Cà khịa"
Toxic	Không có	toxic: false
Mục tiêu	Phân tích quan điểm chính sách	Phân tích cảm xúc dư luận

2.1.3.4. Hệ thống nhãn phân loại dữ liệu (Data Labeling System)

Trong các bài toán Xử lý ngôn ngữ tự nhiên (NLP), việc định nghĩa hệ thống nhãn (Data Labeling) là bước then chốt giúp chuyển hóa văn bản phi cấu trúc thành các đặc trưng định lượng để AI có thể hiểu và phân tích. Đối với "Drama Intelligence System", do tính chất phức tạp của dư luận, hệ thống không sử dụng một bộ nhãn chung mà tiến hành thiết kế hai bộ nhãn phân loại (Taxonomy) độc lập cho hai lĩnh vực Giáo dục và Giải trí (Showbiz), kết hợp cùng các nhãn siêu dữ liệu (Meta-labels) để lọc nhiễu.

Nhãn Siêu dữ liệu (Meta-labels) và Cảnh báo luồng Trước khi đi vào phân loại ngữ nghĩa, mọi bình luận đều được gán các nhãn cấu trúc cơ bản:

- is_trash (Boolean): Nhãn đánh dấu bình luận rác (True) hoặc bình luận hợp lệ (False). Nhãn này được cấp thông qua bộ lọc Rule-based và bước 1 của LLM nhằm gạt bỏ các nội dung vô nghĩa, spam URL hoặc chỉ chứa tên người dùng.
- Controversial (Boolean): Nhãn đánh dấu luồng tranh cãi. Khác với nhãn cấp độ bình luận, nhãn này được gán ở cấp độ luồng (Thread). Nếu trong một cụm bình luận (gồm comment cha và các replies), không có nhãn thái độ nào chiếm tỷ lệ áp đảo trên 70%, luồng đó sẽ được gắn cờ controversial: true.

Bộ nhãn phân tích lĩnh vực Giáo dục (Education Domain) Ngôn ngữ thảo luận trong lĩnh vực Giáo dục thường mang tính lập luận, xoay quanh các chính sách hoặc sự kiện học đường. Do đó, hệ thống sử dụng bộ 4 nhãn (tập trung vào quan điểm - Stance):

- Tích cực (Positive): Các bình luận thể hiện sự đồng tình, ủng hộ hoặc khen ngợi các chính sách, quyết định của cơ quan giáo dục.
- Tiêu cực (Negative): Các ý kiến phản đối, chỉ trích, hoặc bày tỏ sự lo ngại trước các vấn đề, sự kiện học đường.
- Trung lập (Neutral): Những bình luận mang tính chất thờ ơ, chỉ cập nhật thông tin, hoặc không thể hiện rõ lập trường ủng hộ hay phản đối. (Ví dụ tiêu biểu: "Ô GD ô cũng có cái lý của ô. Vì nhiều ngành khác họ cũng tinh hoa dạng chuyên ngành của họ nên chắc dùng song song" -> Bình luận này chỉ nêu hiện trạng khách quan, được hệ thống gán nhãn "Trung lập").
- Ý kiến riêng (Personal Opinion / Mixed): Đây là nhãn đặc thù được xử lý ở Bước 2 (Phân tích sâu) của hệ thống. Nhãn này dành cho các bình luận có lập luận phức tạp: vừa ủng hộ một phần vừa phê phán phần khác, hoặc mang tính chất đề xuất, hiến kế giải pháp thay vì chỉ bộc lộ cảm xúc đơn thuần.

Bộ nhãn phân tích lĩnh vực Giải trí (Showbiz Domain) Trái ngược với tính học thuật của giáo dục, các luồng thảo luận Showbiz thường mang đậm tính cảm xúc, sử dụng nhiều từ lóng (slang), ngôn ngữ Gen Z và tính sát thương cao. Hệ thống áp dụng bộ 5 nhãn cảm xúc (Emotion) và 3 nhãn độc hại (Toxic):

Bộ 5 nhãn Cảm xúc (Emotions):

- Phẫn nộ: Sự tức giận, bức xúc, lên án mạnh mẽ hành vi của nhân vật công chúng.
- Cà khịa: Các bình luận mang tính mỉa mai, châm biếm sâu cay, đùa cợt nhưng có ý định chỉ trích ẩn dụ. (Ví dụ: "Sao nó xấu kinh dị vậy trời" -> Nhãn: Cà khịa).
- Đồng cảm: Thái độ thương xót, chia sẻ cảm xúc với nhân vật bị hại trong sự kiện.
- Ủng hộ: Các bình luận đứng ra bảo vệ, bênh vực hoặc khen ngợi nhân vật.
- Trung lập: Bình luận tóm tắt sự kiện, xin link, quan điểm khác, nhận định của họ hoặc hỏi han thông tin mà không kèm theo cảm xúc cá nhân.

Bộ nhãn phát hiện Độc hại (Toxic Detection): Để hỗ trợ việc kiểm duyệt không gian mạng, riêng lĩnh vực Showbiz được tích hợp thêm hệ thống dán nhãn Toxic. Nếu phát hiện vi phạm, bình luận sẽ được đánh cờ toxic: true và phân loại cụ thể vào một trong ba nhóm: Chửi bới (ngôn từ thô tục), Công kích cá nhân (tấn công trực tiếp nhân vật), hoặc Công kích người thân (xúc phạm gia đình, người không liên quan).

2.1.4. Phương pháp Trích xuất Từ khóa và Phân tích Cụm từ (Bag of Words & PMI)

Để nhận diện các chủ đề cốt lõi (Trending topics) từ hàng chục ngàn bình luận mạng xã hội, việc chỉ đếm tần suất từ đơn thuần (Bag of Words truyền thống) bộc lộ nhiều điểm yếu. Phương pháp này thường đẩy các từ phổ biến nhưng ít mang ý nghĩa đặc trưng lên thứ hạng cao. Để khắc phục, hệ thống "Drama Intelligence System" đã phát triển và triển khai kịch bản phân tích nâng cao analyze_keywords.py (phiên bản v4). Kịch bản này kết hợp kỹ thuật Túi từ vựng (Bag of Words) với chỉ số thống kê PMI (Pointwise Mutual Information), giúp khai phá chính xác các "từ khóa sự kiện" từ dữ liệu bình luận tiếng Việt.

Quá trình trích xuất và phân tích cụm từ được thực hiện qua 4 bước thuật toán chuyên sâu như sau:

2.1.4.1. Tiền xử lý và Chuẩn hóa từ vựng (Preprocessing & Stopwords)

Bước đầu tiên trong đường ống phân tích từ khóa là làm sạch văn bản để loại bỏ nhiễu. Hệ thống thực hiện các thao tác:

- Lọc từ dừng (Stopwords): Bình luận mạng xã hội mang đậm văn phong giao tiếp, chứa nhiều từ ngữ nối, từ cảm thán. Hệ thống định nghĩa một tập từ dừng (stopwords) mở rộng (bao gồm các từ như: "sao", "nữa", "nên", "đừng", "vui", "buồn"...) để gạt bỏ các yếu tố không mang giá trị ngữ nghĩa đặc trưng.

- Làm sạch ký tự (Cleaning): Thuật toán is_valid_token() tự động loại bỏ các từ có độ dài quá ngắn (dưới 3 ký tự), các đường dẫn liên kết (http, www), các chữ số và ký tự đặc biệt.

- Phân tách từ tiếng Việt (Tokenization): Khác với tiếng Anh (phân tách từ bằng khoảng trắng), tiếng Việt sở hữu nhiều từ ghép. Hệ thống tích hợp thư viện NLP tiếng Việt underthesea (hàm word_tokenize) để cắt câu chính xác. Quá trình base_tokenize() sẽ làm phẳng các từ ghép có dấu gạch dưới (Ví dụ: "giáo_dục" thành "giáo dục") để chuẩn bị cho bước tạo n-grams.

2.1.4.2. Khởi tạo N-grams bằng Cửa sổ trượt (Sliding Window)

Thay vì chỉ phân tích từng từ đơn lẻ, hệ thống sử dụng kỹ thuật Cửa sổ trượt (Sliding Window) để tạo ra các cụm từ (N-grams). Kỹ thuật này quét qua từng câu bình luận và bóc tách ra 3 nhóm từ vựng:

- Unigram: Các từ đơn lẻ (1 từ).
- Bigram: Các cụm 2 từ đi liền kề nhau.
- Trigram: Các cụm 3 từ đi liền kề nhau. Việc trích xuất Bigram và Trigram là bước đi quan trọng giúp hệ thống giữ lại được trọn vẹn bối cảnh ngữ nghĩa của các thuật ngữ chuyên ngành hoặc hành vi (Ví dụ: "thi tốt nghiệp", "trốn thuế").

```python
def sliding_window(seq, n):
    it = iter(seq)
    result = tuple(islice(it, n))
    if len(result) == n:
        yield result
    for elem in it:
        result = result[1:] + (elem,)
        yield result
# ── PMI computation (trả về cả score, không chỉ boolean) ──────────
def compute_pmi_scores(all_tokens: list[list[str]],
                       min_count: int = 5,
                       min_pmi: float = 1.0) -> dict[str, float]:
    """
    Tính PMI cho bigram và trigram.
    Trả về {phrase: pmi_score} cho các phrase vượt ngưỡng.
    """
    unigram_count: Counter = Counter()
    bigram_count:  Counter = Counter()
    trigram_count: Counter = Counter()
    total_uni = 0
    total_bi  = 0
    total_tri = 0

    for tokens in all_tokens:
        for tok in tokens:
            unigram_count[tok] += 1
            total_uni += 1
        for bi in sliding_window(tokens, 2):
            bigram_count[" ".join(bi)] += 1
            total_bi += 1
        for tri in sliding_window(tokens, 3):
            trigram_count[" ".join(tri)] += 1
            total_tri += 1

    if total_uni == 0:
        return {}

    pmi_scores: dict[str, float] = {}
```

Hình 2.25. Thuật toán trích xuất N-grams và tính toán PMI.

2.1.4.3. Thuật toán Pointwise Mutual Information (PMI)

Sau khi có danh sách N-grams, hệ thống áp dụng chỉ số PMI (Pointwise Mutual Information). PMI là một thước đo trong lý thuyết thông tin, dùng để đo lường khả năng hai hoặc ba từ xuất hiện cùng nhau có lớn hơn xác suất chúng xuất hiện một cách ngẫu nhiên và độc lập hay không.

Ví dụ: Từ "trốn" và từ "thuế" khi đứng độc lập có thể mang nhiều nghĩa, nhưng khi xuất hiện cạnh nhau với tần suất dày đặc trong một sự kiện, chỉ số PMI của cụm "trốn thuế" sẽ tăng vọt. Hệ thống thực hiện hàm compute_pmi_scores() để đếm tổng số lần xuất hiện của các tổ hợp. Để tối ưu hóa bộ nhớ và loại bỏ các cụm từ vô nghĩa ngẫu nhiên ghép với nhau, thuật toán thiết lập ngưỡng lọc khắt khe: chỉ các cụm bigram/trigram có tần suất xuất hiện tổng (Frequency) >= min_count (3 hoặc 5 lần) và có chỉ số PMI > min_pmi (1.0) mới được giữ lại để đánh giá.

2.1.4.4. Đếm tần suất và Công thức xếp hạng (Scoring & Ranking)

Ở lần quét thứ hai (hàm analyze_file()), thuật toán thực hiện đếm tần suất xuất hiện của các từ đơn (unigrams) và các cụm từ (bigrams/trigrams) đã vượt qua màng lọc PMI. Để tránh hiện tượng "đếm trùng" (double-counting), nếu một token đã là một phần của cụm bigram/trigram được chọn, nó sẽ bị vô hiệu hóa tư cách là một unigram độc lập. Hệ thống cũng đếm tổng số sự kiện (id_content) mà từ khóa đó xuất hiện để đánh giá độ phủ sóng.

Đặc điểm đáng chú ý của thuật toán nằm ở Công thức xếp hạng (Scoring): Score = Frequency × log2(1 + max(PMI, 0))

Công thức này giải quyết triệt để bài toán xếp hạng từ khóa bằng các cơ chế:

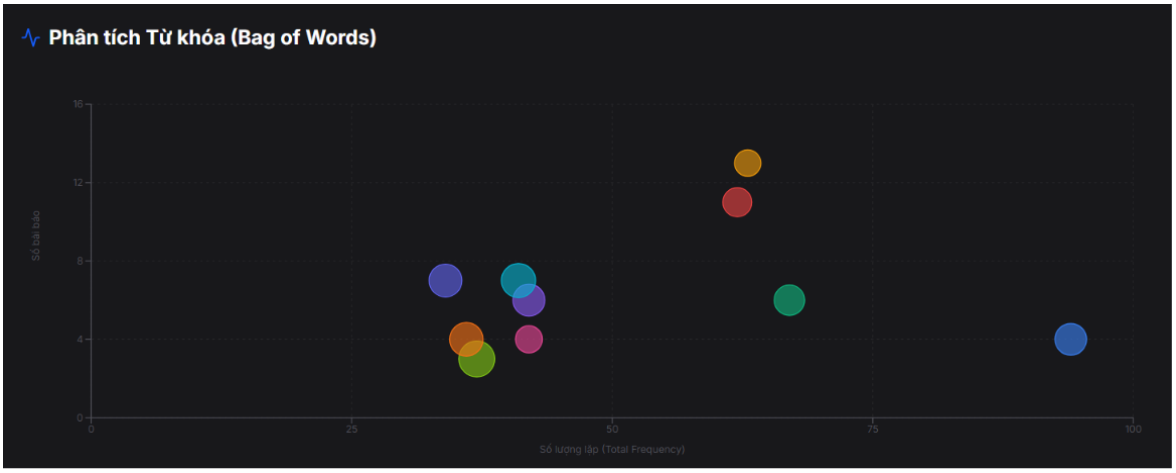
- Triệt tiêu Unigram: Đối với các từ đơn (unigram), giá trị PMI mặc định bằng 0. Khi đó log2(1 + 0) = 0, dẫn đến Score = 0. Cơ chế này cố tình đẩy các từ đơn lẻ xuống đáy bảng xếp hạng, ngăn chặn việc các từ thông dụng có tần suất cao (ví dụ: "chỉ", "cũng") chiếm lấy Top từ khóa.

- Khuếch đại Cụm từ ý nghĩa: Bằng cách nhân tần suất thực tế (Frequency) với giá trị logarit cơ số 2 của PMI, hệ thống tạo ra một lực đòn bẩy. Các cụm từ mang tính chất "sự kiện", "khủng hoảng truyền thông " (như "gian lận", "tống tiền", "trốn thuế") dù có tổng số lần được nhắc đến không cao bằng các từ phổ thông, nhưng nhờ hệ số PMI rất lớn, điểm Score của chúng sẽ được khuếch đại để vươn lên Top đầu bảng xếp hạng.

Hàm **top_keywords()** cuối cùng sẽ sắp xếp danh sách giảm dần theo điểm Score và trích xuất ra Top 10 từ/cụm từ đại diện nhất cho toàn bộ hệ sinh thái dữ liệu.

2.1.4.5. Phân tích kết quả trích xuất đặc trưng hai lĩnh vực

Hệ thống đã chạy thực nghiệm thuật toán trên toàn bộ 85 sự kiện, kết quả đã bóc tách rõ rệt sự khác biệt về mặt từ vựng và chủ đề quan tâm giữa hai luồng dư luận: Giáo dục và Showbiz.



Hình 2.26. Top 10 từ khóa cốt lõi trong lĩnh vực Showbiz.

2.1.4.6. Cấu hình --strip-names cho pipeline PySpark

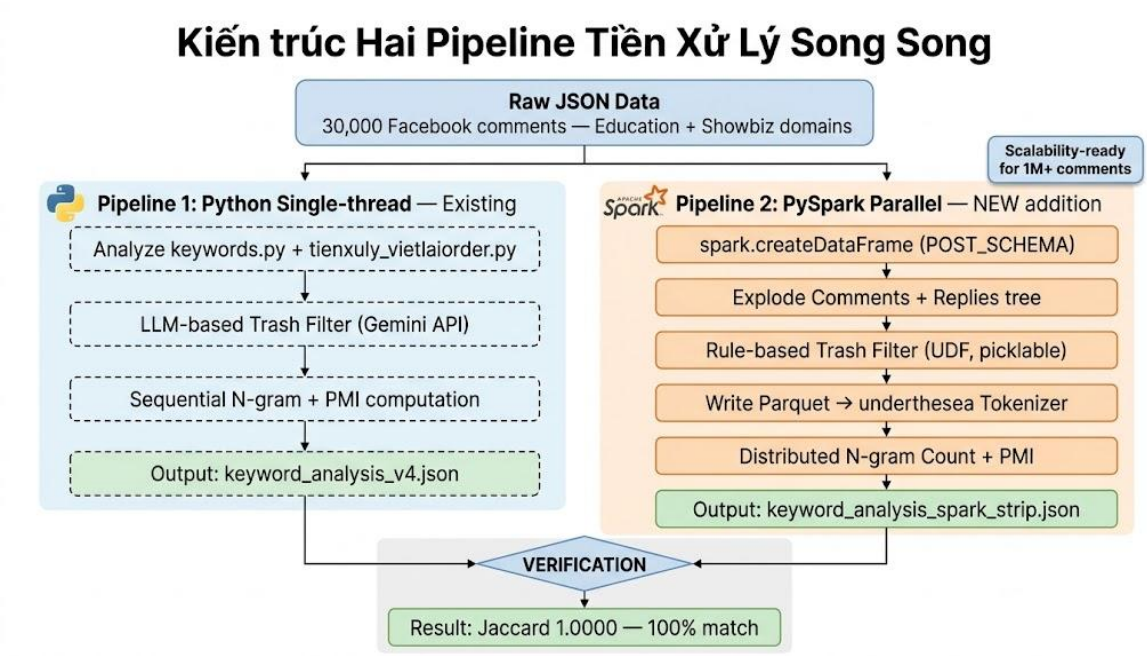
Trong quá trình trích xuất từ khóa bằng pipeline PySpark, hệ thống bổ sung thêm cấu hình --strip-names. Khi cấu hình này được kích hoạt, thuật toán tiền xử lý sẽ tự động nhận diện và loại bỏ các thẻ tag tên người dùng (mentions) ở đầu bình luận trước khi tiến hành cắt từ (tokenization).

Thực nghiệm cho thấy, cấu hình này giúp loại bỏ hiệu quả các từ khóa nhiễu mang tính cá nhân (ví dụ: "thùy vân", "đoàn đức thành") và nhường không gian cho các cụm danh từ mang giá trị phản ánh xu hướng, insight cao hơn như "ngành giáo dục", "trường đại học", "trường thpt". Cấu hình baseline (không strip) vẫn được lưu trữ trong codebase để phục vụ công tác đối chiếu chéo.


```
"keyword": "thùy vân",
"ngram_type": "bigram",
"total_frequency": 30,
"total_articles": 1,
"pmi_score": 12.45,
"score": 112.5,
"details": {
    "education_017": 30
}
```


Hình 2.27. Dữ liệu được xử lý nếu không có "strip".

2.1.4.7. Sơ đồ kiến trúc hai pipeline song song



Hình 2.28. Kiến trúc 2 pipeline xử lý song song.

2.1.5. Tổng hợp dữ liệu

Sau khi toàn bộ sự kiện được phân tích, file analyze_insights.py thực hiện aggregation ba cấp độ để tạo ra bức tranh tổng quan:

Cấp độ sự kiện: Với mỗi sự kiện, hệ thống tổng hợp tổng bình luận thô, số bình luận sạch sau lọc và tỷ lệ rác. Các sự kiện được xếp hạng theo mức độ quan tâm dựa trên tổng bình luận sạch, tạo ra danh sách Top 10 sự kiện nổi bật.

Cấp độ lĩnh vực: Hệ thống tổng hợp phân phối cảm xúc/quan điểm theo tỷ lệ phần trăm cho toàn bộ sự kiện trong lĩnh vực. Riêng showbiz có thêm thống kê tổng nội dung độc hại phân loại theo từng nhóm. Ngoài ra, hệ thống phân bổ số lượng sự kiện theo năm để phản ánh xu hướng thu thập dữ liệu theo thời gian.

Cấp độ tổng thể (Overall): Hợp nhất số liệu từ cả hai lĩnh vực, cho ra các chỉ số toàn hệ thống gồm tổng số sự kiện thu thập, tổng lượng bình luận, tỷ lệ lọc rác và phân bổ theo lĩnh vực.

```
{
 "overall": {
  "total_events": 85,
  "total_comments": {
   "raw": 32076,
   "clean": 29622,
   "trash": 6456,
   "trash_rate": "17.9%"
  },
  "by_category": {
   "education": {"events": 40, "comments_clean": 5370},
   "showbiz":   {"events": 45, "comments_clean": 24252}
  }
 }
}
```


Chú thích: Phần overall của insights.json — tổng hợp toàn hệ thống

Kết quả đầu ra được lưu vào file insights.json với cấu trúc phân cấp rõ ràng ba phần overall, education, showbiz, sẵn sàng phục vụ dashboard phân tích và hệ thống chatbot RAG.

2.2. Xây dựng cơ sở dữ liệu Embedding và AI Agent

2.2.1. Xây dựng cơ sở dữ liệu Embeddings

Mục tiêu: Chuyển đổi dữ liệu sự kiện drama giáo dục từ JSON thô sang vector embeddings và lưu trữ vào Vector Store để phục vụ RAG.

Công nghệ sử dụng:

Thành phần	Công nghệ	Chi tiết
Orchestration	n8n	Visual workflow automation
Embedding Model	Google Gemini	models/embedding -002
Vector Database	Supabase (pgvector)	Table: documents/ showbiz_documents
Data Source	GitHub Raw URL	education.json/ showbiz.json
IVFFlat	pgvector (vector columns trong Postgres)	Tạo bảng documents với embedding vector (3072) và tạo ivfflat index trên embedding vector_cosine_ops.

Luồng xử lý:

Bước 1: embedding

GitHub (education.json)
↓ HTTP Request
Parse JSON string → array
↓ Code in JavaScript
Load từng cục data
↓ Default Data Loader
Embed từng chunk
↓ Gemini Embedding Model
Lưu vector vào Supabase
↓ Supabase Vector Store (insert)

Bước 2: Retrieval (Truy xuất)

Câu hỏi người dùng
Câu hỏi
↓
Lấy câu hỏi
↓ (Extract Query)
Xác định câu hỏi, điều khiển luồng
↓ AI Agent điều phối
Tìm kiếm Top-10 Chunks tương đồng
↓ Gemini Embedding Model
↓ Supabase Vector Store (retrieve)

Bước 3: Reranking (Tái xếp hạng) & Phản hồi

Top-10 Chunks từ Supabase
↓ HTTP/Internal Link Tính toán độ liên quan ngữ nghĩa
↓ Reranker Cohere Lọc lấy Top-3 kết quả tốt nhất
↓ Reranker logic Tổng hợp thông tin và kiểm chứng
↓ Agent chọn lọc data Tạo câu trả lời cuối cùng (Văn xuôi)
↓ LLMs (Llama 3.3 via Groq)

- Lí do sử dụng : Vì dùng LLMs đọc data đơn thuần gặp khó khăn với token, thời gian nên dùng RAG + Reranking

- So sánh giữa agentic RAG và RAG + reranhking
    + Full-file Retrieval (cũ):
        Token rất cao, chậm, nhưng độ chính xác cao.

    + RAG thuần:
        Token thấp, nhanh, nhưng độ chính xác giảm.

    + RAG + Reranking:
        Token thấp + nhanh + độ chính xác gần bằng full-file.
        Trước hết thì cần phải tạo được database trong supabase vector store
        Đầu tiên cần tạo bảng Database trước

Documents with Vector Embeddings | Vector Similarity Document Search Function | Document Similarity Matching Function

```sql
drop table if exists documents;

create table documents (
  id bigserial primary key,
  content text,
  metadata jsonb,
  embedding vector(3072)
);
create index if not exists documents_embedding_idx
on documents using ivfflat (embedding vector_cosine_ops);
```

Hình 2.29. Tạo bảng cho dữ liệu education.

Sau khi đã tạo bảng rồi thì sẽ tạo function:

Documents with Vector Embeddings | Vector Similarity Document Search Function | Document Similarity Matching Function

```sql
create or replace function public.match_documents(
  filter jsonb default '{}'::jsonb,
  match_count integer default 10,
  query_embedding public.vector default null
)
returns table (
  id bigint,
  content text,
  metadata jsonb,
  similarity double precision
)
language sql
stable
as $$
  select
    d.id,
    d.content,
    d.metadata,
    (1 - (d.embedding <=> query_embedding))::double precision as similarity
  from public.documents d
  where query_embedding is not null
  order by d.embedding <=> query_embedding
  limit match_count;
$$;
```

Hình 2.30. Tạo chức năng query từ bên ngoài cho bảng.

Tạo một hàm khác (cũng tên match_documents, nhưng là ở public schema và signature tham số khác) để:

- Nhận filter, match_count, query_embedding (embedding có thể null)
  - Luôn truy cập rõ:
  - bảng public.documents
- schema function là public.match_documents
- Trả ra id, content, metadata, similarity với công thức:
  - (1 - (d.embedding <=> query_embedding))
- Có điều kiện:
  - where query_embedding is not null (để tránh gọi với embedding rỗng)

Nói ngắn gọn: Đoạn 2 dùng để semantic search Top N dựa theo cosine distance, nhưng phần lọc metadata có thể chưa được implement.

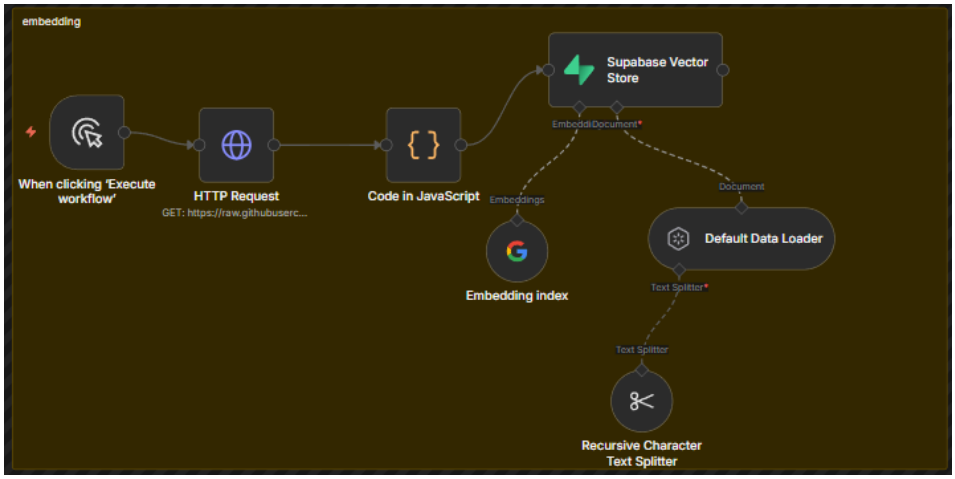
2.2.2. RAG + Reranking

Sau khi đã có bảng thì embedding dữ liệu,cụ thể quá trình thực hiện. Cụ thể, quá trình thực hiện của nó gồm 3 bước tổng thể như sau:

Bước 1: Embedding.

JSON → chunks → embed → Supabase

Load dataset từ github về, sau đó đổ qua file code. Ở đây, nó sẽ có nhiệm vụ convert data từ json qua string để đưa vào máy embedding. Sau khi đã lấy xong, nó sẽ đi qua Supabase vector store để embedding và mã hóa dữ liệu theo 3072 trường dữ liệu. Sử dụng model Gemini embedding 2 để embedding dữ liệu.



Hình 2.31. Luồng tạo bảng dữ liệu embedding.

Bước 2: Retrieval

Câu hỏi → embed → tìm top-10 chunks trong Supabase

Xử lý câu hỏi: Khi người dùng nhập câu hỏi (Query), hệ thống sẽ đưa câu hỏi này qua máy embedding sử dụng cùng model Gemini embedding 2. Câu hỏi sẽ được vector hóa thành một chuỗi số có cùng kích thước 3072 trường dữ liệu để đảm bảo tính đồng nhất.

Vector Search: Hệ thống thực hiện phép toán so sánh Cosine Similarity giữa vector câu hỏi và hàng triệu vector đã lưu trong Supabase.

Lấy dữ liệu thô: Dựa trên độ tương đồng ngữ nghĩa, Supabase sẽ thực hiện truy vấn và trả về top-10 chunks (đoạn văn bản) có điểm số cao nhất. Đây là những ứng viên tiềm năng nhất chứa câu trả lời nhưng vẫn có khả năng bị nhiễu do giới hạn của việc so sánh vector thuần túy.

Bước 3: Reranking

Top-10 chunks → Reranker cohere → chỉ giữ top-3 → Agent chọn 1 cái và đưa ra thông tin cuối cùng → LLMs trả lời.

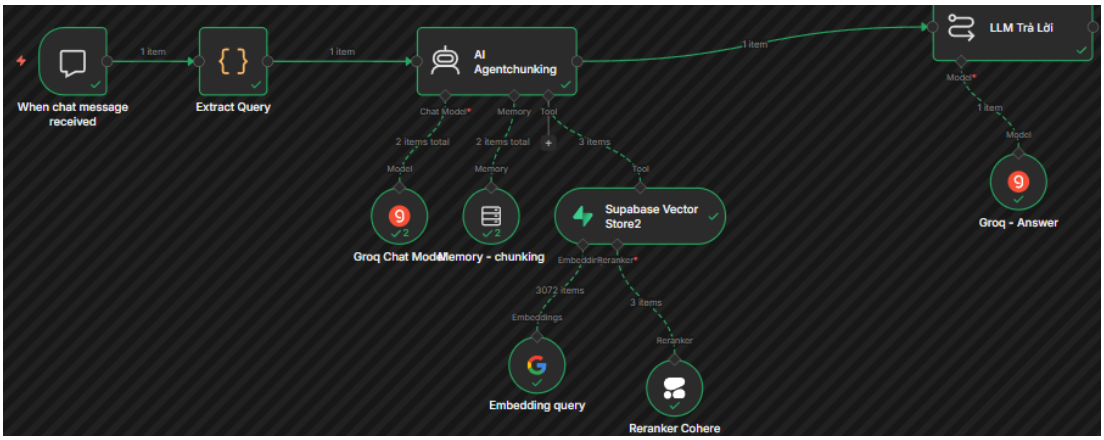
Đây là giai đoạn tinh lọc để đảm bảo độ chính xác tối đa trước khi đưa dữ liệu cho LLM:

Reranking với Cohere: 10 chunks vừa lấy được từ Bước 2 sẽ được đưa vào Cohere Reranker. Khác với embedding chỉ so sánh tọa độ, Reranker sẽ đọc trực tiếp nội dung văn bản và so sánh ngữ nghĩa chuyên sâu hơn giữa câu hỏi và từng chunk.

Tối ưu hóa lựa chọn: Sau khi đánh giá lại, hệ thống chỉ giữ lại top-3 chunks có điểm số cao nhất sau khi rerank. Điều này giúp loại bỏ hoàn toàn các thông tin gây nhiễu hoặc sai lệch.

Agent Decision: Các chunks này được chuyển qua một Agent (đóng vai trò điều phối). Agent sẽ phân tích và chọn ra 1 hoặc các thông tin phù hợp nhất, kết hợp với ngữ cảnh để đưa ra "prompt" cuối cùng.

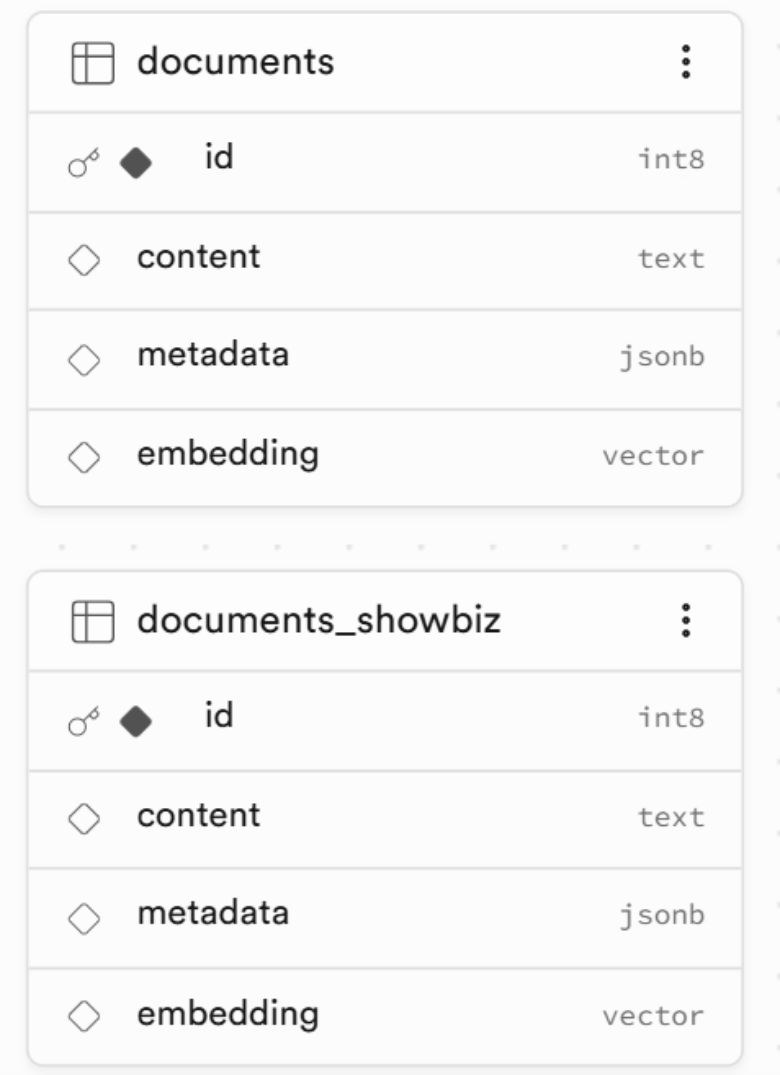
LLM Generation: Cuối cùng, dữ liệu được chuyển đến LLM (mô hình ngôn ngữ lớn). Note “LLM trả lời” sẽ xử lý thông tin đã được chọn lọc kỹ lưỡng để tạo ra câu trả lời chính xác, tự nhiên và đúng trọng tâm nhất cho người dùng.



Hình 2.32. Luồng tạo truy xuất dữ liệu embedding.

Thông số kỹ thuật:

Bảng: documents (education), documents_showbiz (showbiz).
Số chiều vector: 3072 (gemini-embedding-002).
File vector tải về: 261120 dòng (raw vector values).
Rows trong Supabase: 40 cột (education) và 45 cột (showbiz).
Cột lưu trữ: id, content, metadata, embedding.



Hình 2.33. Mô tả bảng dữ liệu trong supabase vector store.

Vấn đề gặp phải trong quá trình:

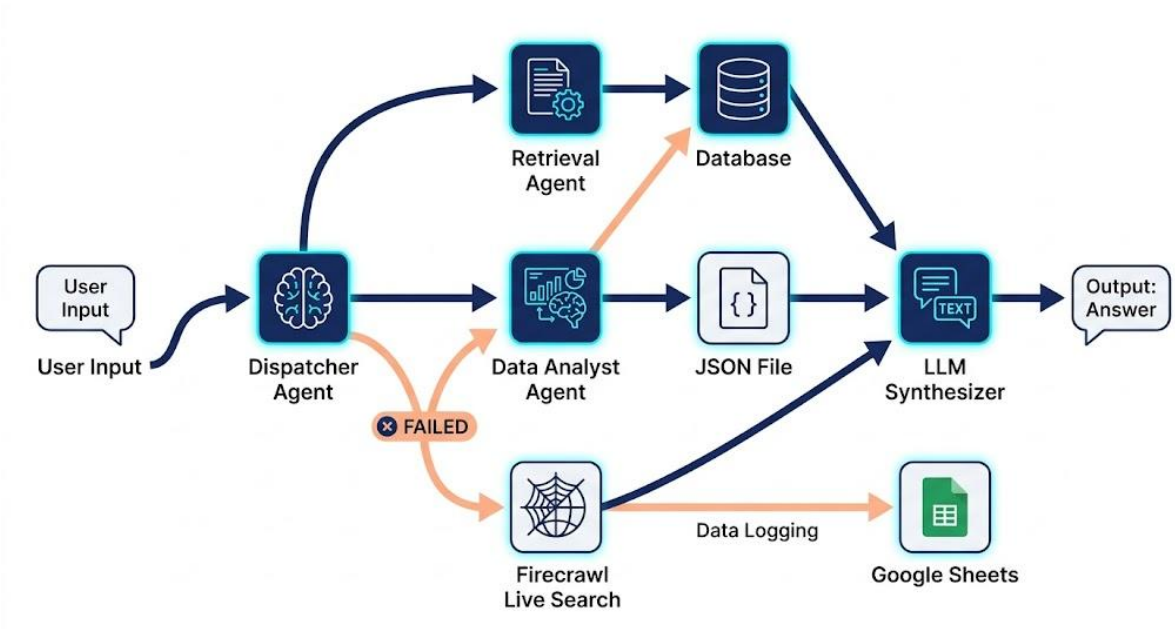
1. HTTP Request trả về data dạng string lồng JSON → cần parse thêm trong Code node
2. Default Data Loader không nhận main input trực tiếp → lấy data qua {{ $json.text }}
3. Quá trình cắt dữ liệu (chunking) gặp vấn đề khi lấy số lượng tối đa là 200, dẫn đến việc phân tách sai dữ liệu. Cách khắc phục: Thay thế việc cắt cố định bằng Node tùy chỉnh, thiết lập chunk_size = 7000

CHƯƠNG 3. TRIỂN KHAI VÀ TÍCH HỢP HỆ THỐNG

3.1. Agent n8n

3.1.1. Hệ thống Multi-Agent

Hệ thống được thiết kế theo kiến trúc Multi-Agent (Đa tác nhân) kết hợp với các chuỗi LLM tĩnh (LLM Chain) nhằm đảm bảo tốc độ phản hồi, tối ưu chi phí và độ chính xác của thông tin.



Hình 3.1. Sơ đồ luồng Agent của n8n.

Phân loại và vai trò của các AI Agents & LLM

Hệ thống sử dụng 3 Tác nhân AI (Agents) chuyên trách cho khâu điều phối và truy xuất, cùng 1 LLM Chain cho khâu tổng hợp cuối cùng:

A. **Router Agent (Agent Điều phối):** Nằm ở đầu luồng, đóng vai trò như "não bộ" phân tích ý định (intent) của người dùng. Agent này quyết định rẽ nhánh câu hỏi vào kho dữ liệu Giáo dục (Education), Giới giải trí (Showbiz), Thống kê (Insights), hoặc từ chối nếu là tin nhắn rác.

Prompt:

> *Bạn là AI Điều phối (Dispatcher Agent) của Drama Intelligence System - Trợ lý thông minh chuyên tổng hợp và phân tích dư luận mạng xã hội Việt Nam.*

Nhiệm vụ DUY NHẤT của bạn là đọc tin nhắn của người dùng, phân tích ý định (intent), quyết định luồng xử lý và xuất ra CHÍNH XÁC một đối tượng JSON. KHÔNG giải thích thêm, KHÔNG in ra văn bản ngoài định dạng JSON.

1. HIỂU VỀ CƠ SỞ DỮ LIỆU HIỆN CÓ CỦA HỆ THỐNG

Hệ thống có 3 tệp dữ liệu chính:

- `education`: Chứa chi tiết sự kiện drama giáo dục, nhân vật liên quan, phân tích cảm xúc bình luận (tích cực/tiêu cực/trung lập/rác) và bình luận tiêu biểu.

- `insights`: Chứa số liệu thống kê toàn hệ thống (tổng số sự kiện, tỷ lệ bình luận rác - trash_rate, top sự kiện nhiều tương tác nhất...).

- `showbiz`: LƯU Ý QUAN TRỌNG: Hiện tại dữ liệu showbiz chưa được cập nhật.

2. QUY TẮC PHÂN LUỒNG (DECISION)

Phân loại tin nhắn vào 1 trong 3 nhánh sau:

- "branch_greeting": Khi user chào hỏi đơn thuần (hi, hello), nhắn tin rác/chửi bới, hoặc hỏi thông tin KHÔNG LIÊN QUAN đến drama (ví dụ: giá vàng, thời tiết, viết code).

- "branch_database": Khi user hỏi về drama/sự kiện có thể nằm trong `education`, hoặc hỏi số liệu thống kê chung (`insights`), hoặc hỏi về drama `showbiz`.

- "branch_firecrawl": Khi user hỏi rõ về một drama cụ thể rất mới mà hệ thống chưa cập nhật, HOẶC yêu cầu tra cứu live trên mạng internet.

3. CẤU TRÚC JSON ĐẦU RA BẮT BUỘC

{

"reasoning": "Giải thích ngắn gọn tại sao chọn nhánh này và trích xuất entity.",

"decision": "branch_greeting" | "branch_database" | "branch_firecrawl",

"type": "greeting" | "spam" | "unrelated" | "education_drama" | "showbiz_drama" | "system_insight" | "need_live_search",

"response_data": "QUY TẮC ĐIỀN: Nếu branch_greeting -> Viết câu chào/từ chối lịch sự. Nếu type là showbiz_drama -> BẮT BUỘC điền: 'Hiện tại dữ liệu showbiz chưa được cập nhật, mình sẽ sớm fix sau nhé! Bạn có muốn mình dùng tính năng Live Search để cào thông tin mới nhất trên mạng không?'. CÁC TRƯỜNG HỢP KHÁC -> Để chuỗi rỗng \"\"",

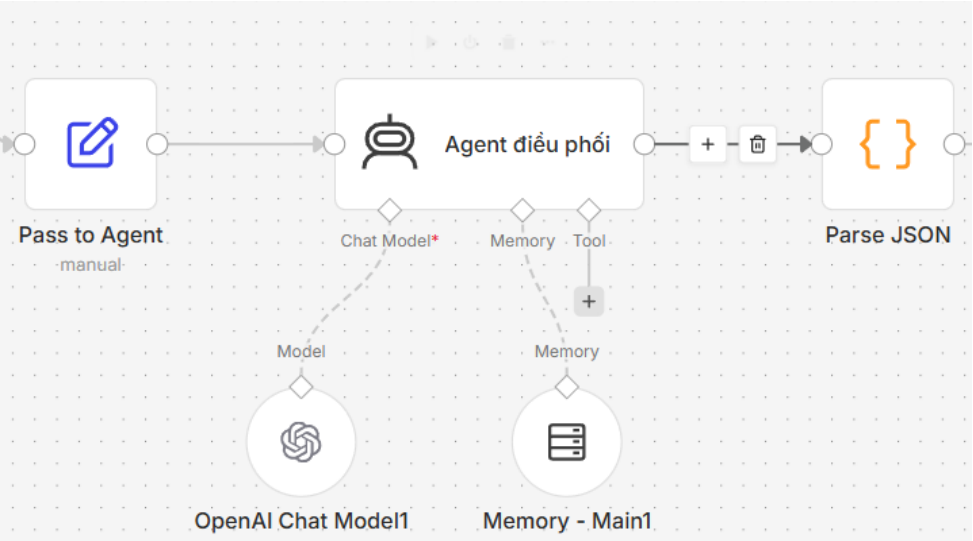
"routing_info": {

"database_needed": "education" | "insights" | "showbiz" | "none",

"entities": ["từ khóa trọng tâm 1", "tên nhân vật 2"]

}

}



Hình 3.2. Sơ đồ luồng Agent Điều Phối của n8n.

B. Retrieval Agent (Tác nhân Truy xuất): Nhận lệnh từ Dispatcher, Agent này sử dụng công cụ Supabase Vector Store để tìm kiếm và trích xuất các đoạn văn bản (chunks) liên quan nhất đến từ khóa. (Gồm 2 luồng song song cho Education và Showbiz).

Prompt:

Nhiệm vụ: Truy xuất dữ liệu từ Supabase để phục vụ Reranking.

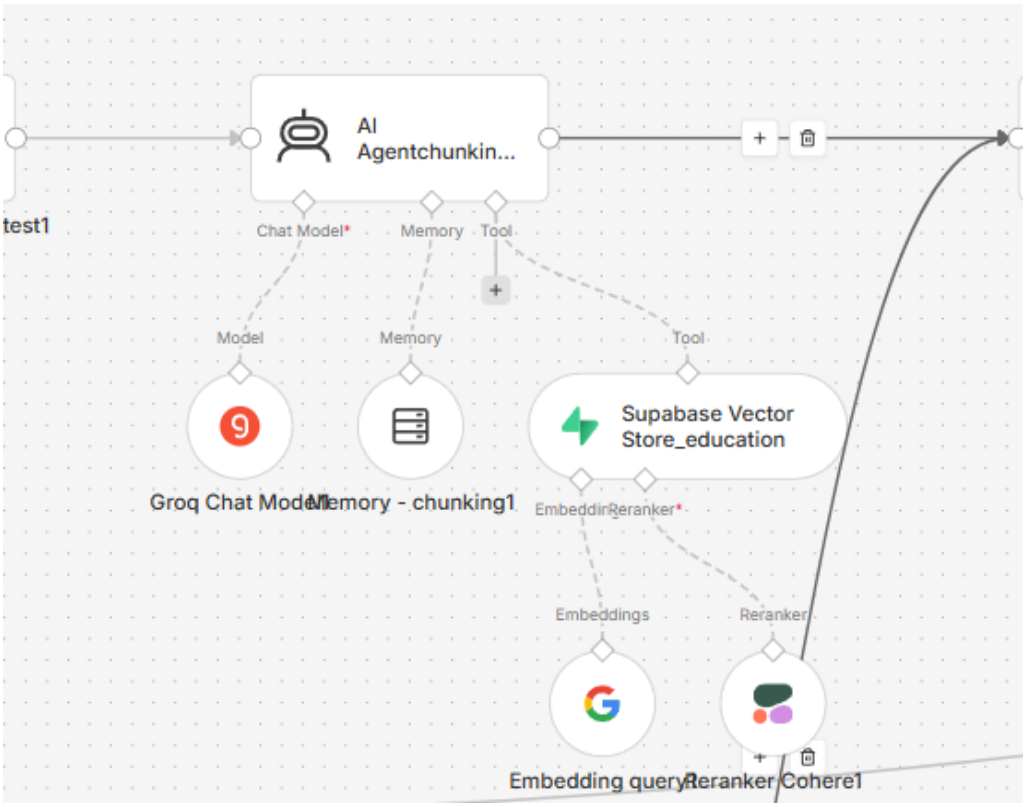
Quy trình:

1. Trích xuất từ khóa chính từ câu hỏi (ví dụ: "Hưng Yên", "KHKT", "đạo nhái").

2. Sử dụng tool Supabase Vector Store để tìm kiếm. Nếu không thấy, thử lại với từ khóa ngắn hơn.

3. OUTPUT: Phải trả về danh sách văn bản thô của các chunks tìm được.

Lưu ý: Không được tự trả lời người dùng, không thêm lời dẫn. Nếu không có dữ liệu, chỉ trả về chuỗi: "Không tìm thấy dữ liệu".



Hình 3.3. Agent Chunking education.

C. **Insight Analyst Agent (Tác nhân Phân tích dữ liệu):** Hoạt động độc lập cho nhánh Insights. Agent này trực tiếp đọc file insights.json từ kho lưu trữ GitHub, tự tính toán, trích xuất số liệu thống kê và trả lời trực tiếp cho người dùng.

Prompt:

Bạn là Chuyên gia Phân tích Số liệu (Data Analyst Agent) của hệ thống Drama Intelligence System.

QUY TRÌNH THỰC HIỆN BẮT BUỘC:

Sử dụng công cụ "Get insight" để lấy dữ liệu mới nhất từ file data/insights.json.

Đọc, hiểu và trích xuất các con số thống kê tổng quan (như tổng số sự kiện, tỷ lệ rác/spam, top drama nhiều tương tác...) từ file JSON này.

Trả lời câu hỏi của người dùng dựa TRÊN ĐÚNG những số liệu vừa lấy được.

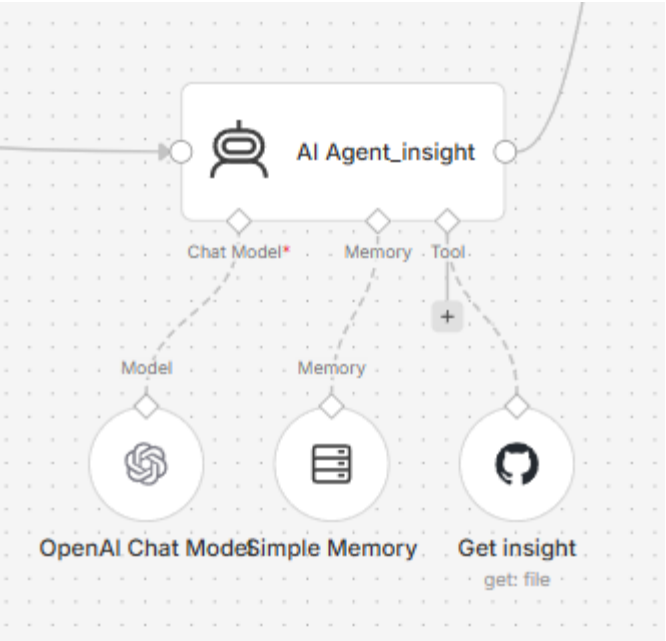
NGUYÊN TẮC TRẢ LỜI:

TÍNH CHÍNH XÁC: Tuyệt đối không tự bịa ra số liệu. Nếu người dùng hỏi một chỉ số không có trong file, hãy thành thật trả lời là hệ thống chưa đo lường chỉ số đó.

*ĐỊNH DẠNG: Trình bày dạng văn xuôi mạch lạc, tự nhiên. TUYỆT ĐỐI KHÔNG dùng ký tự markdown như **, #, _, hoặc gạch đầu dòng.*

XUỐNG DÒNG: Dùng phím Enter để ngắt đoạn, giúp người dùng dễ đọc trên điện thoại.

VĂN PHONG: Khách quan, chuyên nghiệp như một bản báo cáo nhanh.



Hình 3.4. Agent phân tích insight.

D. Final LLM (Mô hình Tổng hợp câu trả lời): Không dùng cơ chế Agent, đây là một chuỗi LLM tĩnh (Chain LLM) nhận đầu vào là bối cảnh (nguyên văn các chunks từ database hoặc từ web) và câu hỏi của người dùng để đúc kết ra câu trả lời cuối cùng.

Prompt:

NGUYÊN TẮC BẮT BUỘC:

1. Chỉ trả lời ĐÚNG dựa trên thông tin nhận được. Tuyệt đối không tự suy diễn.

2. Hỏi về sự kiện/nhân vật → Chỉ tóm tắt diễn biến chính trong tối đa 5 câu. KHÔNG đưa số liệu dư luận vào trừ khi được hỏi.

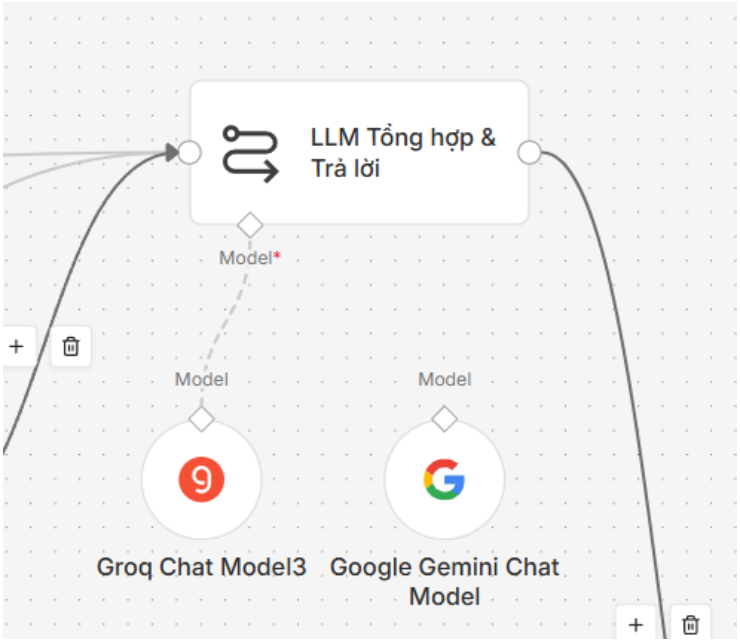
3. Hỏi về dư luận/cảm xúc → Mới trích xuất số liệu cảm xúc và bình luận tiêu biểu.

4. Hỏi chung chung → Tóm tắt ngắn gọn, cuối hỏi người dùng muốn biết thêm gì: diễn biến chi tiết, các bên liên quan, hay phản ứng dư luận.

5. GIỚI HẠN ĐỘ DÀI: Tối đa 150 từ mỗi câu trả lời. Nếu thông tin nhiều, hỏi người dùng muốn đào sâu phần nào.

*6. TUYỆT ĐỐI KHÔNG dùng ký tự markdown *, #, _, gạch đầu dòng.*

7. Trình bày văn xuôi tự nhiên, xuống dòng bằng Enter.



Hình 3.5. LLM trả lời kết quả.

Tiêu chí	AI Agent (Tác nhân AI)	Final LLM (Chuỗi LLM tĩnh)
Cơ chế hoạt động	Suy luận đa bước (ReAct: Reason + Act), tự quyết định dùng công cụ nào, lặp lại cho đến khi có kết quả.	Nhận Prompt cố định + Dữ liệu đầu vào (Context) $\rightarrow$ Trả kết quả ngay lập tức (One-pass).
Tốc độ phản hồi	Chậm (Phải tốn thời gian suy nghĩ, chọn tool, gọi tool, đọc kết quả).	Rất nhanh (Chỉ tốn thời gian sinh text từ context có sẵn).
Tiêu hao Token (Chi phí)	Rất lớn. Mỗi vòng lặp suy nghĩ (thought loop) đều tốn token cho toàn bộ lịch sử hội thoại.	Rất thấp. Cấu trúc Prompt ngắn gọn, cố định, không có chu trình đệ quy.
Vai trò trong hệ thống	Thích hợp để tìm kiếm, phân tích, quyết định luồng đi.	Thích hợp để đọc tóm tắt, tinh chỉnh văn phong và giao tiếp với con người.

Bảng 3.1. Bảng so sánh và Lý do lựa chọn LLM Chain thay vì Agent cho khâu trả lời tin nhắn.

Giải thích quyết định thiết kế:

Việc chỉ sử dụng LLM Chain ở khâu trả lời cuối cùng là một quyết định tối ưu hóa kiến trúc có chủ ý. Ở bước này, dữ liệu đã được các Retrieval Agents hoặc Firecrawl chuẩn bị sẵn. Nếu tiếp tục dùng Agent, hệ thống sẽ bị thừa thãi (Over-engineering), AI có thể rơi vào vòng lặp vô tận (infinite loop) khi cố tìm kiếm công cụ không cần thiết. Một con LLM tĩnh nhận đầu vào và trả lời trực tiếp sẽ giúp **giảm độ trễ (latency) xuống mức thấp nhất, tiết kiệm hàng ngàn token cho mỗi lượt chat**, mang lại trải nghiệm mượt mà cho người dùng.

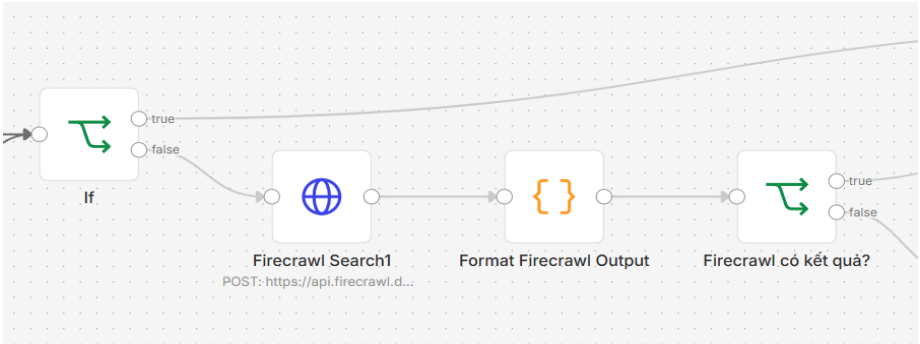
3.1.2. Cơ chế chống Ảo giác (Anti-Hallucination) và Data Loop bằng Firecrawl

Một thách thức lớn của các hệ thống LLM là hiện tượng "Ảo giác" (Hallucination) - tự bịa đặt thông tin khi không biết câu trả lời. Dự án khắc phục triệt để vấn đề này qua cơ chế dự phòng (Fallback Mechanism) nhiều lớp:

- **Rào chắn Prompting:** Các Retrieval Agents được thiết lập quy tắc khắt khe: *Nếu không tìm thấy dữ liệu trong Database, tuyệt đối không tự trả lời mà phải xuất ra chuỗi "Không tìm thấy dữ liệu".* Tuy nhiên, phản hồi này chưa đáp ứng được kỳ vọng của người dùng cuối, do đó hệ thống cần một cơ chế dự phòng phù hợp hơn.

- **Tích hợp Live Search (Firecrawl):** Khi hệ thống nhận được tín hiệu "Không tìm thấy dữ liệu", luồng (Node If) sẽ lập tức chuyển hướng sang API của Firecrawl. Tại đây, Firecrawl sẽ cào dữ liệu trực tiếp trên Internet (các bài báo, mạng xã hội mới nhất) theo thời gian thực để đưa vào cho Final LLM tổng hợp, đảm bảo người dùng luôn nhận được câu trả lời đầy đủ và cập nhật nhất.

- **Vòng lặp Dữ liệu (Data Loop) với Google Sheets:** Hệ thống không chỉ trả lời xong là bỏ qua. Những câu hỏi phải kích hoạt Firecrawl (tức là Database hiện tại đang thiếu sự kiện này) sẽ được tự động ghi nhận (Log) vào Google Sheets (Tên sự kiện, ID, Thể loại). Cơ chế này giúp nhóm phát triển dễ dàng theo dõi các khoảng trống dữ liệu, từ đó tiến hành thu thập và cập nhật Database cho tương lai, giúp hệ thống ngày càng thông minh hơn.



Hình 3.6. FireCrawl Pipeline.

3.2. Triển khai Website tin tức tổng hợp và Hệ thống Chatbot

3.2.1.Triển khai website

3.2.1.1. Xây dựng Frontend và Backend

Thông Tin Cơ Bản

Thông Tin	Chi Tiết
Tên Dự Án	Drama News App
Loại	Single Page Application (SPA)
Build Tool	Vite
Framework Chính	React 19.2.0

3.2.1.2. Kiến trúc hệ thống

3.2.1.2.1. Sơ Đồ Cấu Trúc

```
code_web/
├── src/
│   ├── pages/
│   │   ├── HomePage/          (Trang chủ)
│   │   ├── FacebookPage/      (Trang Facebook)
│   │   └── NewsPage/          (Trang Tin tức)
│   ├── components/
│   │   ├── Header/        (Thanh header)
│   │   ├── Footer/        (Chân trang)
│   │   ├── PostCard/      (Thẻ bài viết)
│   │   └── Pagination/    (Phân trang)
│   ├── services/
│   │   └── api.js         (API client)
│   ├── styles/            (Stylesheet toàn cục)
│   └── utils/             (Utility functions)
├── public/                (Static assets)
└── package.json           (Phụ thuộc dự án)
```

3.2.1.2.2. Luồng Dữ Liệu

1. API Service → Tải dữ liệu từ backend API
2. Pages → Xử lý logic từng trang
3. Components → Hiển thị giao diện
4. Styles → SCSS modules cho styling

3.2.1.3. Các công nghệ sử dụng

3.2.1.3.1. Core Dependencies

Công Nghệ	Phiên Bản	Mục Đích
React	19.2.0	Framework UI
React DOM	19.2.0	Rendering DOM
React Router DOM	7.13.1	Client-side routing
Vite	8.0.0-beta.13	Build tool & dev server
SASS	1.98.0	CSS preprocessor
Ant Design	6.3.3	UI Component library

3.2.1.3.2. Dev Dependencies

- **ESLint** 9.39.1 - Linting & code quality
- **Eslint-plugin-react-hooks** - Kiểm tra React hooks
- **@vitejs/plugin-react** - React integration cho Vite

3.2.1.3.3. Triển khai Apps Script

3.2.1.3.3.1. Giới thiệu

Trong quá trình phát triển hệ thống web, việc xây dựng backend để cung cấp dữ liệu là một bước quan trọng. Tuy nhiên, đối với các dự án nhỏ hoặc prototype, việc triển khai server riêng (NodeJS, Python, v.v.) có thể gây tốn thời gian và tài nguyên.

Vì vậy, nhóm đã lựa chọn sử dụng **Google Apps Script (GAS)** kết hợp với **Google Sheets** để xây dựng một API đơn giản, nhanh chóng và dễ triển khai.

3.2.1.3.3.2. Mục tiêu

- Tận dụng Google Sheets làm cơ sở dữ liệu.
- Sử dụng Google Apps Script để xử lý và cung cấp dữ liệu dạng API.
- Kết nối dữ liệu với frontend (React).
- Hỗ trợ hiển thị danh sách bài viết (posts) theo nhiều nền tảng (Facebook, News).

3.2.1.3.3.3. Kiến trúc hệ thống

Google Sheets (Data)
↓
Google Apps Script (API)
↓
Web App (React)

Thành phần:

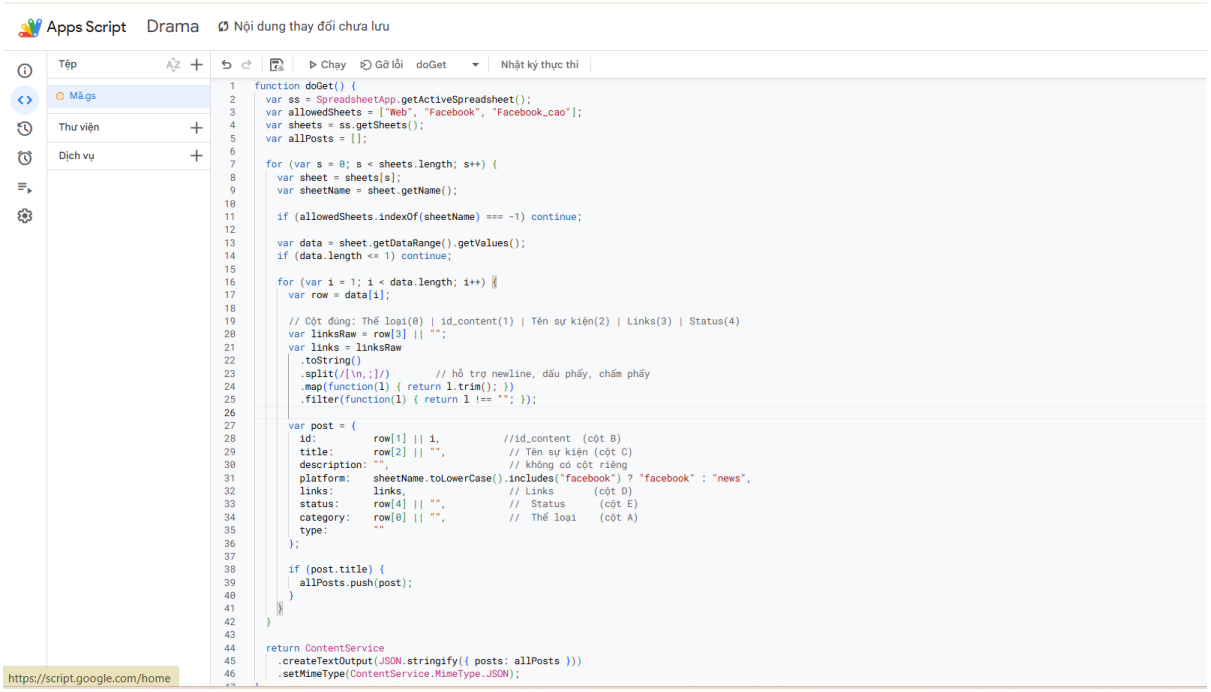
- Google Sheets: lưu trữ dữ liệu bài viết.
- Apps Script: xử lý logic + trả JSON.
- Frontend (React): gọi API và hiển thị.

3.2.1.3.3.4. Cấu trúc dữ liệu

Dữ liệu được lưu trong Google Sheets với các cột:

- id_content: ID bài viết
- Tên sự kiện: tiêu đề
- Links: có thể chứa nhiều link (phân cách bằng dấu phẩy)
- Danh mục, Thể loại: phân loại nội dung

3.2.1.3.3.5. Xây dựng API bằng Apps Script



Hình 3.7. Cấu hình tạo JSOL từ App Script.

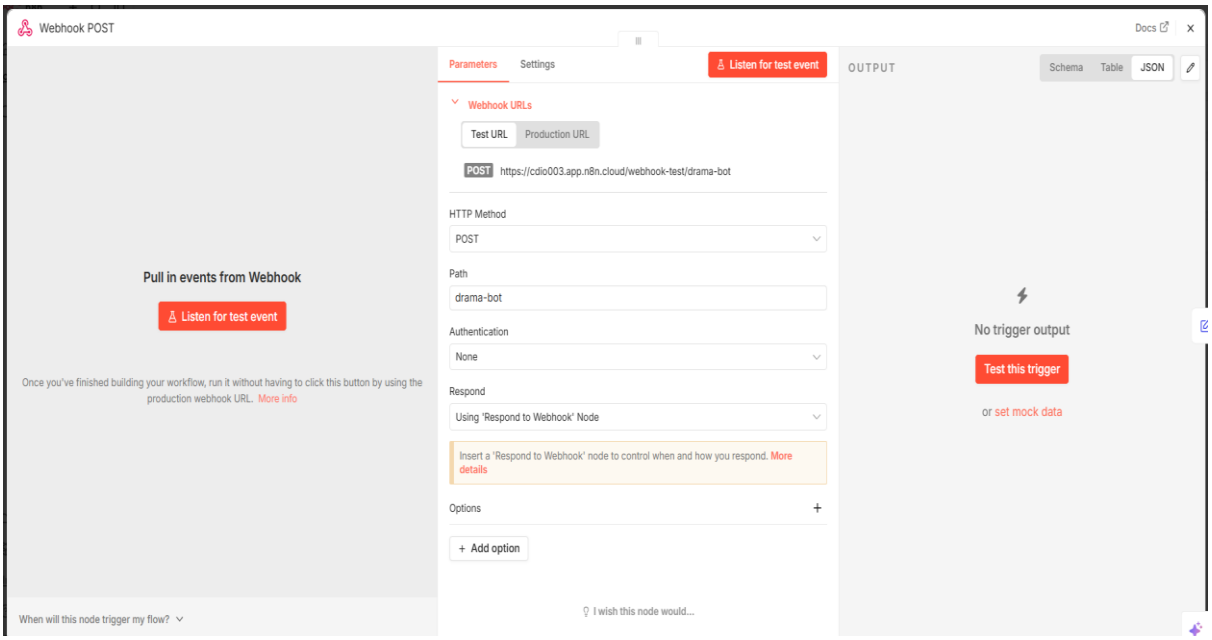
3.2.1.3.3.6. Kết nối với Frontend (React)



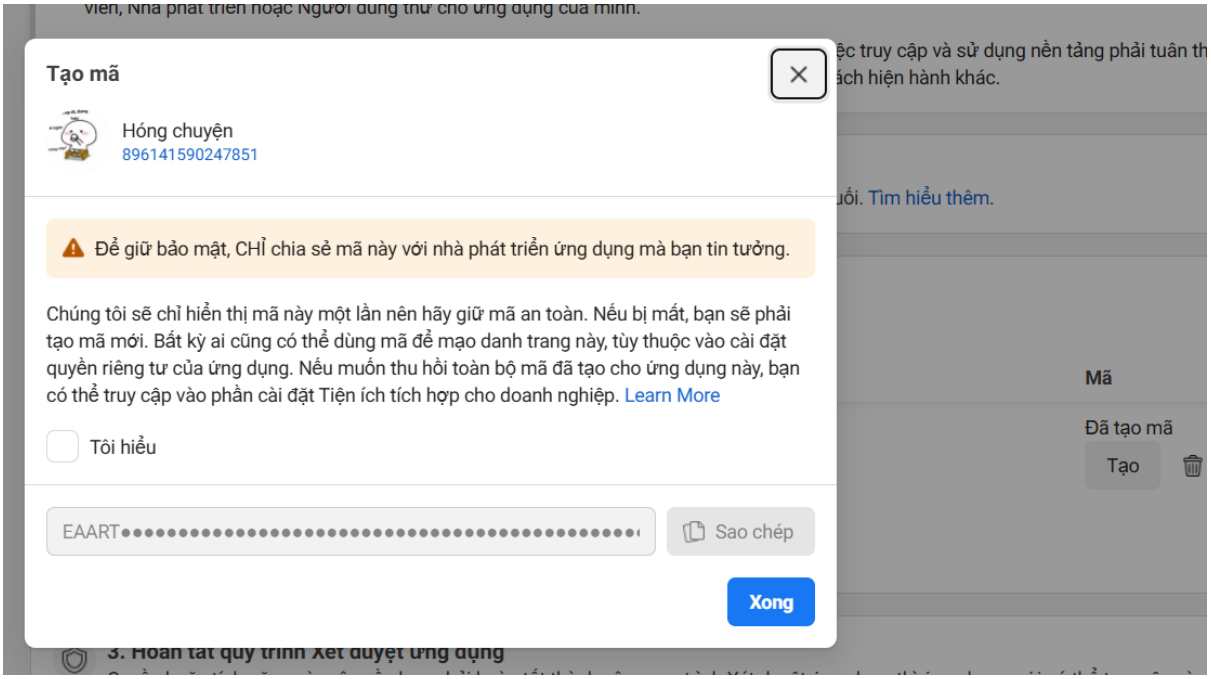
Hình 3.8. Cấu hình React gọi API App Script.

3.2.2. Tích hợp Chatbot

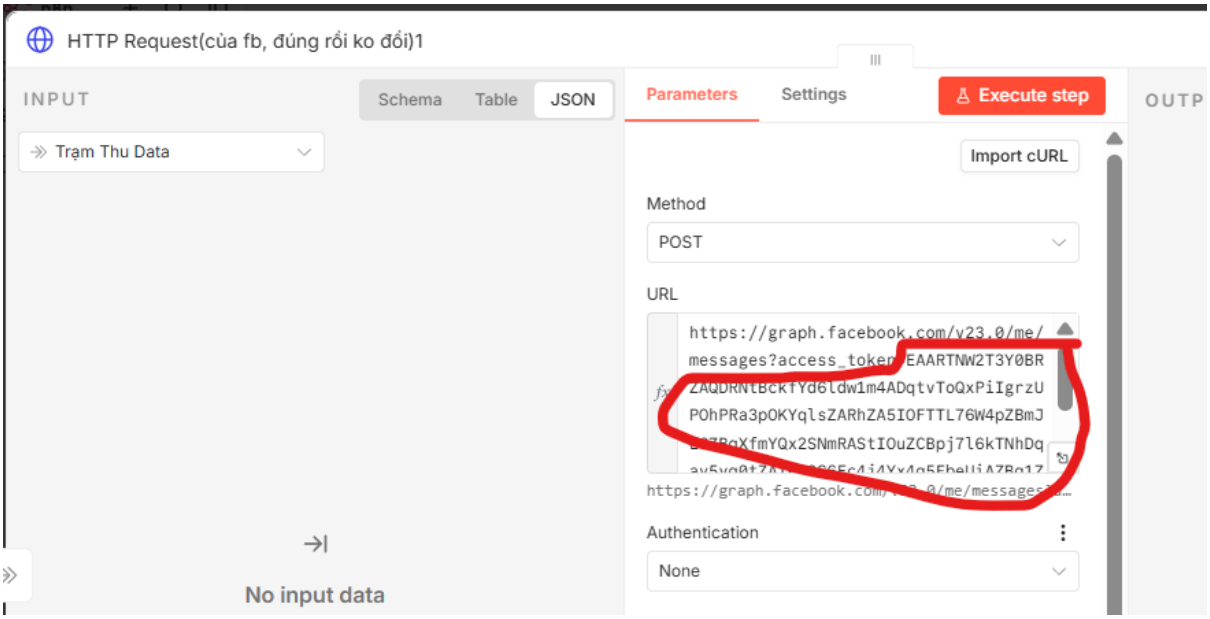
Triển khai Chatbot UI trên Facebook Messenger.
Cấu hình và kết nối Fanpage để cấp quyền bot tự động trả lời thông qua n8n.



Hình 3.9. Cấu hình n8n nhận tin nhắn messenger.



Hình 3.10. Lấy API để cấu hình đầu ra cho chatbot.



Hình 3.11. Dán API vào node HTTP Request.

3.3. Trực quan hóa Dữ liệu và Phân tích Insight trên Dashboard (AI Studio)

Hệ thống "Drama Intelligence System" xử lý khối lượng thông tin lớn qua nhiều giai đoạn. Để phục vụ việc giám sát hiệu năng và nghiên cứu xã hội học số, toàn bộ dữ liệu được trực quan hóa trên nền tảng AI Studio thông qua 3 tầng dữ liệu cốt lõi:

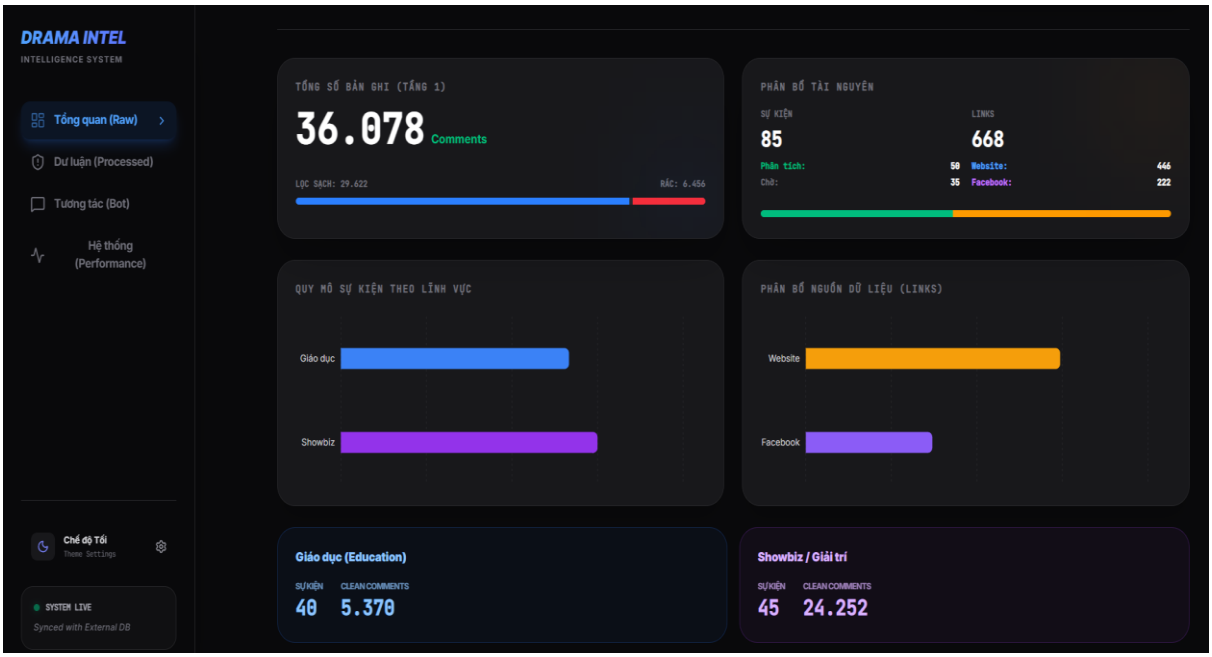
3.3.1. Trực quan hóa Dữ liệu thô (Raw Data)

Tầng này tập trung vào việc hiển thị quy mô, khối lượng dữ liệu đầu vào và hiệu suất hoạt động của đường ống thu thập (Data Pipeline).

- Dữ liệu được hiển thị: Các chỉ số tổng quan về số lượng sự kiện, nguồn link và tổng lượng bình luận thu thập được trước và sau khi làm sạch. Ví dụ thực tế từ hệ thống: Đã thu thập tổng cộng 85 sự kiện (với 668 links liên quan), cào được 36.078 bình luận thô, trong đó phát hiện 6.456 bình luận rác (chiếm tỷ lệ 17,9%).

- Ý nghĩa: Cung cấp cái nhìn toàn cảnh (Bird's-eye view) về sức mạnh cào dữ liệu của hệ thống n8n. Tỷ lệ rác (trash rate) lên tới 17.9% tổng thể (và đặc biệt cao ở mảng giáo dục với 31.3%) là bằng chứng rõ ràng cho sự cần thiết của module "Tiền lọc Rule-based" và "LLM gán nhãn" đã trình bày ở các chương trước.

- Thể hiện trên Dashboard:
    - Biểu đồ Thẻ số (Scorecards): Dùng để hiển thị các con số khổng lồ (Metrics) một cách trực diện như: Tổng số sự kiện (85), Tổng Links Web (446), Tổng Links Facebook (222), Tổng số Comments. Sử dụng tone màu Đen/Xám đậm để thể hiện tính nguyên bản.
    - Biểu đồ Cột ngang (Horizontal Bar Chart): Thống kê và so sánh hiệu suất cào giữa các lĩnh vực (Giáo dục: 40 sự kiện vs Showbiz: 45 sự kiện) hoặc tỷ trọng nguồn cào giữa Facebook và Báo chí.

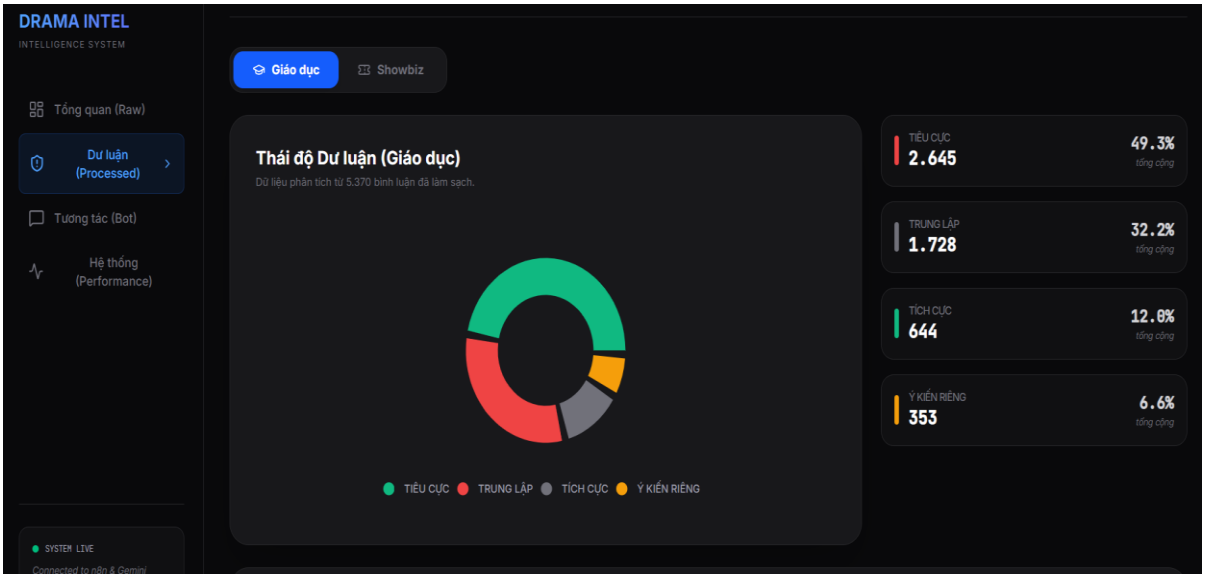


Hình 3.12. Trực quan hóa dữ liệu thô.

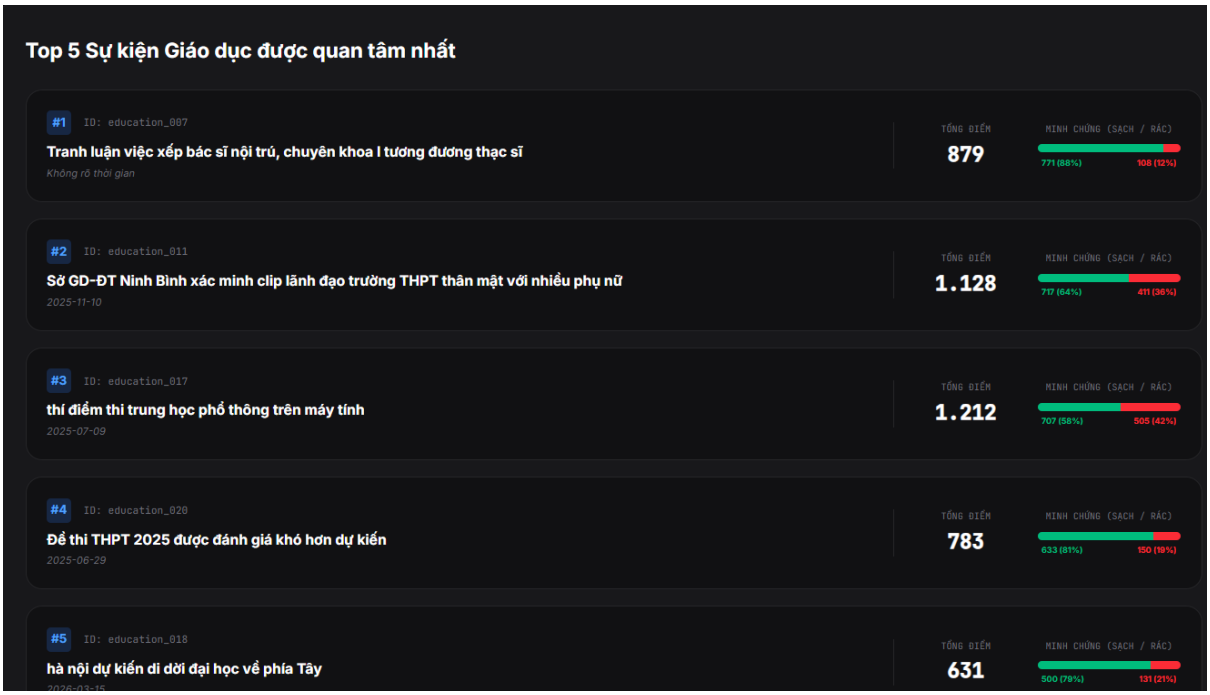
3.3.2. Trực quan hóa Dữ liệu phân tích (Processed Data)

Đây là thành phần trung tâm của AI Studio, nơi các tệp JSON thô biến thành những phân tích xã hội học mang tính định lượng nhờ sức mạnh của các mô hình LLM.

- Dữ liệu được hiển thị & Ý nghĩa:
    - Phân loại thái độ (Stance/Emotion): Hệ thống bóc tách dữ liệu thành 5 sắc thái cốt lõi: Phẫn nộ, Cà khịa, Đồng cảm, Ủng hộ và Trung lập. Đối với lĩnh vực giáo dục, hệ thống đo lường được sự chênh lệch lớn giữa thái độ Tiêu cực (49.3%) và Tích cực (12.0%). Điều này phản ánh rõ nét bức tranh tâm lý của cộng đồng trước các sự kiện dư luận.
    - Bình luận tranh cãi nhất: Trích xuất các bình luận có tỷ lệ tương tác bất thường (likes và replies). Đây là nơi khơi mào cho các cuộc tranh luận đa chiều (ví dụ: sự kiện Hoàng Hường có 4.740 comments hay Mailisa có 5.190 comments).
    - Nội dung độc hại (Toxic): Thống kê các bình luận vi phạm chuẩn mực cộng đồng. Hệ thống đã phát hiện 54 trường hợp toxic, trong đó có 32 ca công kích cá nhân và 20 ca chửi bới. Tính năng này mang ý nghĩa rất lớn trong việc kiểm duyệt và cảnh báo nội dung mạng xã hội.
- Thể hiện trên Dashboard:
    - Biểu đồ Khuyết (Donut Chart): Vẽ tỷ lệ phần trăm của 5 thái độ dư luận. Sử dụng các mảng màu sắc thái học: Đỏ (Phẫn nộ/Tiêu cực), Cam (Cà khịa), Xanh lá (Ủng hộ/Đồng cảm), và Xám (Trung lập).
    - Biểu đồ Phân tán (Scatter Plot): Trục X biểu diễn lượt Thích (Likes), Trục Y biểu diễn lượt Phản hồi (Replies). Sử dụng dải màu gradient đỏ tía (Purple/Red sẫm). Những điểm (dots) nằm ở góc trên cùng bên phải (High Likes, High Replies) sẽ tự động được hệ thống highlight là "Bình luận gây tranh cãi nhất".
    - Biểu đồ Mạng nhện (Radar Chart): Trực quan hóa các nhóm hành vi Toxic (Công kích cá nhân, Công kích người thân, Chửi bới). Dùng nền Đỏ nhạt để tạo hiệu ứng cảnh báo thị giác cho quản trị viên.



Hình 3.13. Lấy API để cấu hình đầu ra cho chatbot.



Hình 3.14. Top 5 sự kiện giáo dục được quan tâm nhất.

3.3.3. Trực quan hóa Dữ liệu tương tác Chatbot (Interaction Data).

Tầng dữ liệu này không đến từ việc cào web, mà được sinh ra trực tiếp từ lịch sử giao tiếp (QA History) giữa người dùng và Chatbot RAG, phản ánh đúng "nhu cầu theo doi dữ kiện dư luận" thực tế.

- Dữ liệu được hiển thị & Ý nghĩa: Phân tích thời gian người dùng online nhắn tin cho Bot, các sự kiện họ hay hỏi nhất, và những nhân vật bị "réo tên" nhiều nhất trong câu hỏi. Điều này mang ý nghĩa định hướng, giúp quản trị viên biết được sự kiện nào đang thực sự "nóng" trên nền tảng (Trending) để ưu tiên thu thập thêm dữ liệu.

- Thể hiện trên Dashboard:

    - Bản đồ Nhiệt (Heatmap): Trục X là ngày trong tuần, Trục Y là khung giờ. Các ô có mật độ truy vấn Chatbot cao sẽ được tô dải màu từ Vàng nhạt sang Đỏ sậm, qua đó xác định chính xác "Giờ vàng hóng drama" của cộng đồng mạng.

    - Biểu đồ Cột (Vertical Bar Chart): Tìm "Sự kiện Trending". Cột nào cao nhất thể hiện sự kiện đó (Ví dụ: "Hủy kết quả thi KHKT" hay "Sở GD Ninh Bình xác minh clip") đang được người dùng bắt bot tóm tắt nhiều nhất.

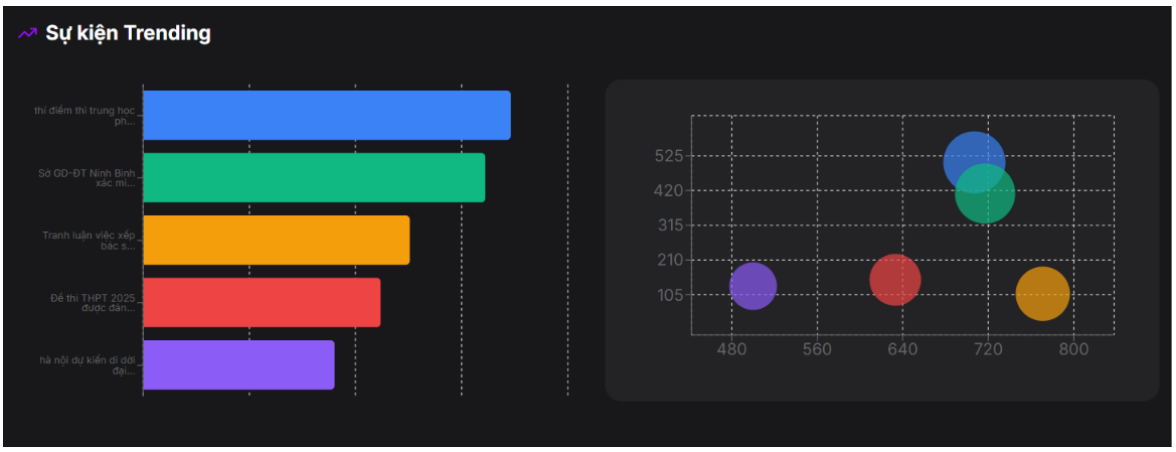
    - Đám mây từ vựng (Word Cloud): Hệ thống dùng thuật toán NER (Nhận diện thực thể có tên) để quét lịch sử chat và trích xuất tên nhân vật. Nhân vật nào bị hỏi nhiều nhất (Ví dụ: Mailisa, Shark Bình, ViruSs) sẽ xuất hiện với Font chữ to nhất, màu sắc rực rỡ nhất ngay giữa biểu đồ.

3.3.4. Trực quan hóa Dữ liệu Sự kiện và Từ khóa bằng Biểu đồ Bong bóng (Bubble Chart)

Thay vì sử dụng các biểu đồ cột đơn điệu, AI Studio nâng cấp trải nghiệm trực quan bằng cách sử dụng Biểu đồ Bong bóng (Bubble Chart) cho hai luồng dữ liệu quan trọng:

- Biểu đồ Bong bóng Sự kiện Trending: Trực quan hóa dữ liệu từ tệp insight.json (phần top_events_by_comments). Mỗi bong bóng đại diện cho một sự kiện (Ví dụ: "Mailisa bị bắt", "Đề thi THPT 2025"). Kích thước của bong bóng tỷ lệ thuận với tổng số lượng bình luận tương tác. Màu sắc được phân bổ theo lĩnh vực (Giáo dục hoặc Showbiz), giúp người quản trị nhìn nhận ngay lập tức tâm điểm truyền thông.

- Biểu đồ Bong bóng Từ khóa (Bag of Words / Top Keywords): Trực quan hóa kết quả đầu ra của thuật toán phân tích PMI. Biểu đồ này hiển thị Top 10 từ/cụm từ khóa nổi bật nhất cho từng lĩnh vực. Mỗi bong bóng đại diện cho một cụm từ (Ví dụ: "xét học bạ", "trốn thuế"), trong đó kích thước bong bóng thể hiện Điểm số tổng hợp (Score) và tần suất người dùng nhắc đến cụm từ đó trong tập dữ liệu bình luận



Hình 3.15. Top các sự kiện trending được biểu diễn dưới dạng đồ thị bong bóng.

3.4. Triển khai môi trường thực nghiệm cho pipeline PySpark song song

Để vận hành thành công pipeline PySpark song song, hệ thống đòi hỏi một môi trường thực nghiệm được cấu hình chuyên biệt, đặc biệt là trên hệ điều hành Windows.

3.4.1. Bảng cấu hình phần mềm

Thành phần	Phiên bản	Vai trò
Python	3.11.15	Driver process
Java JDK	Eclipse Temurin 17.0.13+11	JVM cho Spark executor
PySpark	3.5.3	Khung xử lý phân tán
Hadoop binaries	3.3.6 (winutils.exe + hadoop.dll)	Hỗ trợ ghi Parquet
underthesea	9.5.0	Tokenizer tiếng Việt

3.4.2. Lưu ý cài đặt trên hệ điều hành Windows

Việc triển khai PySpark trên Windows đòi hỏi ba thiết lập hệ thống đặc thù:

- Binding Native cho Hadoop: Yêu cầu đặt hai tệp winutils.exe và hadoop.dll tại thư mục HADOOP_HOME/bin/. Các tệp này cung cấp native binding cho hệ thống file Hadoop, bắt buộc phải có khi Spark thực hiện ghi dữ liệu định dạng Parquet (do hàm setPermission gọi qua native code). Thiếu thiết lập này sẽ gây ra lỗi UnsatisfiedLinkError.

- Cấu hình bảng mã (Encoding): Thiết lập biến môi trường PYTHONIO ENCODING=utf-8 và PYTHONUTF8=1 cho các hàm tự định nghĩa (UDF). Điều này nhằm ngăn chặn lỗi UnicodeEncodeError khi UDF in các ký tự tiếng Việt ra môi trường console mặc định (cp1252) của Windows.

- Workaround đọc dữ liệu: Do giới hạn của Hadoop FS native trên Windows, pipeline hiện tại đọc file JSON thông qua driver Python rồi khởi tạo DataFrame bằng spark.createDataFrame, thay vì sử dụng trực tiếp spark.read.json. Giải pháp tạm thời này sẽ được gỡ bỏ khi hệ thống dịch chuyển lên cụm phân tán (đọc trực tiếp từ S3 hoặc HDFS).

CHƯƠNG 4. TÍNH ỨNG DỤNG THỰC TẾ VÀ ĐÁNH GIÁ KẾT QUẢ

Chương này tập trung trình bày các kết quả đạt được của dự án "Drama Intelligence System" khi triển khai trong môi trường thực tế. Thông qua việc đánh giá các nền tảng ứng dụng (Website, Chatbot, Dashboard) và đo lường hiệu năng của đường ống dữ liệu (Data Pipeline), nghiên cứu sẽ làm rõ mức độ đáp ứng mục tiêu ban đầu, đồng thời chỉ ra những giới hạn kỹ thuật và hướng phát triển trong tương lai.

4.1. Tính ứng dụng thực tế

Hệ sinh thái "Drama Intelligence System" không chỉ dừng lại ở một mô hình thử nghiệm lý thuyết mà đã được thiết kế và triển khai thành một hệ thống phần mềm hoàn chỉnh, giải quyết trực tiếp các bài toán thực tiễn trong thời đại bùng nổ thông tin. Tính ứng dụng thực tế của dự án được đánh giá cao nhờ việc lấp đầy khoảng trống công nghệ của các hệ thống Social Listening thương mại (như Buzzmetrics, YouNet Media vốn chỉ mạnh về thống kê định lượng nhưng thiếu khả năng sinh văn bản) và các mô hình Học máy truyền thống (chỉ có khả năng phân loại tĩnh). Giá trị ứng dụng thực tiễn của hệ thống được thể hiện mạnh mẽ qua ba khía cạnh và nhóm đối tượng mục tiêu sau đây:

4.1.1. Cung cấp giải pháp tiếp nhận thông tin thông minh cho người dùng cá nhân đại chúng

Trong bối cảnh mạng xã hội, đặc biệt là Facebook với hơn 79 triệu người dùng tại Việt Nam, thông tin thường xuyên rơi vào tình trạng phân mảnh, nhiễu loạn và chứa nhiều tin giả (fake news). Đối với nhóm người dùng cá nhân (đặc biệt là giới trẻ, học sinh, sinh viên) không có nhiều thời gian theo dõi liên tục các diễn biến sự kiện truyền thông, hệ thống mang lại một giải pháp cập nhật thông tin hoàn toàn mới:

- Tổng hợp ngữ cảnh đa chiều: Thay vì phải lướt hàng chục bài viết rời rạc, người dùng có thể truy cập Public Web để đọc một bản tóm tắt sự kiện mạch lạc, khách quan được tổng hợp từ nhiều nguồn báo chí chính thống. Hệ thống tự động trích xuất các nhân vật liên quan (thông qua NER), giúp người đọc nhanh chóng nắm bắt các bên liên đới trong sự kiện.
- Trợ lý ảo tương tác thời gian thực (Conversational UI): Sự tích hợp của Chatbot RAG đa nền tảng (Messenger và Web) biến kho dữ liệu tĩnh thành một trợ lý đắc lực. Người dùng có thể đặt các câu hỏi truy vấn sâu (Ví dụ: "Tại sao nhân vật A lại bị cộng đồng mạng chỉ trích?"). Đặc biệt, với cơ chế chống ảo giác AI (Live Web Research kết hợp Firecrawl), chatbot có khả năng tự động tra cứu Internet để trả lời chính xác các sự kiện nóng vừa mới xảy ra mà không bị giới hạn bởi dữ liệu huấn luyện cũ của mô hình.

4.1.2. Hỗ trợ cơ quan quản lý và nhà hoạch định chính sách (Đặc thù trong lĩnh vực Giáo dục)

Ngành giáo dục đang trong giai đoạn chuyển mình mạnh mẽ với nhiều chính sách và phương thức thi cử mới, từ đó nảy sinh nhiều luồng quan điểm trái chiều, gây xáo trộn nhất định trong dư luận (đặc biệt từ phía phụ huynh và học sinh). "Drama Intelligence System" chứng minh giá trị thực tiễn to lớn khi đóng vai trò là một công cụ "đo lường sức khỏe dư luận" cho các nhà quản lý, cơ quan nhà nước hoặc các trường học:

- Đo lường mức độ đồng thuận: Thông qua Dashboard phân tích (AI Studio), các nhà quản lý có thể nhìn thấy ngay tỷ lệ phần trăm dư luận phản đối hay ủng hộ một chính sách (Ví dụ: Sự kiện "Đề thi THPT 2025" hay "Xếp bác sĩ nội trú tương đương thạc sĩ").
- Phát hiện ý kiến đóng góp mang tính xây dựng: Điểm đột phá trong ứng dụng thực tế của dự án là việc tinh chỉnh LLM phân biệt rõ ràng giữa nhóm bình luận "Tiêu cực/Phẫn nộ" (chỉ trích đơn thuần) và nhóm "Ý kiến riêng" (ủng hộ một phần, đề xuất giải pháp, hiến kế). Điều này giúp các nhà làm chính sách dễ dàng lọc ra những góp ý chất lượng từ hàng ngàn bình luận hỗn tạp để kịp thời điều chỉnh các thông tư, quy định cho phù hợp với thực tiễn.

4.1.3. Ứng dụng trong kiểm duyệt nội dung và quản trị khủng hoảng truyền thông (Lĩnh vực Giải trí/Showbiz)

Môi trường Showbiz trên mạng xã hội mang đặc thù tần suất phát sinh tần suất phát sinh sự kiện dư luận cao, ngôn từ sử dụng mang đậm tính cảm xúc, nhiều từ lóng (slang) và tính sát thương lớn. Hệ thống cung cấp các bộ công cụ mạnh mẽ hỗ trợ các doanh nghiệp truyền thông, tổ chức quản lý nghệ sĩ hoặc đội ngũ kiểm duyệt nội dung mạng lưới:

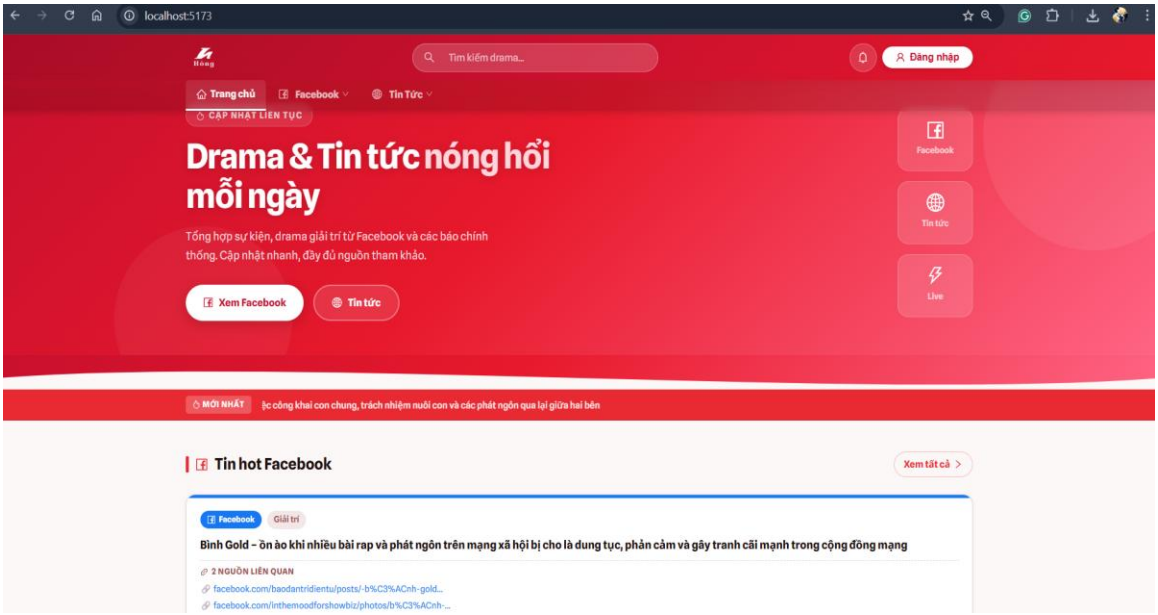
- Phát hiện và cảnh báo rủi ro trực tuyến (Toxic Detection): Hệ thống tự động gắn cờ và phân loại các nội dung độc hại thành các nhóm vi phạm chuẩn mực cộng đồng (Chửi bới, Công kích cá nhân, Công kích người thân). Thực nghiệm cho thấy hệ thống đã quét và đánh dấu hàng loạt các bình luận độc hại trong tổng số hơn 24.000 bình luận Showbiz, giúp các quản trị viên Fanpage/Group dễ dàng loại bỏ nội dung rác, làm sạch không gian mạng.
- Truy vết luồng tranh cãi phi tuyến tính: Nhờ cấu trúc lưu trữ dạng Đồ thị thảo luận (Discussion Graph) sử dụng khóa ngoại reply_to_id, hệ thống không chỉ phân tích các bình luận đơn lẻ mà còn tái tạo lại toàn bộ chuỗi tranh luận (thread). Các luồng bình luận có tỷ lệ cảm xúc hỗn loạn (không có cảm xúc nào chiếm trên 70%) sẽ tự động kích hoạt cờ cảnh báo controversial: true. Tính năng này cho phép các chuyên gia xử lý khủng hoảng truyền thông xác định chính xác đâu là bình luận "châm ngòi" (High Likes, High Replies) để đưa ra phương án dập tắt khủng hoảng kịp thời.

Tóm lại, "Drama Intelligence System" đã vượt ra khỏi giới hạn của một bài toán phân tích dữ liệu hàn lâm. Bằng việc kết hợp sức mạnh thu thập tự động của n8n, khả năng suy luận linh hoạt của LLM và giao diện tương tác đa nền tảng, dự án đã mang đến một công cụ thực chiến, phục vụ hiệu quả cho cả cộng đồng trong việc tiếp nhận thông tin sạch, lẫn các tổ chức quản lý trong việc thấu hiểu và định hướng dư luận xã hội.

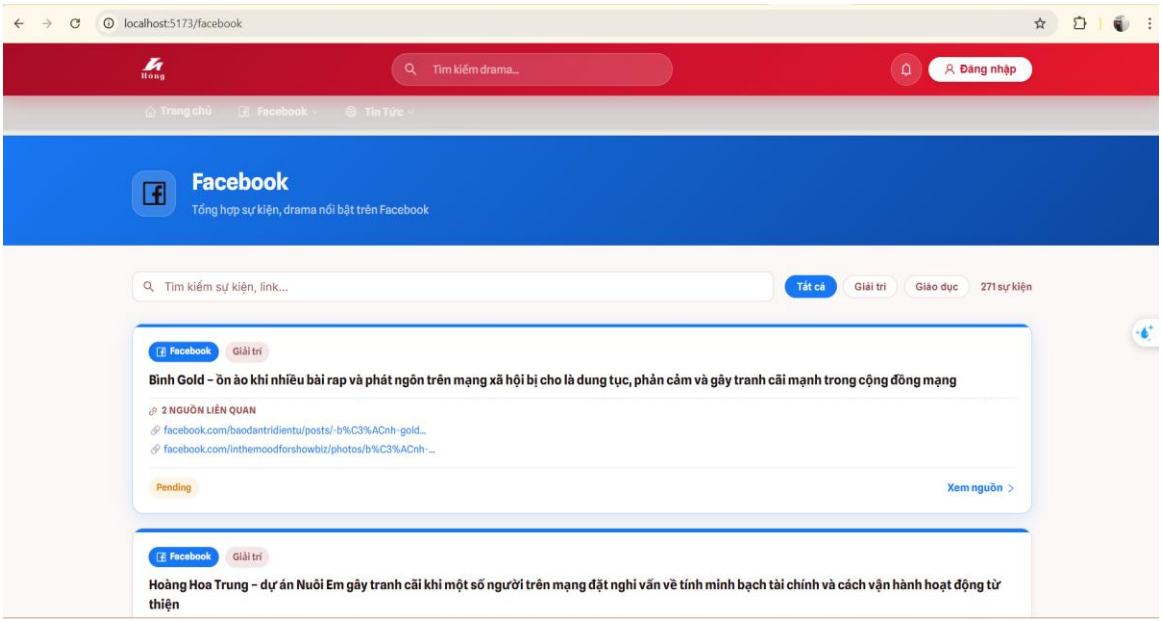
4.2. Website Tin Tức và Fanpage vệ tinh

4.2.1. Website Báo chí Tổng hợp (Public Web).

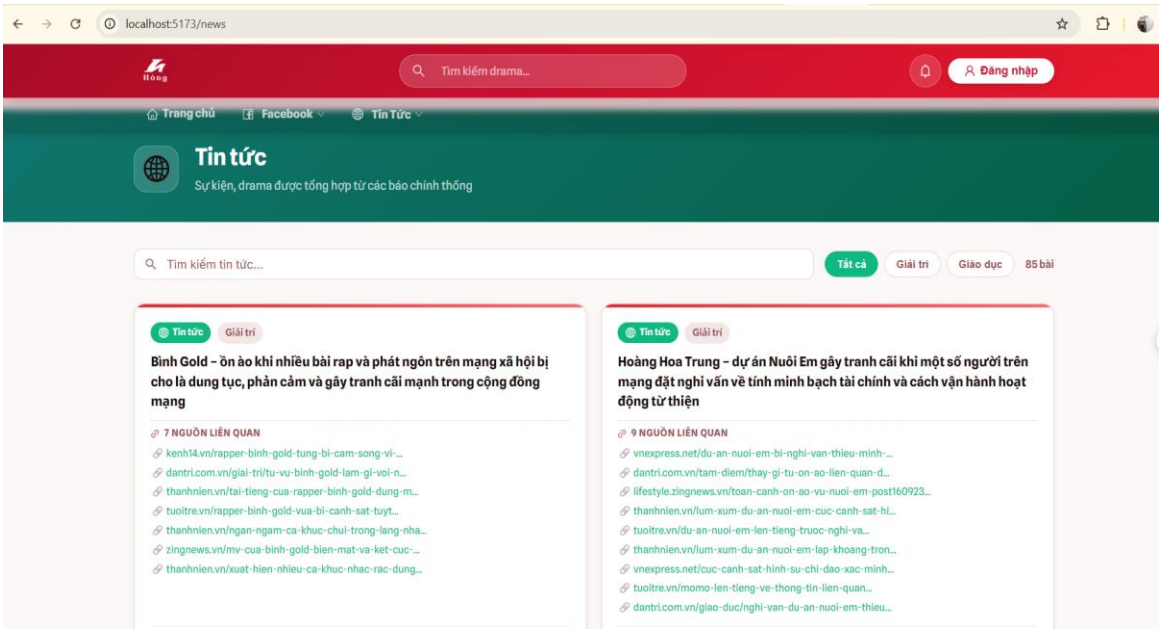
Để giải quyết bài toán phân mảnh thông tin nghiêm trọng trên không gian mạng, hệ thống đã triển khai thành công một nền tảng Web với giao diện thân thiện (sử dụng tone màu đỏ chủ đạo mang tính báo chí cập nhật). Chức năng cốt lõi của Website là hợp nhất các mảnh thông tin rời rạc từ nhiều nguồn báo chí khác nhau thành một bài viết duy nhất có ngữ cảnh đầy đủ. Kiến trúc này hỗ trợ hệ thống không chỉ hiển thị văn bản tĩnh, mà còn cung cấp danh sách các nhân vật liên quan (trích xuất qua NER) và tích hợp trực tiếp Chatbot AI lên giao diện web để người dùng có thể hỏi đáp ngay lập tức.



Hình 4.1. Hình ảnh giao diện Web chính



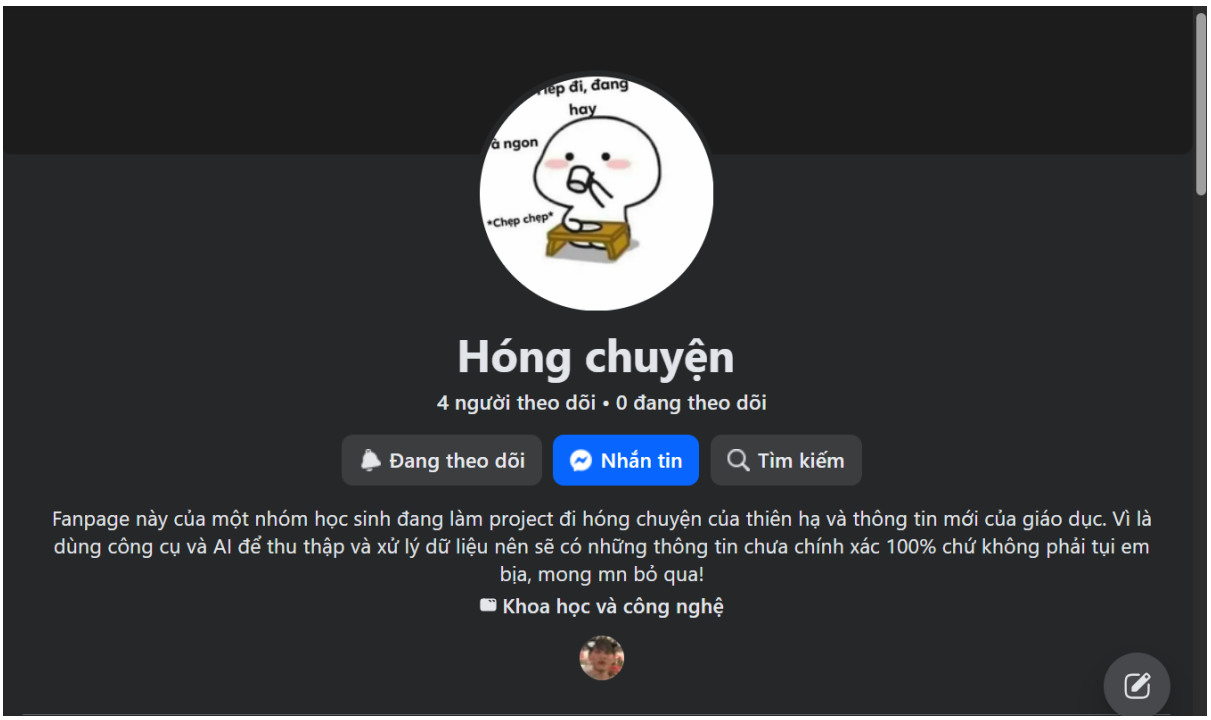
Hình 4.2. Hình ảnh giao diện Web cho tin tức facebook.



Hình 4.3. Hình ảnh giao diện Web cho tin tức từ các trang báo.

4.2.2. Fanpage vệ tinh "Hóng chuyện"

Nhằm đo lường mức độ tiếp cận thực tế và thu thập thêm dữ liệu tương tác (Interaction Data), dự án đã xây dựng Fanpage vệ tinh mang tên "Hóng chuyện" trên nền tảng Facebook. Fanpage này đóng vai trò như một kênh phân phối lại (redistribute) các insight mà AI đã tổng hợp được, đồng thời là một minh chứng cho thấy hệ thống có thể kết nối với công chúng mục tiêu (đặc biệt là nhóm học sinh/sinh viên quan tâm đến mảng giáo dục và giải trí).



Hình 4.4. Giao diện Fanpage.

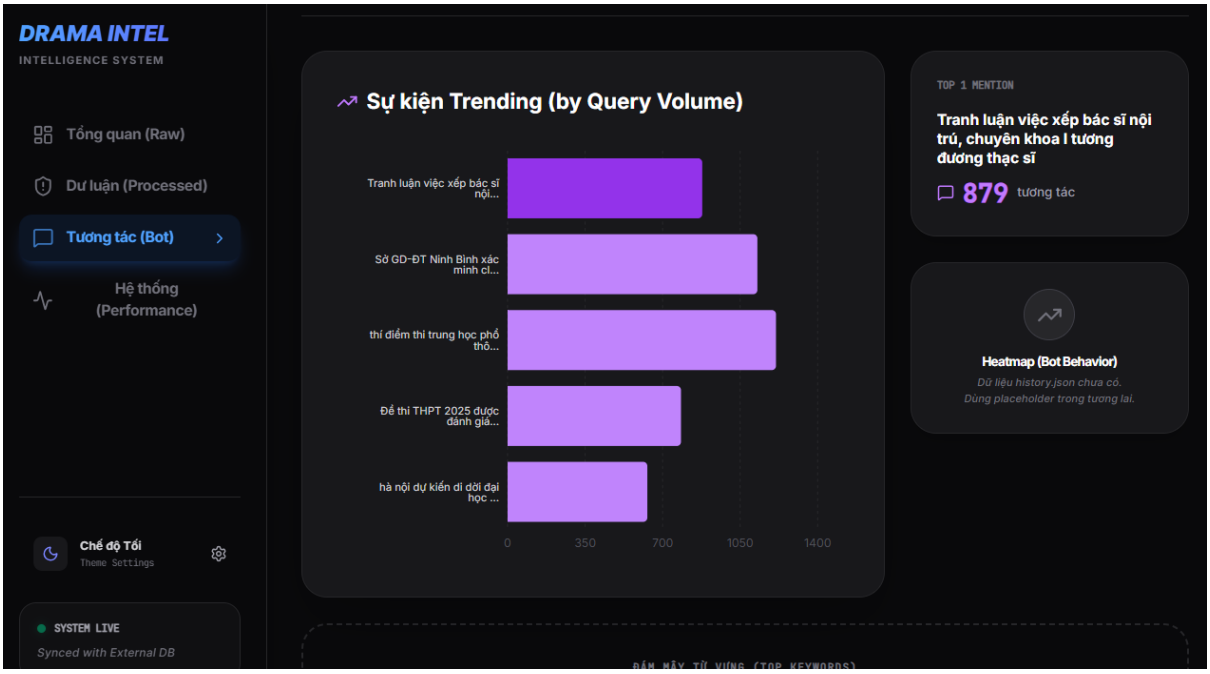
4.3. Dashboard Phân tích Dư luận (AI Studio) trong thực tiễn

AI Studio là giao diện quản trị và phân tích dữ liệu chuyên sâu, trực quan hóa toàn bộ khối lượng JSON thô thành các dạng biểu đồ mang giá trị xã hội học.

4.3.1. Trực quan hóa kết quả phân tích

Trực quan hóa kết quả phân tích Dashboard thành công trong việc hiển thị 3 tầng dữ liệu cốt lõi:

- Phân tích cảm xúc: Sử dụng Biểu đồ Khuyết (Donut Chart) để trực quan hóa tỷ lệ 5 sắc thái dư luận: Ủng hộ, Phản đối (Phẫn nộ), Trung lập, Cà khịa, và Đồng cảm.
- Cảnh báo tiêu cực: Áp dụng Biểu đồ Mạng nhện (Radar Chart) để theo dõi và cảnh báo các bình luận độc hại (toxic), chứa ngôn từ chửi bới hoặc công kích cá nhân.
- Hành vi người dùng: Thông qua Heatmap và Word Cloud, hệ thống đo lường được "giờ vàng hóng drama" và chỉ ra những sự kiện (Trending Event) được người dùng truy vấn nhiều nhất thông qua Chatbot.



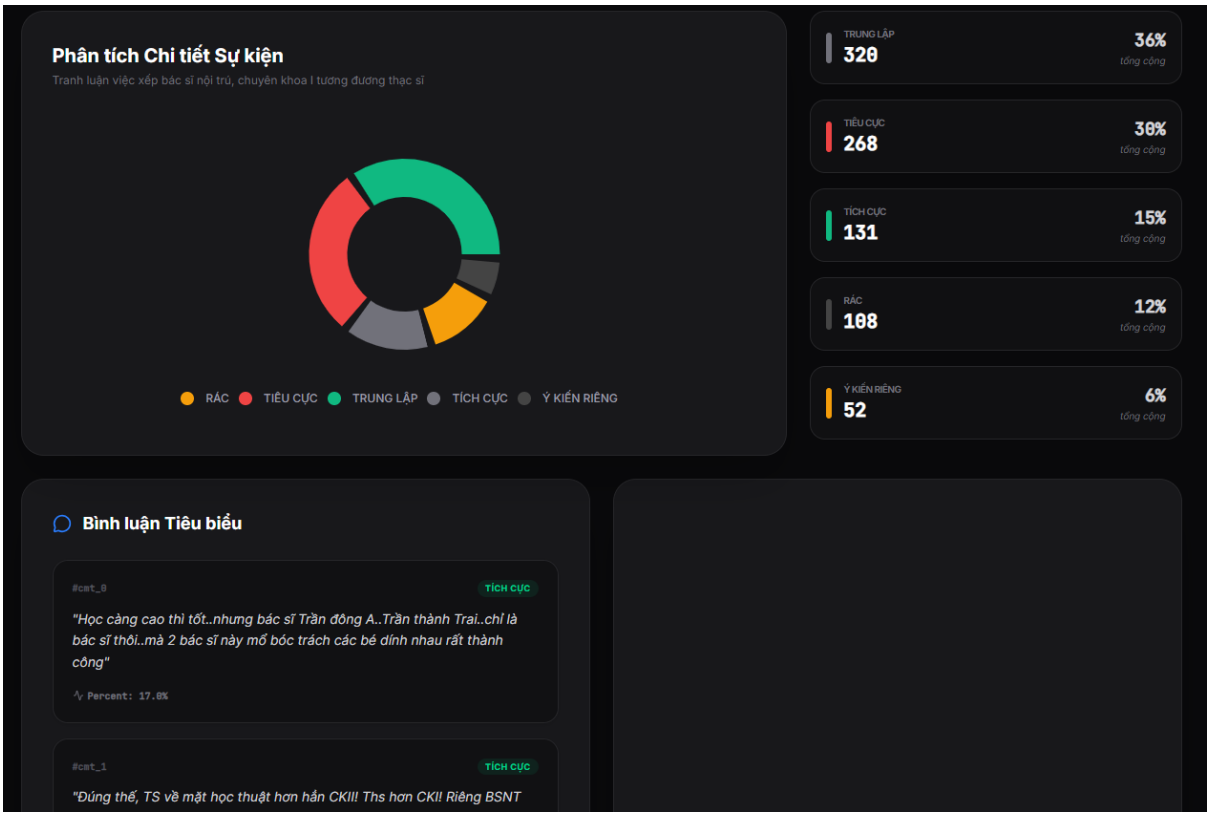
Hình 4.5. Hình ảnh thể hiện sự quan tâm của người dùng đối với các sự kiện.

4.3.2. Case Study (Nghiên cứu tình huống):

Sự kiện "Hủy kết quả thi Khoa học kỹ thuật quốc gia vì đạo nhái" Để chứng minh năng lực rút ra insight thực sự, hệ thống tiến hành phân tích sự kiện "Hủy kết quả thi KHKT".

- Về tỷ lệ cảm xúc: Khác với các tin tức giải trí mang nhiều tính "cà khịa", biểu đồ phân tích sự kiện giáo dục này cho thấy sự áp đảo của nhóm cảm xúc Phẫn nộ và Ủng hộ việc xử lý nghiêm.

- Về tính tranh cãi: Dựa trên bộ đếm likes và reply_count, biểu đồ Scatter Plot (vùng màu đỏ tía) đã tự động trích xuất được những bình luận tiêu biểu nhất (Ví dụ: Các luồng ý kiến phân tích trách nhiệm của Ban giám khảo hoặc đòi dẹp bỏ cuộc thi). Cấu trúc đồ thị thảo luận (reply_to_id) giúp hệ thống nhận diện rõ ràng các cuộc tranh luận trái chiều nảy lửa bên dưới các bình luận gốc này.



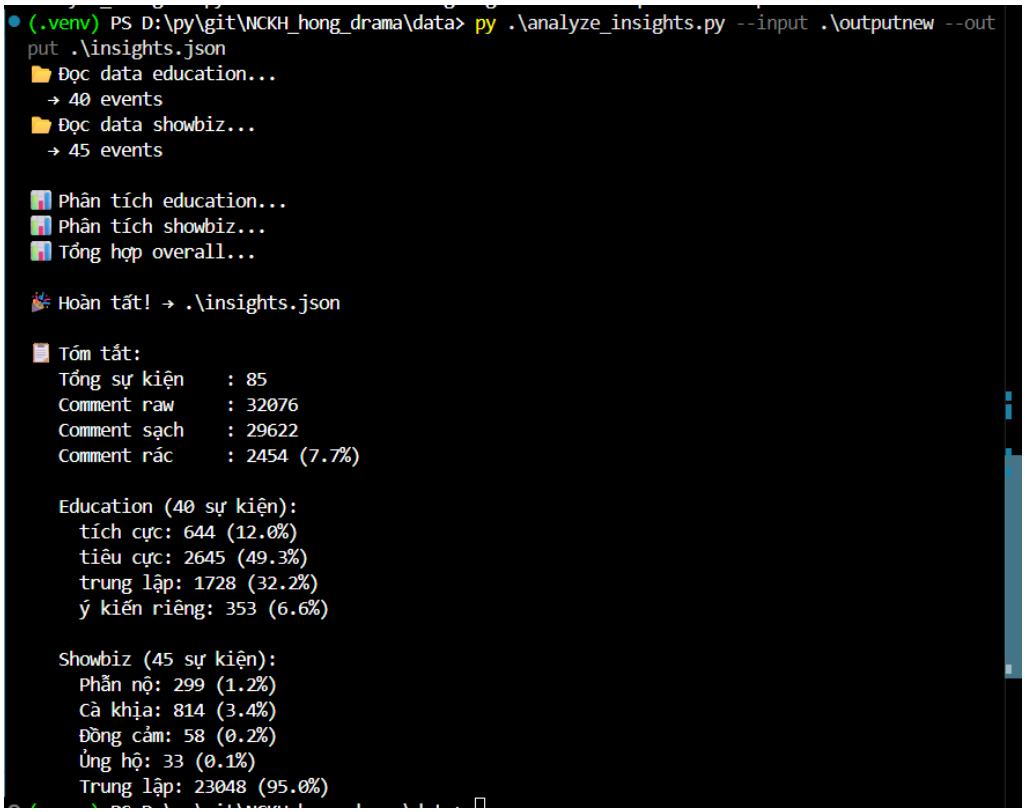
Hình 4.6. Hình ảnh thể hiện sự quan tâm của người dùng đối với sự kiện bác sĩ nội trú, chuyên khoa I tương dương với thạc sĩ.

4.4. Đánh giá thực nghiệm

4.4.1. Đánh giá năng lực thu thập và xử lý AI

- Khối lượng và Tốc độ xử lý Pipeline:

Hệ thống đã thu thập và xử lý thành công một khối lượng lớn dữ liệu JSON thô từ cả Web và Facebook. Điểm sáng lớn nhất trong khâu thu thập là sự thành công của chiến lược Account Rotation (Xoay vòng tài khoản) trên nền tảng Apify. Bằng cách tự động chuyển đổi tài khoản khi cạn gói 5$ credit, hệ thống đã vượt qua rào cản chặn tải trang của Facebook trong phần lớn các trường hợp thử nghiệm mà không tốn kém chi phí hạ tầng lớn.

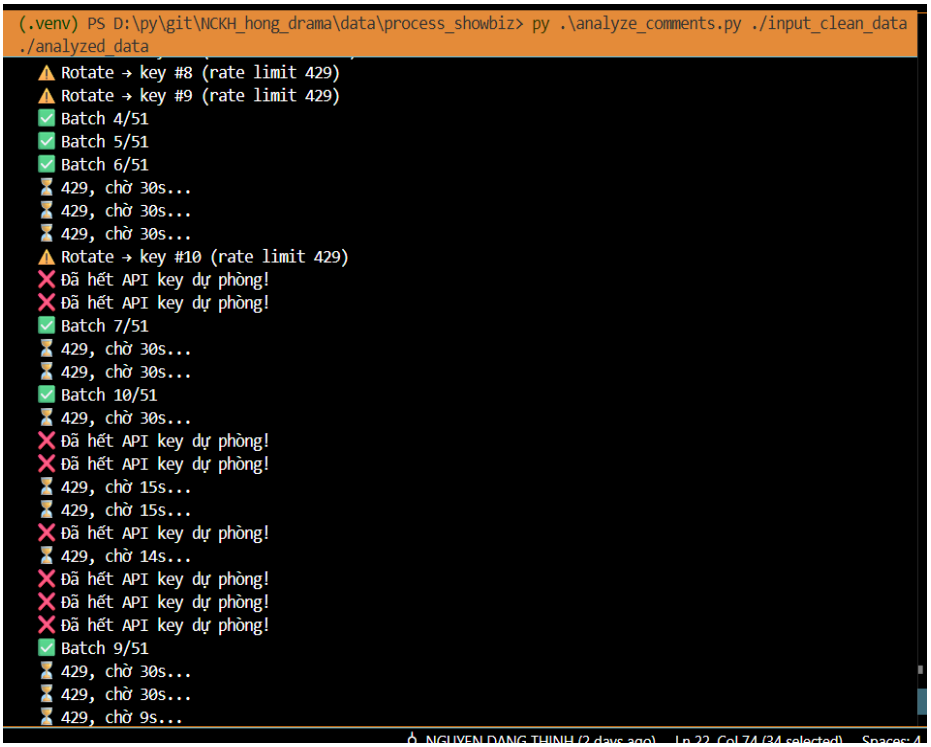


Hình 4.7. Tổng quan về dữ liệu thu thập được, đã xử lý và phân tích.

- Đánh giá luồng xử lý AI (AI Processing Performance) Khả năng xử lý dữ liệu lớn tại máy trạm (Local) đã được chứng minh qua hai khía cạnh kỹ thuật phức tạp:

- Hiệu năng xử lý bất đồng bộ: Việc áp dụng chiến lược phân tích 2 bước (Two-step Processing), kết hợp luồng xử lý song song asyncio.gather và giới hạn Semaphore(5) đã giúp giảm thiểu đáng kể thời gian phân tích các lô dữ liệu (Batching) lớn.

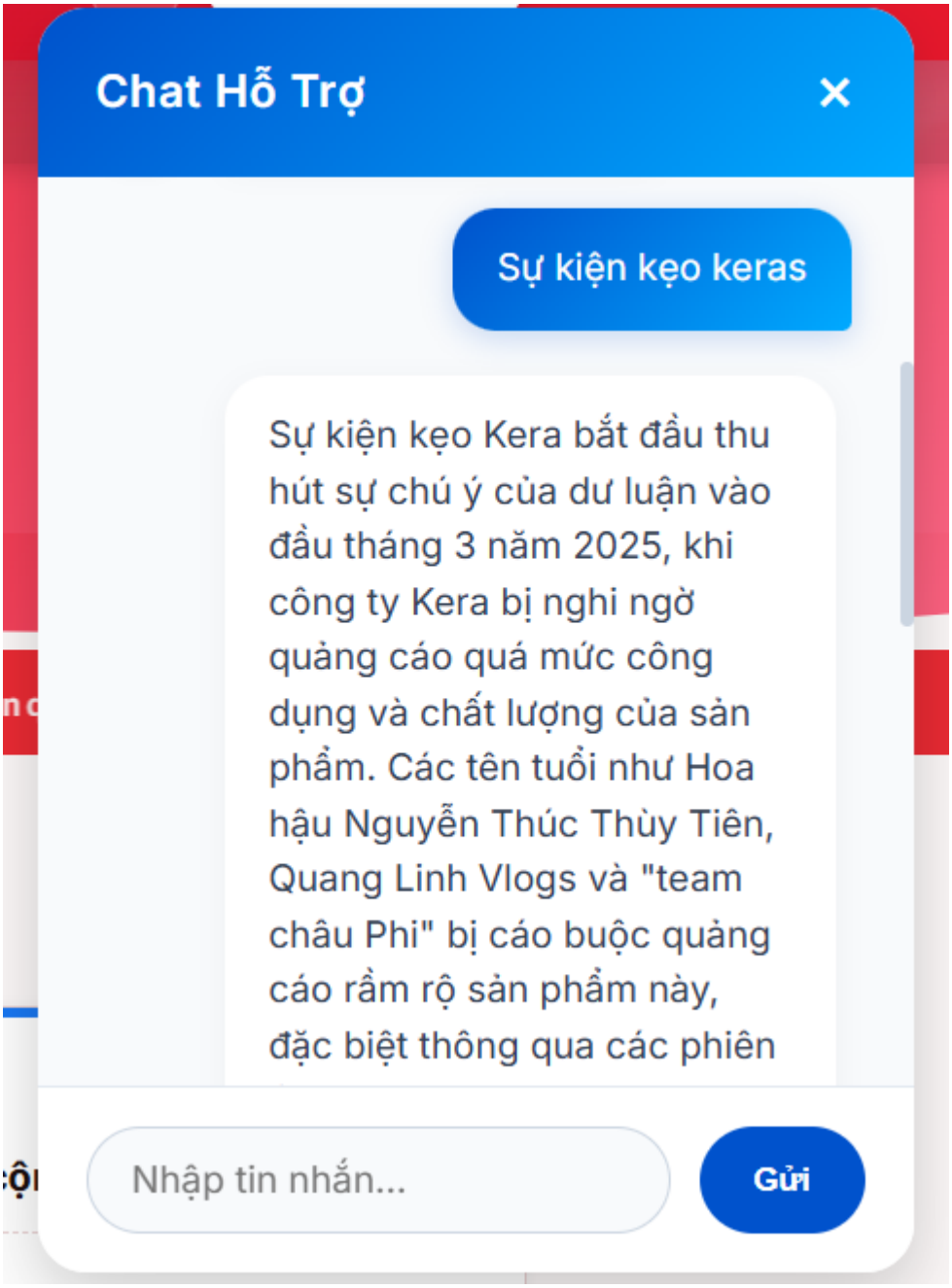
- Độ ổn định của hệ thống: Quá trình gọi API của các mô hình LLM (như Groq, Gemini) thường xuyên đối mặt với lỗi HTTP 429 và 503. Module AccountManager đã hoạt động xuất sắc khi tự động kích hoạt cơ chế Exponential Backoff (chờ và thử lại) và API Key Rotation. Nhờ đó, đường ống dữ liệu (Data Pipeline) không bị crash (đứt gãy) trong suốt quá trình xử lý hàng ngàn bình luận.
- (Ghi chú cho tác giả: Nếu bạn có đánh giá độ chính xác (F1-score) của model LLM khi phân loại 5 cảm xúc, hãy điền một câu vào đây, ví dụ: "Mô hình đạt độ chính xác khoảng X% khi phân loại các bình luận cà khịa mang tính châm biếm sâu cay").



Hình 4.8. Xử lý dữ liệu dưới máy và quá trình chờ API.

- Hiệu suất Chatbot và Cơ chế chống ảo giác AI Hệ thống Chatbot áp dụng RAG đã thay thế hoàn toàn cấu trúc "if-else" cứng nhắc

Việc áp dụng RAG mang lại khả năng giao tiếp tự nhiên. Kết quả thực nghiệm quan trọng nhất là sự hiệu quả của cơ chế Live Web Research. Khi đối mặt với các câu hỏi về sự kiện ngoài luồng (Out-of-domain) chưa có trong cơ sở dữ liệu Vector, n8n Agent đã tự động kích hoạt Firecrawl để cào dữ liệu Google Search theo thời gian thực. Kết quả là Chatbot đưa ra câu trả lời chính xác kèm theo nguồn trích dẫn, giảm đáng kể hiện tượng ảo giác AI (Hallucination) – một hạn chế phổ biến của các mô hình LLM hiện nay



Hình 4.9. Cuộc hội thoại giữa người và chatbot.

4.4.2. Phương pháp xây dựng tập dữ liệu kiểm thử (Ground Truth Dataset)

Để có cơ sở khách quan đo lường độ chính xác của hệ thống Trí tuệ Nhân tạo (được trình bày tại Bảng 4.2), nghiên cứu tiến hành xây dựng các tập dữ liệu kiểm thử (Ground Truth) dựa trên dữ liệu thực tế. Quá trình lấy mẫu được chia thành hai nhóm mục tiêu đánh giá cụ thể:

1. Tập dữ liệu kiểm thử Phân loại cảm xúc (Sentiment Analysis Dataset):

Dữ liệu được trích xuất bằng phương pháp lấy mẫu phân tầng (Stratified Sampling) nhằm đảm bảo sự phân bố đồng đều của các nhãn cảm xúc. Quá trình thực hiện qua hai bước:

- Tổng hợp (Aggregation): Hệ thống chạy kịch bản aggregate_comments_v2.py để quét toàn bộ các thư mục phân tích, gộp các bình luận hợp lệ và loại bỏ hoàn toàn các bình luận rác (is_trash = True).
- Lấy mẫu phân tầng (Stratified Sampling): Kịch bản sample_comments.py được sử dụng để trích xuất ngẫu nhiên dựa trên cấu hình tỷ lệ cảm xúc. Kết quả, hệ thống trích xuất chính xác 616 mẫu bình luận (316 mẫu thuộc lĩnh vực Showbiz và 300 mẫu thuộc lĩnh vực Giáo dục). Tập dữ liệu này được xuất ra định dạng Excel để nhóm nghiên cứu tiến hành dán nhãn thủ công (Human Annotation). Kết quả dán nhãn của con người sẽ làm "Ground Truth" để đối chiếu và tính toán các chỉ số Accuracy, F1-score cho mô hình LLM.yy

Phân loại cảm xúc (Sentiment Analysis)	**Tổng số mẫu**	**Dán nhãn đúng**	**Dán nhãn sai**	**Accuracy (%)**
Tập dữ liệu Showbiz	316	253	63	80,1%
Tập dữ liệu giáo dục	300	226	74	75.3%

Bảng 4.1. Kết quả dán nhãn thủ công tổng thể (Human Annotation - Overall Accuracy).

2. Tập dữ liệu kiểm thử RAG (Retrieval & Generation Dataset):

Đối với luồng Hỏi-Đáp của Chatbot, thay vì tạo câu hỏi giả định, hệ thống trích xuất dữ liệu trực tiếp từ Nhật ký thực thi (Execution Logs) của nền tảng n8n trong quá trình vận hành thực tế.

- Nhóm nghiên cứu trích xuất ngẫu nhiên 30 lượt hội thoại (bao gồm Câu hỏi người dùng, Ngữ cảnh được trích xuất từ Vector Store, và Câu trả lời cuối cùng của Chatbot).
- Tập dữ liệu thực tế này được đưa vào quy trình đánh giá LLM-as-a-Judge để chấm điểm khả năng truy xuất (Hit Rate của Supabase) và khả năng sinh văn bản (độ trung thực - Faithfulness, và độ liên quan - Answer Relevance).

Thành phần đánh giá	**Metric**	**Kết quả**
Truy xuất dữ liệu (Retrieval)	Hit Rate	29/30 = 97%
Sinh văn bản (Generation)	Faithfulness (TB)	9.34 / 10
Sinh văn bản (Generation)	Answer Relevance (TB)	9.20 / 10

Bảng 4.2. Kết quả đánh giá hệ thống RAG.

Thành phần AI	Mục tiêu đánh giá	Metric cụ thể	Ý nghĩa / Phương pháp đo lường
1. Phân loại cảm xúc (Sentiment Analysis)	Độ chính xác của LLM khi dán nhãn thái độ dư luận	Accuracy, F1-Score, Precision, Recall	Đánh giá so sánh kết quả của LLM (Llama-3.3) với kết quả dán nhãn thủ công của con người (Ground Truth).
2. Truy xuất dữ liệu RAG (Retrieval)	Khả năng tìm kiếm vector của Supabase	Hit Rate, MRR, Recall@K (thường dùng K=3 hoặc K=5)	Recall@K: Trong top K đoạn văn bản (chunks) lấy lên từ database, có chứa đoạn văn bản thực sự mang thông tin để trả lời câu hỏi hay không.
3. Sinh văn bản RAG (Generation)	Chất lượng câu trả lời bằng ngôn ngữ tự nhiên	Faithfulness (Độ trung thực/ Không ảo giác) Answer Relevance (Độ liên quan)	Đánh giá xem câu trả lời sinh ra có dựa hoàn toàn vào dữ liệu (Context) tìm được không, hay AI tự bịa đặt thông tin. *Đo lường bằng phương pháp LLM-as-a-Judge.* Câu trả lời có đi thẳng vào trọng tâm câu hỏi của người dùng không, hay trả lời lan man. *Đo lường bằng phương pháp LLM-as-a-Judge.*

Bảng 4.3. Các độ đo (Metrics) đánh giá hệ thống AI

4.4.3. Kiểm chứng đối chiếu pipeline tiền xử lý song song

Để đánh giá tính chính xác của kiến trúc mới, hai pipeline tiền xử lý (Python và PySpark) được kích hoạt chạy độc lập trên cùng một bộ dữ liệu đầu vào.

4.4.3.1. Đối chiếu số bình luận sạch

Chỉ số	Pipeline Python	Pipeline PySpark	Khớp
Số bình luận sạch	29.660	29.660	Có
Hệ số Jaccard giữa hai tập ID	—	—	1.0000
Số bản ghi khác biệt	0	0	Có

Bảng 4.4. Bảng đối chiếu bình luận sạch.

Kết quả thực nghiệm cho thấy cả hai pipeline xuất ra chính xác tập 29.660 bình luận sạch với hệ số Jaccard đạt mức tuyệt đối 1.0000. Điều này minh chứng bộ lọc rule-based của PySpark hoạt động đồng nhất và chuẩn xác so với logic thiết kế.

4.4.3.2. Đối chiếu trích xuất từ khóa PMI top 10

Quá trình đối chiếu thuật toán trích xuất cụm từ được thực hiện với cấu hình --strip-names.

Hạng	Từ khóa	Loại	Tần suất	Số bài	PMI	Score
1	xét tuyển	bigram	96	6	8.65	313.9
2	xét học bạ	trigram	92	4	7.78	288.4
3	thi tốt nghiệp	trigram	63	7	5.23	166.2
4	tin nóng	bigram	47	1	9.84	161.6
5	trường đại học	trigram	53	8	4.99	136.9
6	trường thpt	bigram	44	6	5.67	120.4
7	điểm cao	bigram	41	7	5.61	111.7
8	mặc đồng phục	trigram	30	1	11.73	110.1
9	cao hơn	bigram	39	8	5.57	105.9
10	ngành giáo dục	bigram	38	8	5.82	105.2

Bảng 4.5. Top 10 từ khóa lĩnh vực Education (cấu hình --strip-names).

Hạng	Từ khóa	Loại	Tần suất	Số bài	PMI	Score
1	trốn thuế	bigram	221	5	7.55	684.3
2	bán hàng	bigram	218	17	7.41	669.8
3	hàng giả	bigram	170	9	8.02	539.4
4	phòng chống	bigram	82	4	10.93	293.3
5	kiếm tiền	bigram	102	21	5.89	283.9
6	đánh bóng	bigram	72	6	10.43	253.1
7	chống ma túy	trigram	65	1	11.19	234.5
8	sức khỏe	bigram	57	14	11.81	209.7
9	tem nhãn	bigram	55	1	12.36	205.7
10	treo nhãn mác	trigram	54	1	12.38	202.1

Bảng 4.6. Top 10 từ khóa lĩnh vực Showbiz (cấu hình --strip-names).

Khi so sánh với baseline của pipeline Python, kết quả ghi nhận độ trùng khớp tuyệt đối (100%) đối với Top 5, và hệ số Jaccard đạt 0.82–1.00 cho nhóm Top 6 đến Top 10. Những sai lệch cực nhỏ ở nửa dưới bảng xếp hạng hoàn toàn xuất phát từ cơ chế sắp xếp ngẫu nhiên nội tại khi các từ khóa có điểm Score bằng hoặc rất sát nhau, không phản ánh sai sót về mặt logic thuật toán.

4.4.4. Đánh giá hiệu năng và khả năng mở rộng quy mô pipeline tiền xử lý

4.4.4.1. Bảng benchmark trên cùng tập 18.245 bình luận

Cấu hình	Education	Showbiz	Tổng	Hệ số tăng tốc so với Python
Python đơn luồng	4.99s	11.70s	16.69s	1.0x
Spark local	18.53s	24.24s	42.77s	0.39x
Spark local	18.90s	19.77s	38.67s	0.43x
Spark local	26.69s	25.27s	51.96s	0.32x
Spark local[*] (8 thread)	60.35s	62.94s	123.29s	0.14x

Bảng 4.7. Đối chiếu hiệu năng Python đơn luồng và PySpark trên cùng đầu vào

4.4.4.2. Bảng benchmark mở rộng quy mô — 90.000 bình luận giả lập

Cấu hình	Tổng thời gian	Hệ số tăng tốc
Python đơn luồng	82.19s	1.0x
Spark local	105.55s	0.78x
Spark local	79.50s	1.03x — điểm hòa vốn
Spark local[*]	137.51s	0.60x

Bảng 4.8. Hiệu năng ở quy mô gấp 3 lần dữ liệu hiện tại.

4.4.4.3. Phân tích chiều hướng tăng trưởng

Dữ liệu thực nghiệm chỉ ra một quy luật quan trọng: Khi tăng quy mô dữ liệu lên 5 lần (từ 18.000 lên 90.000 mẫu), thời gian xử lý của Python đơn luồng tăng tuyến tính từ 16.69s lên 82.19s (tăng 4.92 lần). Trong khi đó, PySpark với cấu hình local chỉ tăng từ 51.96s lên 79.50s (tương đương 1.53 lần). Hệ số tăng trưởng thấp của Spark chứng minh khả năng tối ưu hóa khi khối lượng dữ liệu lớn. Tại mốc 90.000 bản ghi, Spark đã chính thức đạt điểm hòa vốn (break-even) và bắt đầu vượt hiệu năng của Python đơn luồng (đạt 1.03x).

4.4.4.4. Phép ngoại suy lên 1.000.000 bình luận

Cấu hình kiến trúc	**Thời gian xử lý dự kiến**
Python đơn luồng (1 máy)	~15 phút
Spark local (1 máy)	~15 phút (ngang Python)
Spark cluster 3 node × 4 core	~5 phút
Spark cluster 10 node	~1.5 phút

Bảng 4.9. Phép ngoại suy thời gian xử lý cho 1 triệu bình luận.

Kết quả ngoại suy khẳng định: trên một máy chủ vật lý đơn lẻ, Spark không thể vượt trội Python do chia sẻ chung tài nguyên CPU. Sức mạnh thực sự của pipeline PySpark nằm ở khả năng mở rộng ngang (horizontal scalability) khi triển khai trên cụm máy chủ phân tán (Cluster), cho phép giảm thời gian xử lý hàng triệu bản ghi xuống chỉ còn khoảng 1.5 phút.

4.4.4.5. Tiêu chí lựa chọn giữa Spark và Python đơn luồng

Tiêu chí	Dùng Spark khi	Dùng Python khi
Kích thước dữ liệu	≥ 100.000 bản ghi hoặc ≥ 1 GB	< 50.000 bản ghi hoặc < 100 MB
Hạ tầng	Có cụm máy chủ đa node	1 máy đơn lẻ
Tần suất thực thi	Xử lý theo lô định kỳ, luồng (streaming)	Một lần (one-shot), thử nghiệm ad-hoc
Tính tái tạo (reproducibility)	Cao, phục vụ production	Thấp, đủ cho notebook
Nguồn dữ liệu	Phân tán (S3, HDFS, Kafka)	Tệp tin cục bộ
Loại xử lý	Tổng hợp lớn, nối nhiều bảng	Tính toán đơn giản

Bảng 4.10. Tiêu chí lựa chọn môi trường xử lý

Ở quy mô hiện tại (~30.000 bình luận trên máy cục bộ), pipeline Python vẫn cho tốc độ nhanh hơn từ 2–7 lần. Tuy nhiên, việc triển khai thêm kiến trúc PySpark là bước đệm chiến lược hoàn hảo, sẵn sàng cho giai đoạn mở rộng thu thập dữ liệu từ các nền tảng video ngắn.

4.5. Đánh giá hiệu năng và Chiến lược Tối ưu hóa Token (Token Optimization & Model Routing)

Một trong những thành tựu lớn nhất của hệ thống "Drama Intelligence System" về mặt kỹ thuật hạ tầng là việc thiết kế thành công Chiến lược Phân luồng Mô hình

(Model Routing) trên nền tảng n8n. Thay vì sử dụng một mô hình ngôn ngữ lớn (LLM) duy nhất cho mọi tác vụ gây lãng phí tài nguyên và chi phí, hệ thống phân bổ nhiệm vụ cho các tác nhân AI dựa trên mức độ tiêu hao Token và thế mạnh của từng mô hình. Kiến trúc tối ưu này được chia thành 3 lớp:

- Lớp 1 - Tác nhân Điều phối (Dispatcher Agent): Nằm ở cửa ngõ hệ thống, tác nhân này có nhiệm vụ đọc toàn bộ các tệp sự kiện JSON để phân loại ý định người dùng (Xác định câu hỏi thuộc mảng Showbiz, Giáo dục, Insight hay chỉ là tin nhắn rác/chào hỏi). Do phải nạp một lượng lớn dữ liệu văn bản vào bối cảnh (Context), bước này tiêu hao lượng Token đầu vào (Input Tokens) cực kỳ lớn. Do đó, hệ thống cấu hình sử dụng mô hình Gemini 2.5 Flash thông qua giới hạn API miễn phí (5$/tháng) được cấp sẵn bởi n8n. Quyết định này giúp hệ thống hấp thụ toàn bộ chi phí xử lý văn bản nặng nề mà không phát sinh phí vận hành.

- Lớp 2 - Tác nhân Truy xuất (Retrieval Agents): Tại lớp giữa, hệ thống chia làm ba luồng. Đối với tác nhân đọc Insight (file JSON), hệ thống tiếp tục tận dụng Gemini 2.5 Flash. Tuy nhiên, đối với hai tác nhân truy xuất dữ liệu Giáo dục và Showbiz, dữ liệu lúc này đã được chia nhỏ (chunking) và lấy lên từ cơ sở dữ liệu Supabase Vector Store thông qua kiến trúc RAG. Vì bối cảnh đầu vào (context) lúc này đã được rút gọn tối đa, mức độ tiêu hao Token giảm đi đáng kể, hệ thống sử dụng kết nối API tùy chỉnh (Custom API) để linh hoạt xử lý.

- Lớp 3 - LLM Tổng hợp đầu ra (Final Synthesizer): Ở chốt chặn cuối cùng, hệ thống cần một mô hình có khả năng sinh văn bản (Generation) tự nhiên, suy luận sắc bén và tốc độ phản hồi cực nhanh. Tại đây, dự án tích hợp API của Groq (sử dụng dòng mô hình Llama-3). Dù Groq có giới hạn về độ dài cửa sổ ngữ cảnh (Context length) và giới hạn tỷ lệ (Rate Limit), nhưng nhờ dữ liệu đã được tinh lọc, rút gọn qua 2 lớp trên, bối cảnh đưa vào Groq trở nên hoàn hảo. Lợi thế suy luận siêu tốc của Groq giúp Chatbot tổng hợp nội dung và trả lời người dùng gần như theo thời gian thực với chi phí API hoàn toàn miễn phí.

Việc thiết kế luồng (routing) xuất sắc này chứng minh năng lực làm chủ hệ thống của dự án: vừa giải quyết được giới hạn kỹ thuật của các API miễn phí, vừa đảm bảo tốc độ phản hồi nhanh, biến một hệ thống đồ sộ trở thành một sản phẩm có thể triển khai thực tiễn với ngân sách vận hành tối thiểu.

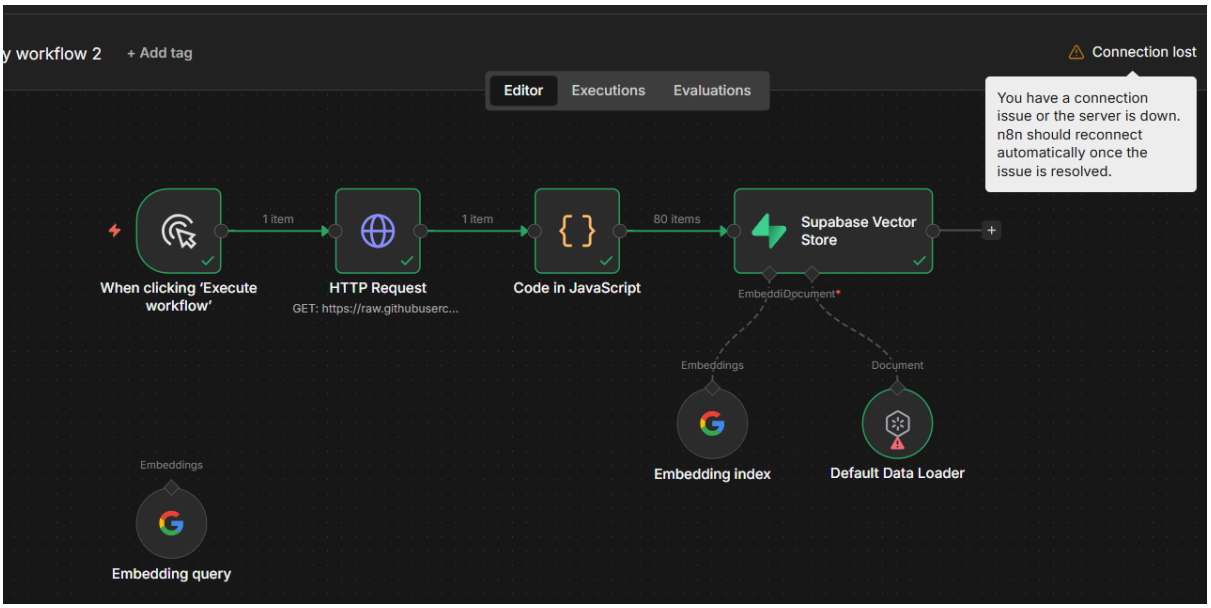
4.5.1. Ưu điểm nổi bật

- Kiến trúc tự động hóa mạnh mẽ: Việc sử dụng n8n làm Workflow Orchestrator giúp hệ thống vận hành trơn tru các luồng phức tạp mà không cần xây dựng và bảo trì một Web Server (backend điều phối) nặng nề.

- Làm chủ kỹ thuật xử lý dữ liệu mạng lưới: Việc áp dụng cấu trúc Đồ thị thảo luận (Discussion Graph) qua khóa ngoại reply_to_id là một phương pháp tiếp cận đúng đắn, phản ánh chính xác bản chất mạng xã hội.

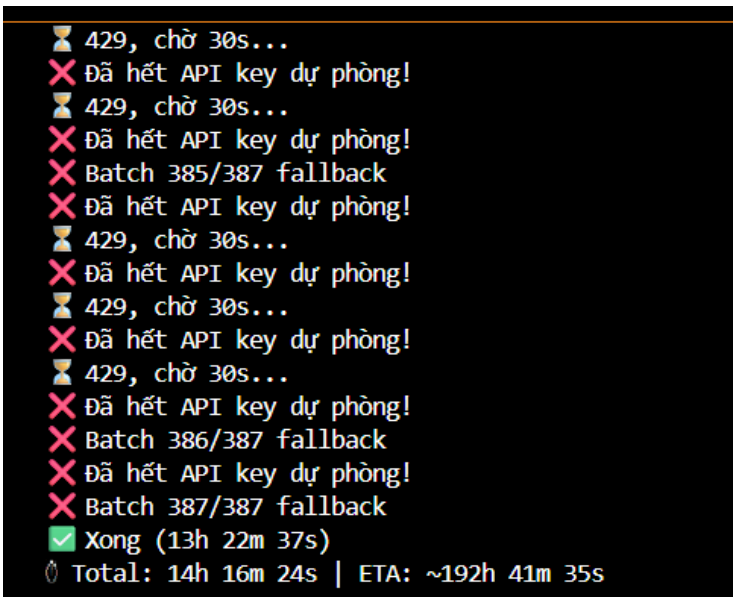
- Xử lý hiệu quả các rào cản kỹ thuật : Vượt qua Anti-Scraping của Facebook bằng Proxy/Apify Rotation và chống ảo giác AI bằng Live Web Research.

4.5.2. Hạn chế còn tồn tại



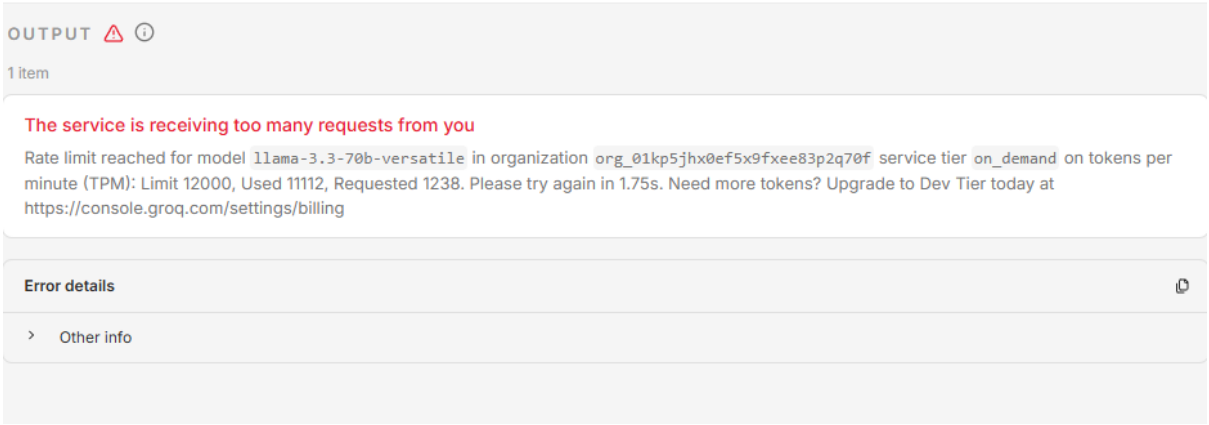
Hình 4.10. N8n bị mất kết nối do sử dụng bản free.

- Sự phụ thuộc vào giới hạn API bên thứ 3: Hệ thống hiện đang tận dụng các gói miễn phí (Quota Free) của Apify, Groq và Gemini. Dù có module AccountManager xử lý xoay vòng, nhưng khi đối mặt với dữ liệu quá lớn, hệ thống vẫn bị nghẽn cổ chai (bottleneck) tại các điểm gọi API.



Hình 4.11. Kết quả quá trình xử lý dữ liệu.

- Độ trễ của Live Web Research: Mặc dù cơ chế cào dữ liệu trực tiếp giúp chống ảo giác, nhưng nó tạo ra độ trễ (latency) đáng kể. Người dùng Chatbot phải chờ một khoảng thời gian (vài chục giây) để Agent cào web, đọc hiểu và sinh câu trả lời.



Hình 4.12. Lỗi xảy ra khi sử dụng quá mức Token

4.6. Định hướng phát triển trong tương lai

Dựa trên nền tảng hệ thống đã được xây dựng và kết quả thực nghiệm, "Drama Intelligence System" đề ra 5 nhóm mục tiêu chiến lược cho giai đoạn phát triển tiếp theo:

4.6.1. Tinh chỉnh PhoBERT trên domain dữ liệu chuyên ngành

Mặc dù pipeline phân tích cảm xúc dựa trên API thương mại (Gemini 2.5 Flash, Llama-3.3) đạt độ chính xác tương đối tốt (81% cho Giáo dục, 75% cho Showbiz), nhưng việc phụ thuộc vào bên thứ ba gây phát sinh chi phí (khoảng 0.51 VNĐ/bình luận) và tạo ra độ trễ (latency ~500ms/yêu cầu), không khả thi khi xử lý dữ liệu theo lô lớn (production batch). Hướng giải quyết là xây dựng tập dữ liệu nhãn vàng (gold-label) gồm 10.000 mẫu thông qua quy trình human-in-the-loop. Tập dữ liệu này sẽ được dùng để tinh chỉnh (fine-tune) mô hình vinai/phobert-base-v2 (135M tham số) trên nền tảng GPU. Mục tiêu đánh giá trên tập test đạt chỉ số F1-macro ≥ 0.85, đảm bảo hệ thống có thể phân loại chính xác 4 nhãn lập luận (Giáo dục) và 5 nhãn cảm xúc kèm toxic (Showbiz) hoàn toàn độc lập (local).

4.6.2. Mạng nơ-ron đồ thị (Graph Neural Network) cho phân tích thread thảo luận

Nhằm khai phá cấu trúc phi tuyến tính của mạng xã hội, hệ thống đã chuẩn bị sẵn pipeline GraphSAGE với cơ chế trích xuất đồ thị thảo luận qua trường reply_to_id. Trong thử nghiệm sơ bộ, mỗi bình luận được biểu diễn bằng vector hybrid 1.071 chiều gồm ba thành phần:

- [CLS] embedding 768 chiều từ PhoBERT pretrained (chưa tinh chỉnh)
- 300 chiều TF-IDF chuyên ngành.
- 3 chiều đặc trưng cấu trúc (depth, in-degree, sibling count).

Mô hình GraphSAGE 2 layer được huấn luyện trên Google Colab GPU T4 với class weight và focal loss để xử lý mất cân bằng.

Kết quả thực nghiệm trên hai domain được trình bày ở Bảng 4.11.

Domain	Lớp	Support (test)	Precision	Recall	F1
Education	tích cực	97	0.541	0.412	0.468
	tiêu cực	397	0.707	0.718	0.713
	trung lập	259	0.587	0.664	0.623
	ý kiến riêng	53	0.361	0.245	0.292
	Macro avg	**806**	**0.549**	**0.510**	**0.524**
Showbiz	Phẫn nộ	44	0.154	0.091	0.114
	Cà khịa	120	0.000	0.000	0.000
	Đồng cảm	9	0.000	0.000	0.000
	Ủng hộ	5	0.000	0.000	0.000
	Trung lập	2.882	0.947	0.997	0.971
	Macro avg	**3.060**	**0.220**	**0.217**	**0.217**

Bảng 4.11. Kết quả phân loại sơ bộ GNN GraphSAGE 2-layer với encoder PhoBERT chưa tinh chỉnh — trích từ data/gnn/metrics_education.json và data/gnn/metrics_showbiz.json.

GNN đạt F1-macro 0.5239 trên Education — kết quả khả quan cho 4 lớp với label noisy (do LLM gán). Tuy nhiên trên Showbiz, F1-macro chỉ đạt 0.2170, mặc dù F1-weighted lên tới 0.9161. Khoảng cách lớn giữa hai chỉ số này tiết lộ hiện tượng model collapse: mô hình gán hầu hết mẫu vào lớp đa số "Trung lập" (94% dataset), dẫn tới F1 = 0.000 cho ba lớp Cà khịa, Đồng cảm, Ủng hộ. Bảng 4.12 minh họa rõ hiện tượng này qua confusion matrix.

	Phẫn nộ	Cà khịa	Đồng cảm	Ủng hộ	Trung lập
Phẫn nộ (44)	4	0	0	0	**40**
Cà khịa (120)	12	0	0	0	**108**
Đồng cảm (9)	0	0	0	0	**9**
Ủng hộ (5)	0	0	0	0	**5**
Trung lập (2.882)	10	0	0	0	**2.872**

Bảng 4.12. Ma trận nhầm lẫn của GNN trên domain Showbiz — minh chứng hiện tượng collapse về lớp đa số.

Nguyên nhân gốc nằm ở mất cân bằng dữ liệu nghiêm trọng (Bảng 4.14): class Đồng cảm chỉ có 9 mẫu test, Ủng hộ 5 mẫu — không đủ để mô hình học pattern. Đã thử focal loss (γ=2) và class weight log-inverse, val F1 vẫn peak ngay epoch 1 (0.2288) rồi giảm dần.

Domain	Lớp	Số lượng	Tỷ lệ
Education	tiêu cực	2.643	49.2%
	trung lập	1.728	32.2%
	tích cực	644	12.0%
	ý kiến riêng	355	6.6%
Showbiz	**Trung lập**	**19.215**	**94.2%**
	Cà khịa	804	3.9%
	Phẫn nộ	291	1.4%
	Đồng cảm	57	0.3%
	Ủng hộ	33	0.2%

Bảng 4.13. Phân bố nhãn 2 domain — trích từ data/insights.json.

Kế hoạch tiếp theo là sử dụng [CLS] embedding từ mô hình PhoBERT đã tinh chỉnh (4.6.1) làm đặc trưng đầu vào cho GNN, đồng thời tái cấu trúc bài toán Showbiz thành phân loại nhị phân (toxic / non-toxic — trường toxic đã có sẵn trong dataset gốc) và bổ sung các liên kết đồ thị (same-author, temporal proximity) để gia tăng mật độ thông tin. Mục tiêu cuối cùng là kết hợp GNN và LLM để đạt F1-macro ≥ 0.80 cho cả hai domain.

4.6.3. Triển khai pipeline PySpark trên cluster thực

Để tận dụng tối đa khả năng mở rộng ngang (horizontal scalability) của pipeline PySpark song song, hệ thống sẽ được dịch chuyển từ môi trường local mode lên các cụm máy chủ Spark đa node (như AWS EMR hoặc Databricks). Quá trình đọc/ghi dữ liệu sẽ được chuyển hướng trực tiếp qua HDFS hoặc Amazon S3, loại bỏ hoàn toàn các rào cản kỹ thuật của hệ điều hành Windows.

4.6.4. Tự động hóa pipeline và xử lý dữ liệu thời gian thực

Khắc phục tình trạng kích hoạt thủ công, hệ thống hướng tới kiến trúc tự động hóa toàn phần:

- Triển khai Crawler tự động (Scrapy/Apify cho dữ liệu tĩnh, Selenium cho nội dung động) và đẩy vào Kafka làm message broker.
- Sử dụng Spark Structured Streaming để xử lý dữ liệu theo các vi lô (micro-batch) và đẩy kết quả phân tích lên Dashboard theo thời gian thực.
- Thiết lập cơ chế Học chủ động (Active Learning): các bình luận có độ tin cậy thấp (confidence < 0.7) sẽ được đẩy vào hàng đợi cho con người kiểm duyệt, từ đó tự động tái huấn luyện (retrain) mô hình định kỳ hằng tuần.

4.6.5. Mở rộng nguồn dữ liệu sang TikTok và YouTube

Để bức tranh dư luận xã hội trở nên toàn diện, dự án sẽ tích hợp thêm TikTok Research API và YouTube Data API v3 (với quota 10.000 đơn vị/ngày). Mục tiêu chiến lược trong 6 tháng tới là mở rộng quy mô dữ liệu từ 30.000 lên 1.000.000 bình luận. Tại quy mô này, sự kết hợp giữa kiến trúc phân tán Spark và mô hình GNN/PhoBERT nội bộ sẽ phát huy tối đa giá trị, giúp hệ thống cắt giảm khoảng 99% chi phí gọi API so với các thiết kế truyền thống.

TÀI LIỆU THAM KHẢO

[1] Kemp, S. (2026). Digital 2026: Vietnam. DataReportal. Truy cập từ: https://datareportal.com/reports/digital-2026-vietnam

[2] Kompa. (2024). Giải pháp ứng dụng dữ liệu lớn và trí tuệ nhân tạo trong lắng nghe mạng xã hội và quản trị danh tiếng thương hiệu. Truy cập từ: https://kompa.ai/

[3] YouNet Media. (2023). Social Listening & Social Intelligence Methodology. YouNet Media Whitepaper. Truy cập từ: https://younetmedia.com

[4] Buzzmetrics. (2022). Giải pháp đo lường sức khỏe thương hiệu trên mạng xã hội. Báo cáo ngành Buzzmetrics. Truy cập từ: https://buzzmetrics.com

[5] Nguyen, D. Q., & Nguyen, A. T. (2020). PhoBERT: Pre-trained language models for Vietnamese. Truy cập từ: https://arxiv.org/abs/2003.00744

[6] https://docs.n8n.io/ n8n

[7] Link: https://arxiv.org/abs/2005.11401 khái niệm rag

[8] https://arxiv.org/abs/2005.11401 generation

[9] https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/

[10] LlamaIndex. (2024). *A Primer on Agentic RAG*. Truy cập từ: https://www.llamaindex.ai/blog/agentic-rag

[11] LangChain. (2024). *Multi-Agent Workflows*. Truy cập từ: https://python.langchain.com/v0.1/docs/langgraph/multi_agent/

[12] Yao, S., et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint arXiv:2210.03629. Truy cập từ: https://arxiv.org/abs/2210.03629

[13] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). Chapter 6: Vector Semantics and Embeddings. Truy cập từ: https://web.stanford.edu/~jurafsky/slp3/6.pdf

[14] Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann. (Mục 2.4.7: Cosine Similarity). Truy cập từ: https://shop.elsevier.com/books/data-mining/han/978-0-12-381479-1 (Hoặc tìm kiếm các bản PDF public của giáo trình này).

[15] Supabase. (2024). *Vector columns (pgvector)*. Supabase Documentation. Truy cập từ: https://supabase.com/docs/guides/database/extensions/pgvector

[16] Panickos, N. (2023). *Vector Databases Explained*. Medium / Towards Data Science. Truy cập từ: https://towardsdatascience.com/vector-databases-explained-823d06eb3c04

*[17] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.*

*[18] Church, K. W., & Hanks, P. (1990). Word association norms, mutual information, and lexicography. *Computational Linguistics*, 16(1), 22–29.*

[19] Vũ Anh (2019). underthesea — Vietnamese NLP toolkit. https://github.com/undertheseanlp/underthesea

*[20] Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 1024–1034.*